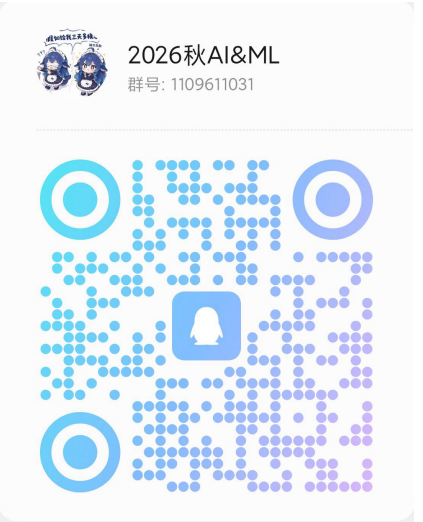
人工智能与机器学习基础

Lecture 07: 神经网络 II—反向传播

冯文杰

人工智能与数据科学学院

中国科学技术大学



提纲

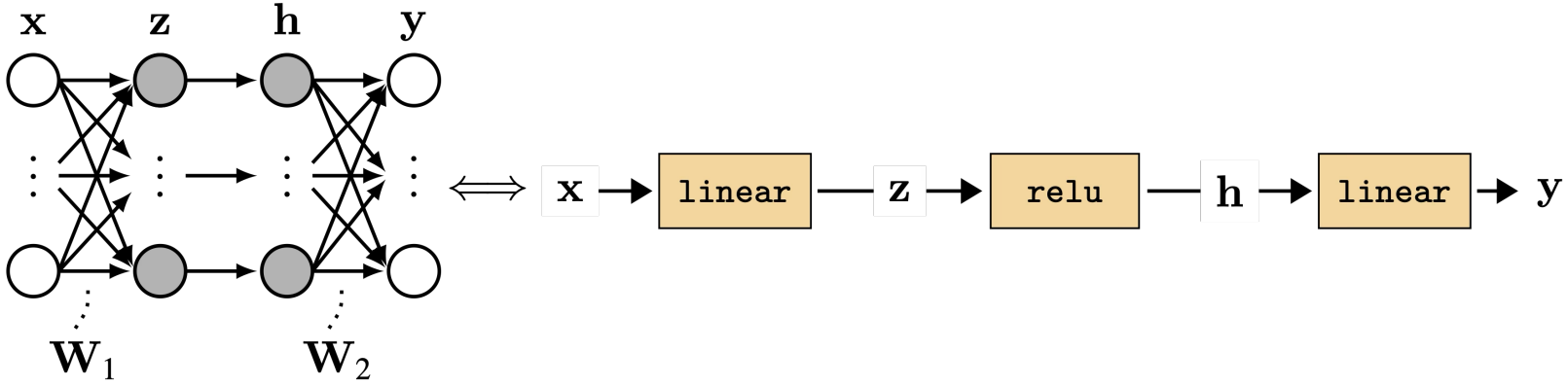
- 计算图与反向传播
- 向量与张量的反向传播
- 神经网络训练中的挑战
- 数据流形与模型泛化

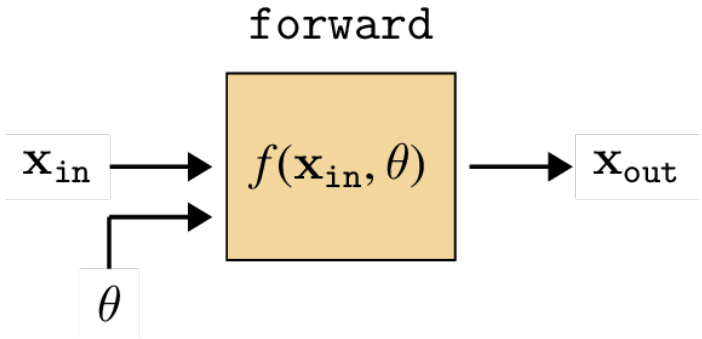
提纲

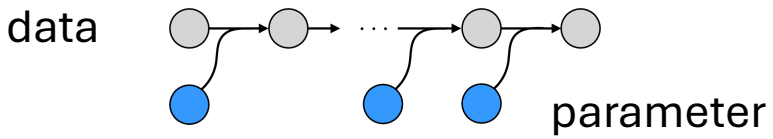
- **计算图与反向传播**
  - 标量计算图与链式法则
  - 常见运算节点的梯度规则
- 向量与张量的反向传播
- 神经网络训练中的挑战
- 数据流形与模型泛化

计算图与反向传播

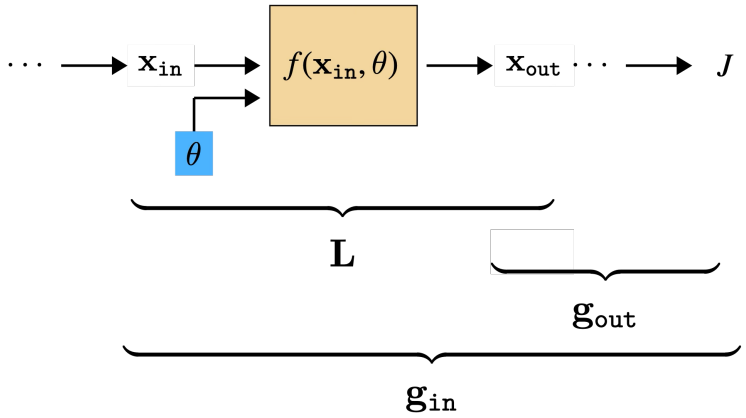
- 示例

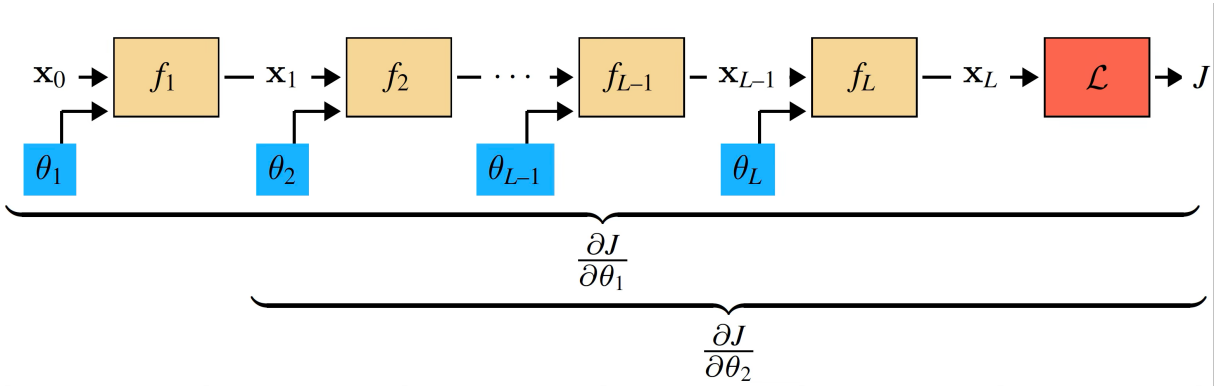

x
z
h
y
W_1
W_2
x
linear
z
relu
h
linear
y

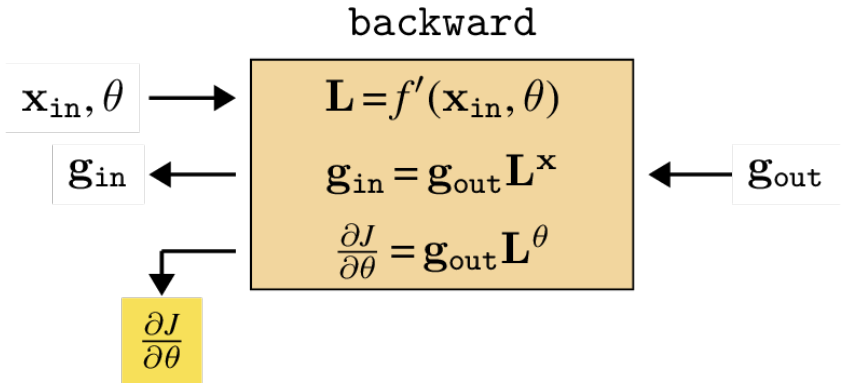

forward
x_{in}
$f(x_{in}, \theta)$
x_{out}
θ

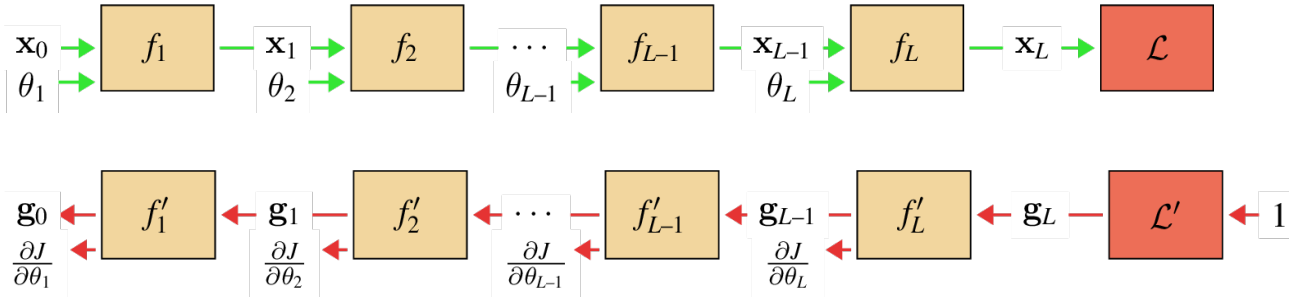

data
parameter

计算图与反向传播



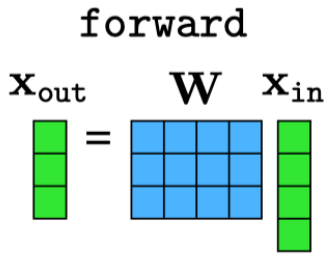


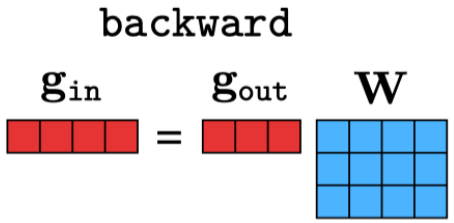


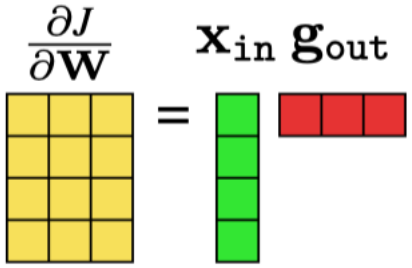


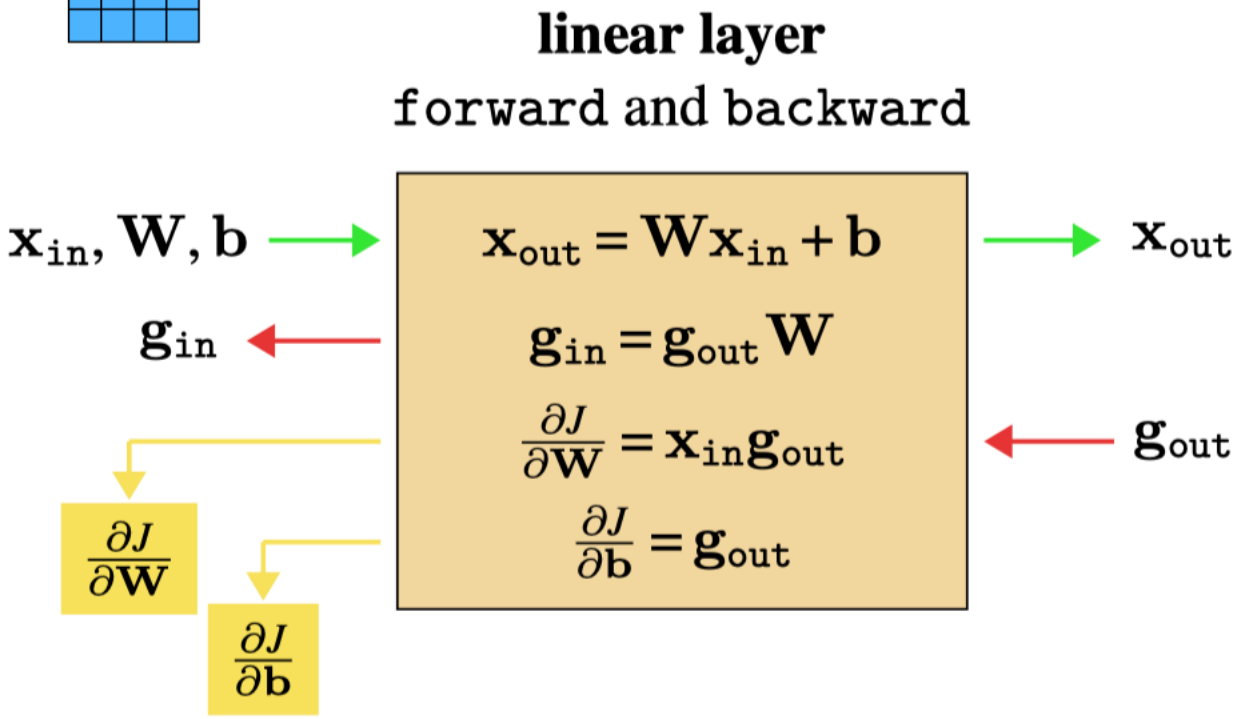
前向和反向传播

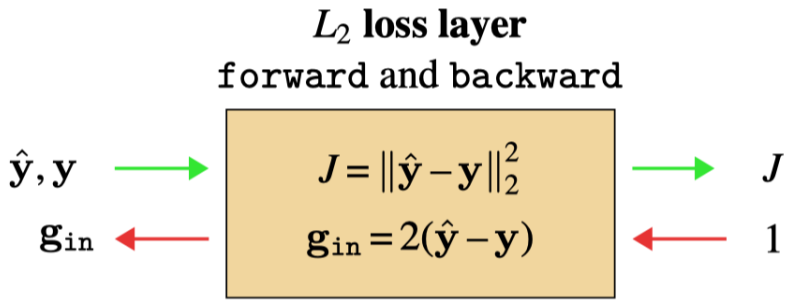
计算图与反向传播


forward
$\mathbf{x}_{out}$ $\mathbf{W}$ $\mathbf{x}_{in}$

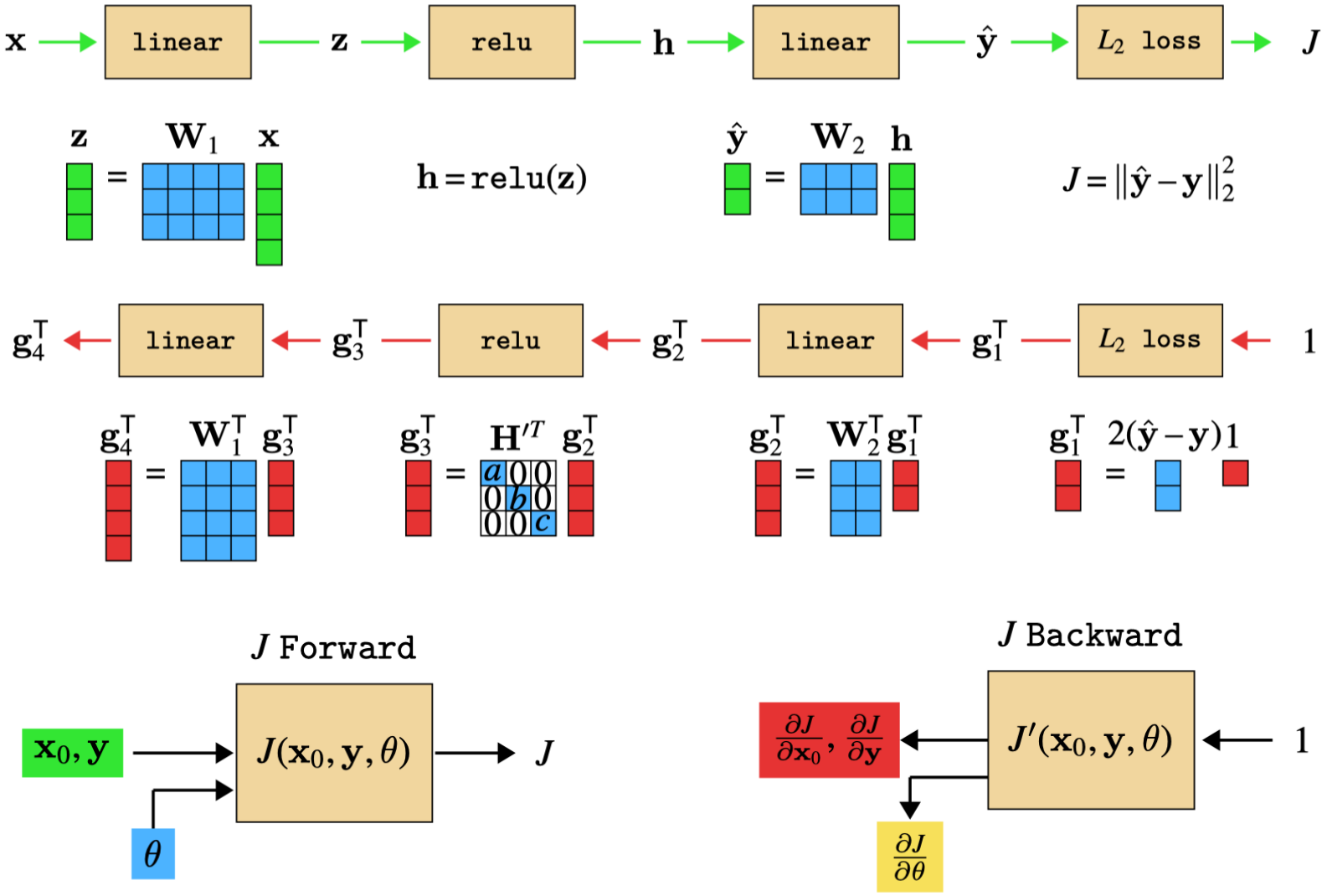

backward
$\mathbf{g}_{in}$ $\mathbf{g}_{out}$ $\mathbf{W}$


$\frac{\partial J}{\partial \mathbf{W}}$ $\mathbf{x}_{in}\,\mathbf{g}_{out}$

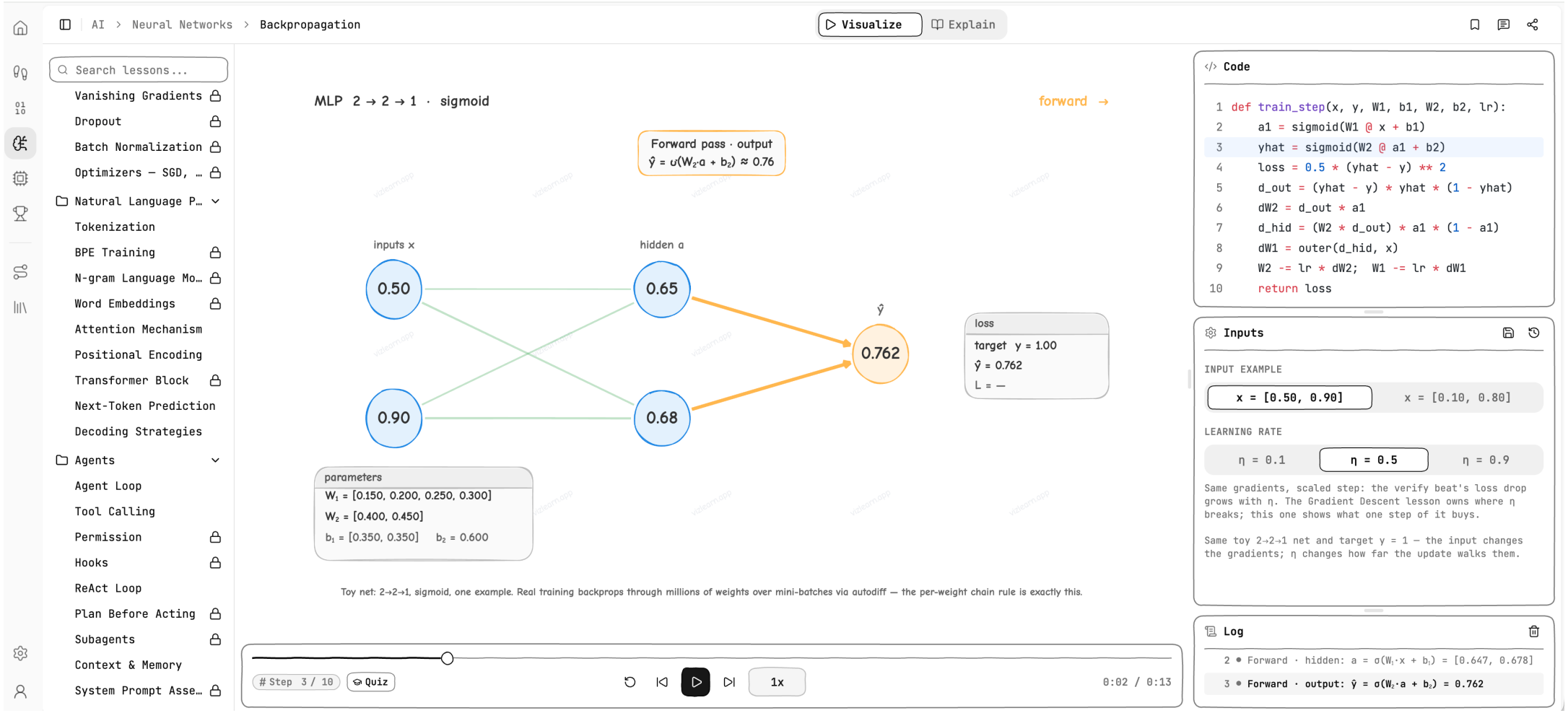

linear layer
forward and backward
$\mathbf{x}_{in}, \mathbf{W}, \mathbf{b}$
$\mathbf{g}_{in}$
$\mathbf{x}_{out} = \mathbf{W}\mathbf{x}_{in} + \mathbf{b}$
$\mathbf{g}_{in} = \mathbf{g}_{out}\mathbf{W}$
$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{x}_{in}\mathbf{g}_{out}$
$\frac{\partial J}{\partial \mathbf{b}} = \mathbf{g}_{out}$
$\mathbf{x}_{out}$
$\mathbf{g}_{out}$
$\frac{\partial J}{\partial \mathbf{W}}$
$\frac{\partial J}{\partial \mathbf{b}}$


L_2 loss layer
forward and backward
$\hat{\mathbf{y}}, \mathbf{y}$
$\mathbf{g}_{in}$
$J = \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$
$\mathbf{g}_{in} = 2(\hat{\mathbf{y}} - \mathbf{y})$
J
1

计算图与反向传播


x
linear
z
relu
h
linear
ŷ
L_2 loss
J
z = W1 x
h = relu(z)
ŷ = W2 h
$J = \|\hat{y} - y\|_2^2$
g_4^T
linear
g_3^T
relu
g_2^T
linear
g_1^T
L_2 loss
1
$g_4^T = W_1^T g_3^T$
$g_3^T = H'^T g_2^T$
$g_2^T = W_2^T g_1^T$
$g_1^T = 2(\hat{y} - y)1$
J Forward
x_0, y
$J(x_0, y, \theta)$
J
θ
J Backward
$\frac{\partial J}{\partial x_0}, \frac{\partial J}{\partial y}$
$J'(x_0, y, \theta)$
1
$\frac{\partial J}{\partial \theta}$

计算图与反向传播



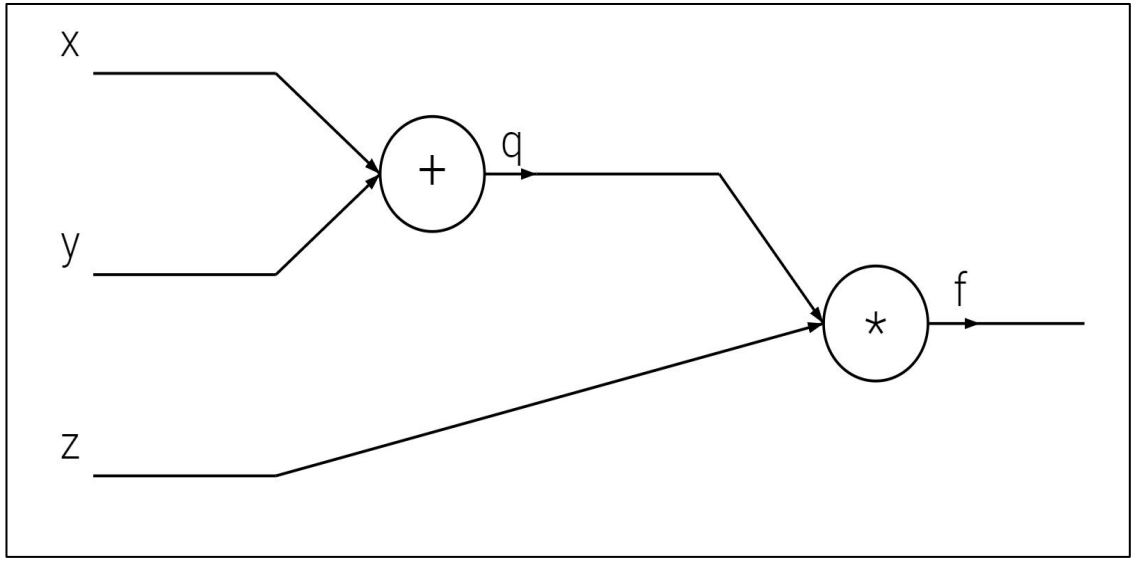
https://vizlearn.app/ai/neural-networks/backpropagation

反向传播：简单示例

- $f(x, y, z) = (x + y)z$

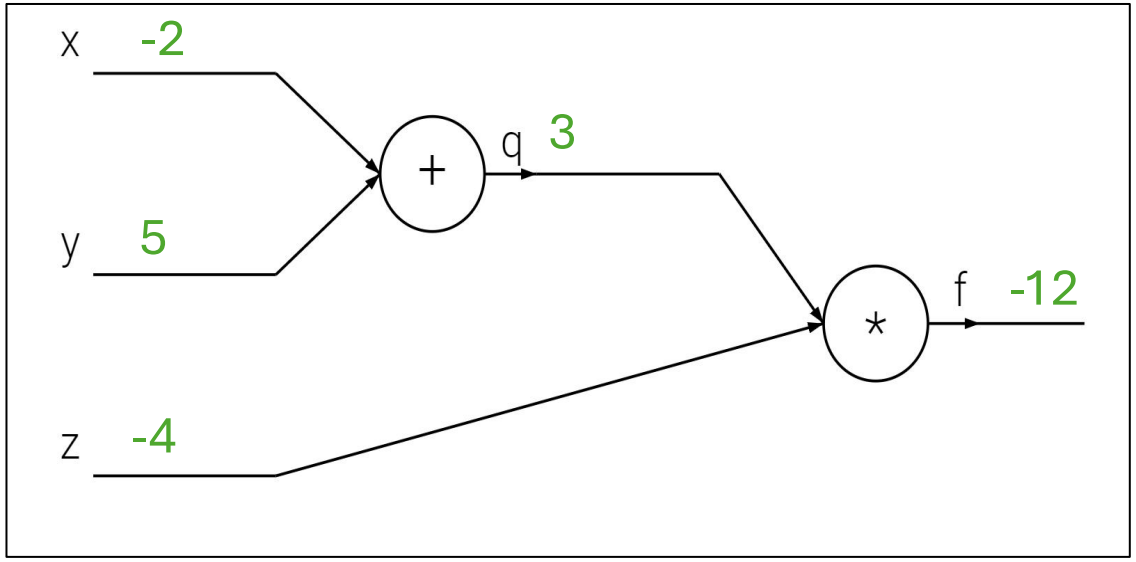
反向传播：简单示例

- $f(x, y, z) = (x + y)z$



反向传播：简单示例

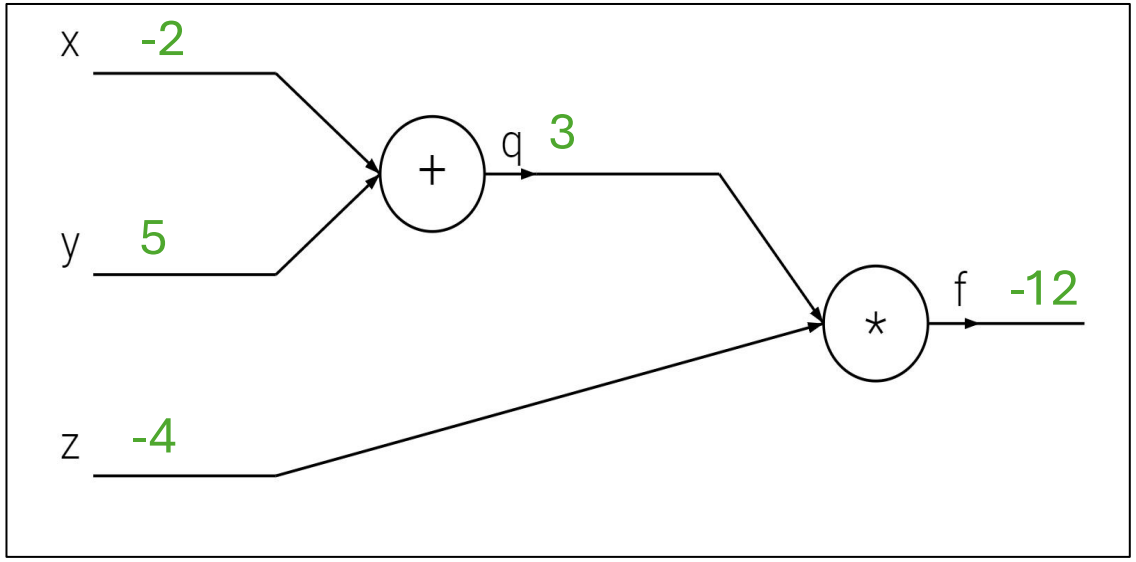
- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4



反向传播：简单示例

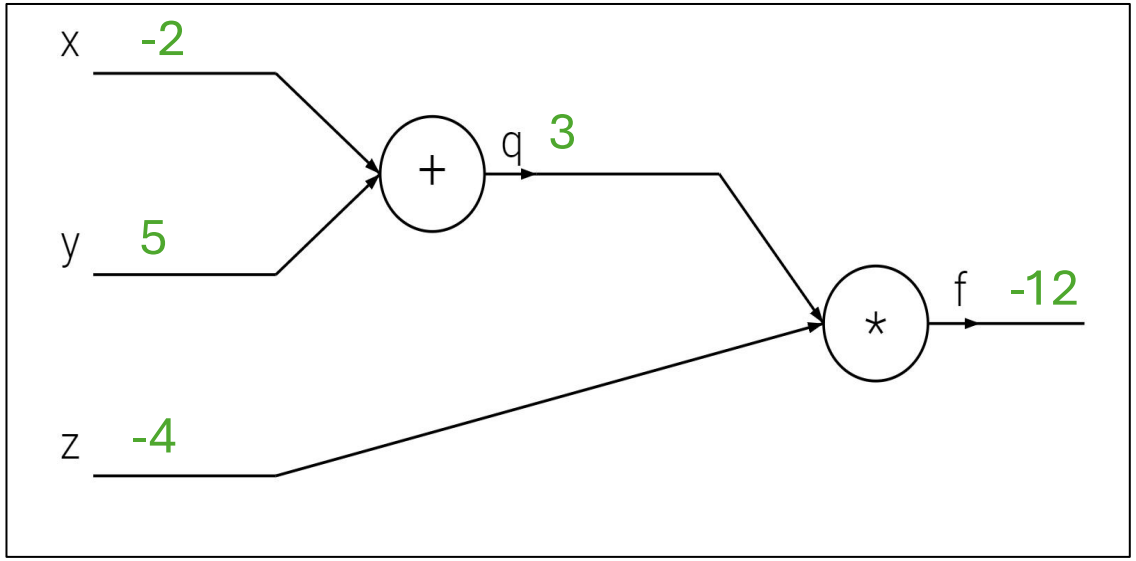
- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4

- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$



反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4

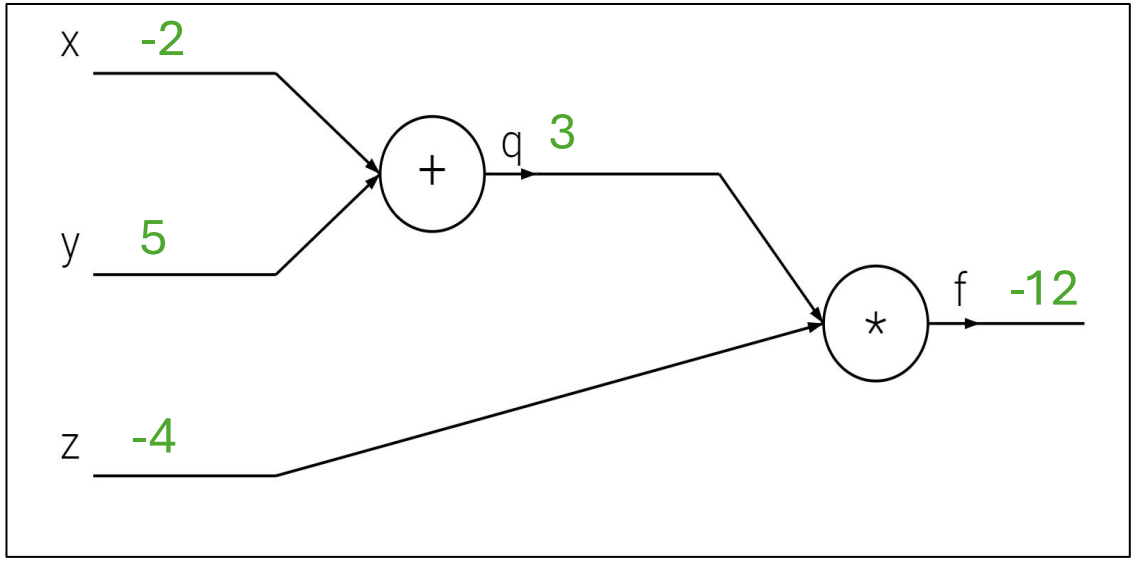


- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$

- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4

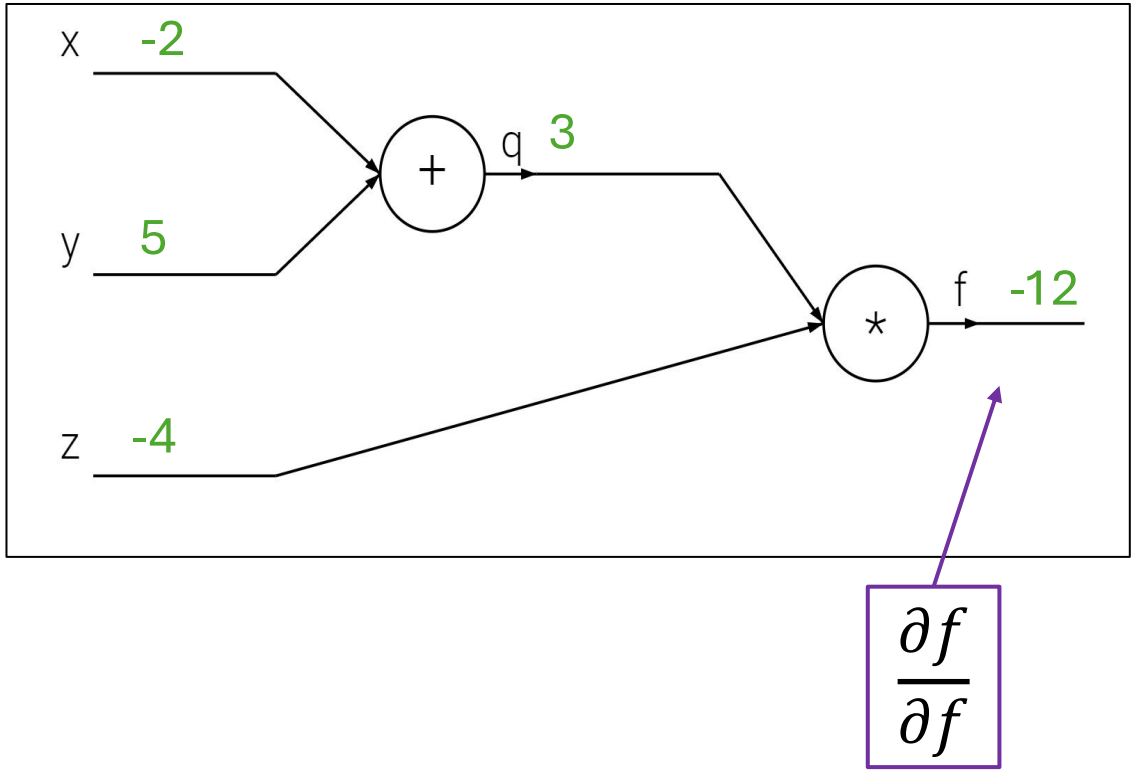


- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4

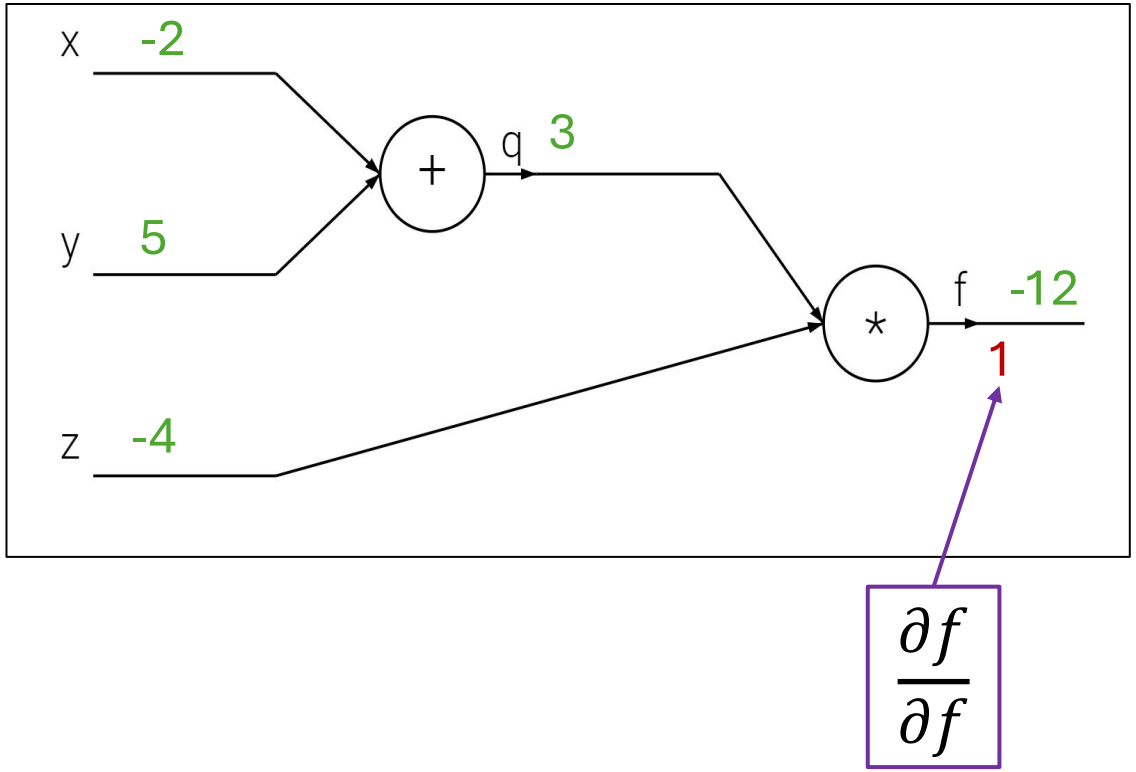


- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4

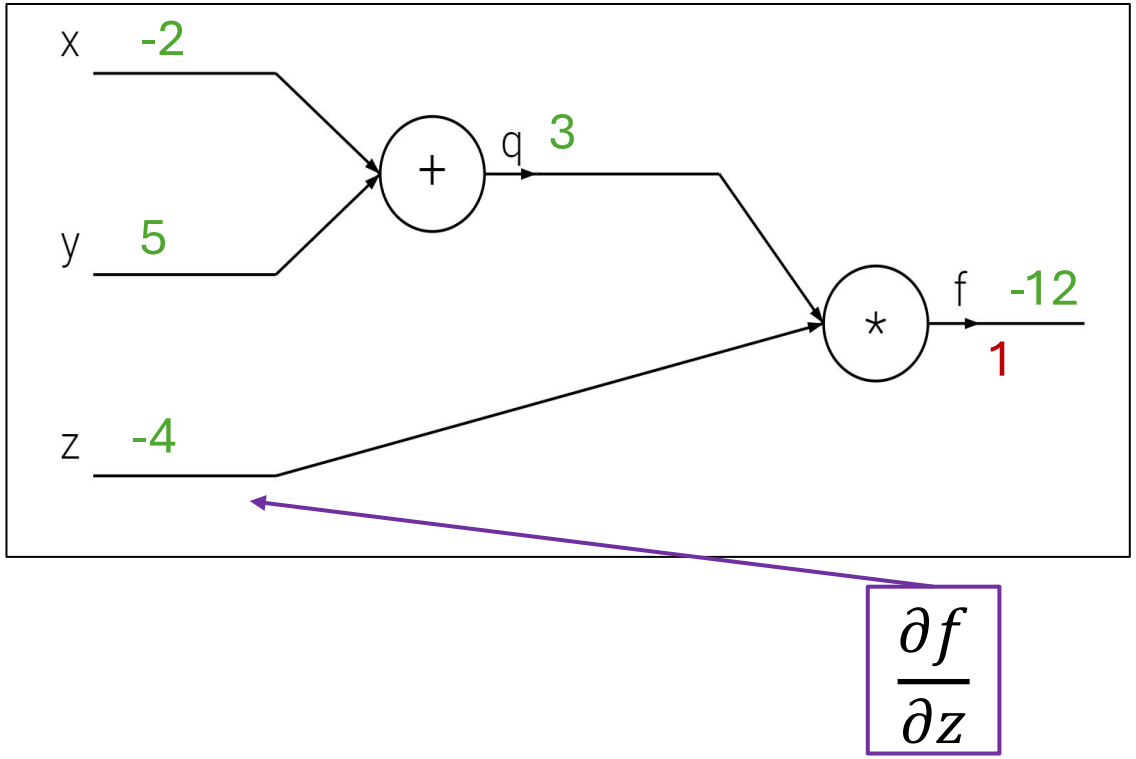


- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4

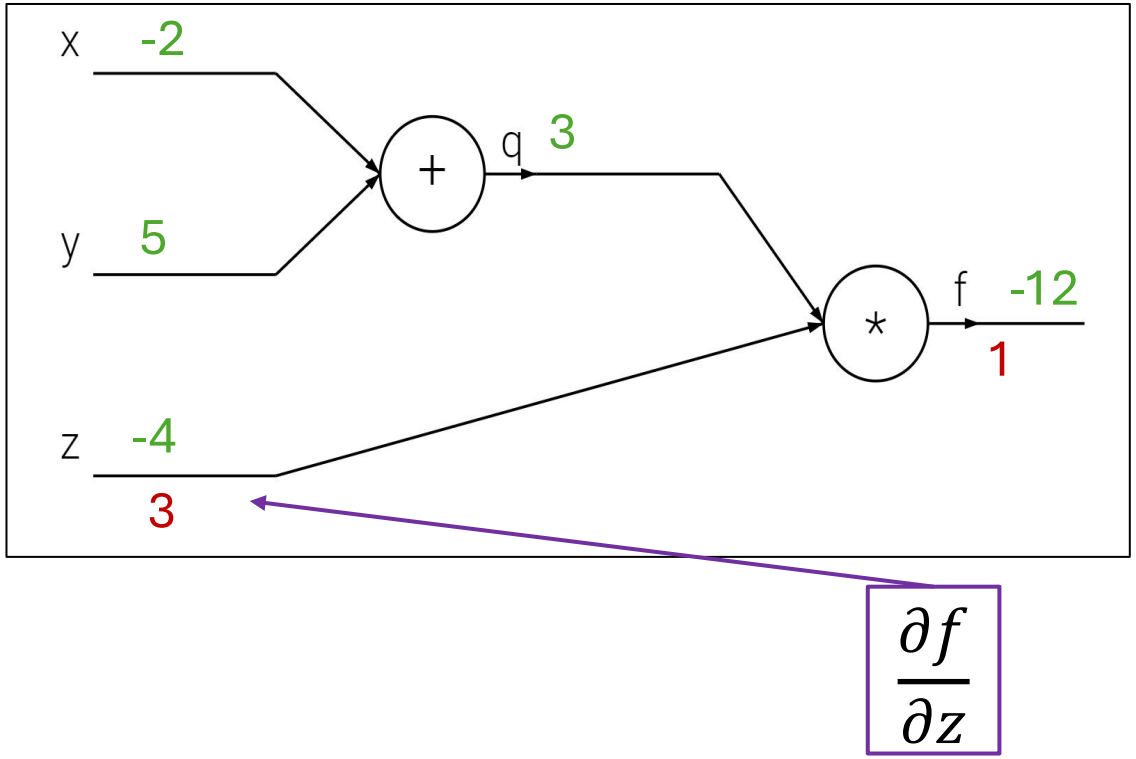


- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4

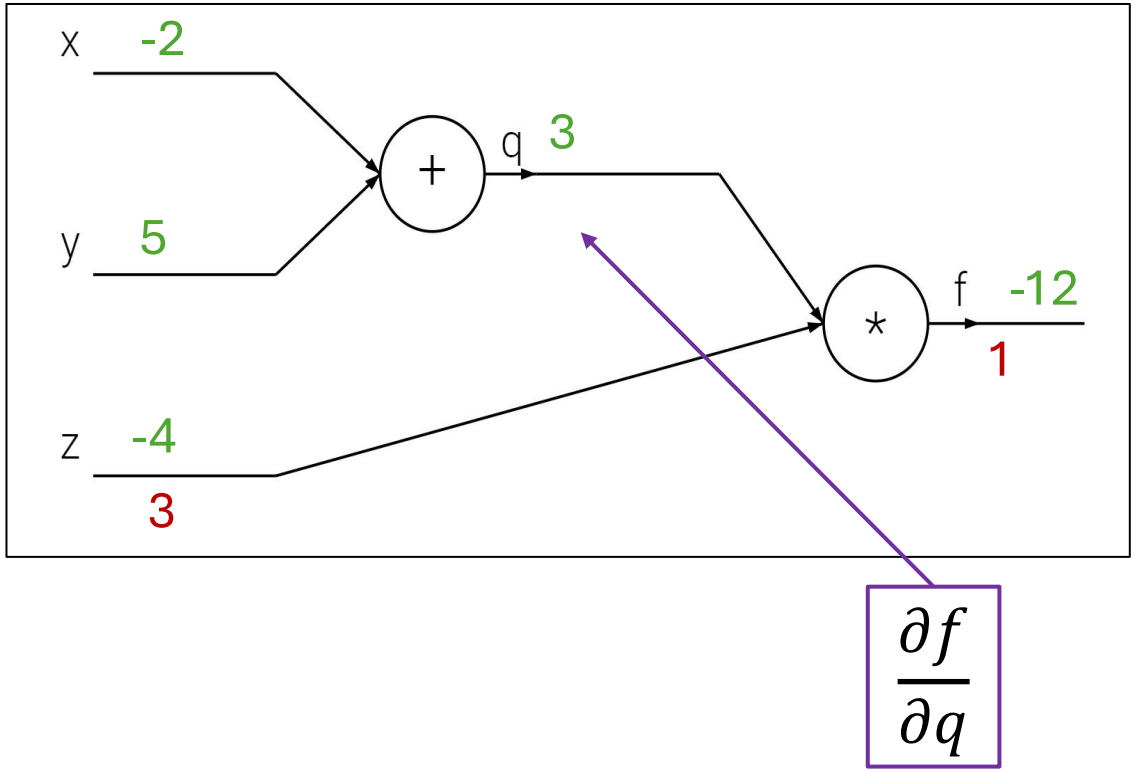


- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4

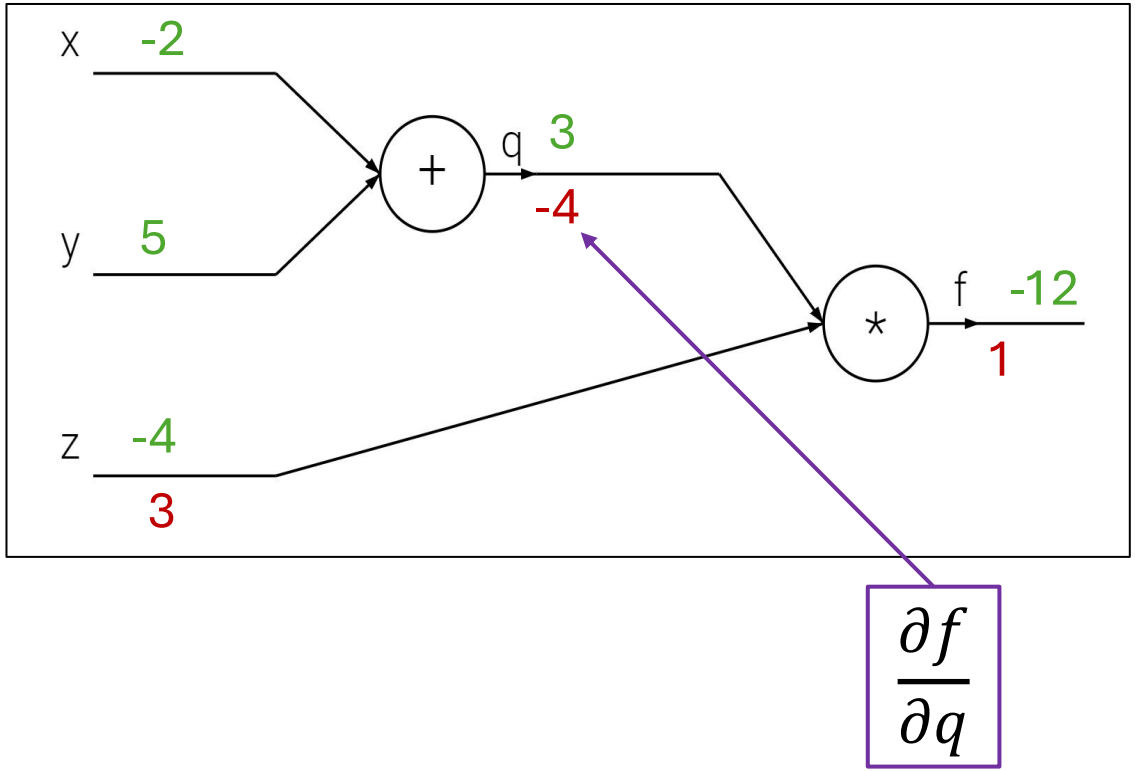


- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4

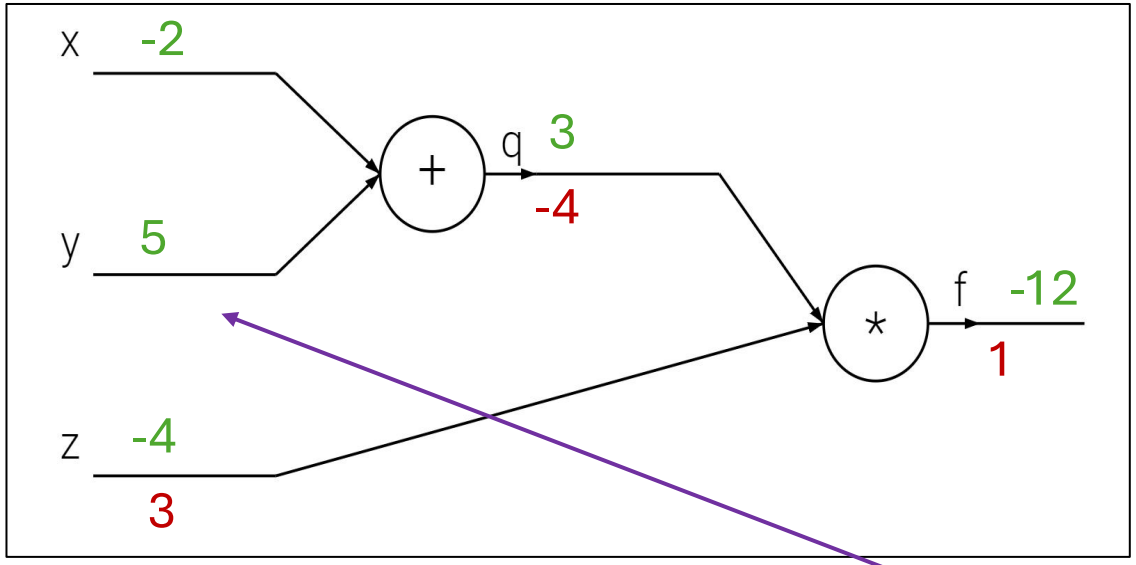


- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4



- $q = x + y$; $\frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz$; $\frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

$\frac{\partial f}{\partial y}$

Chain rule:

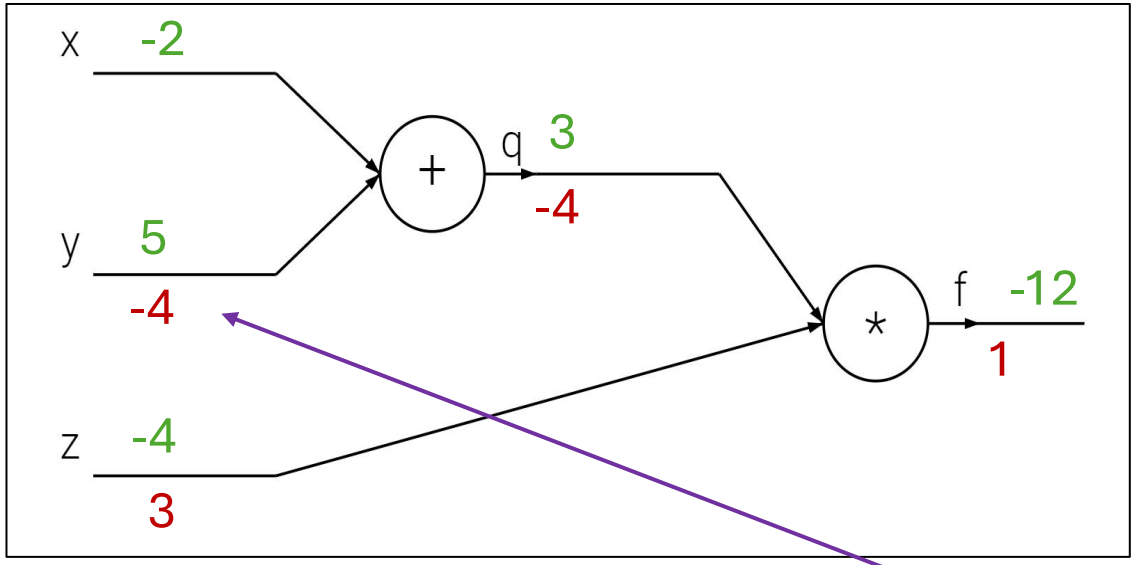
$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4



- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

$\frac{\partial f}{\partial y}$

Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y}$$

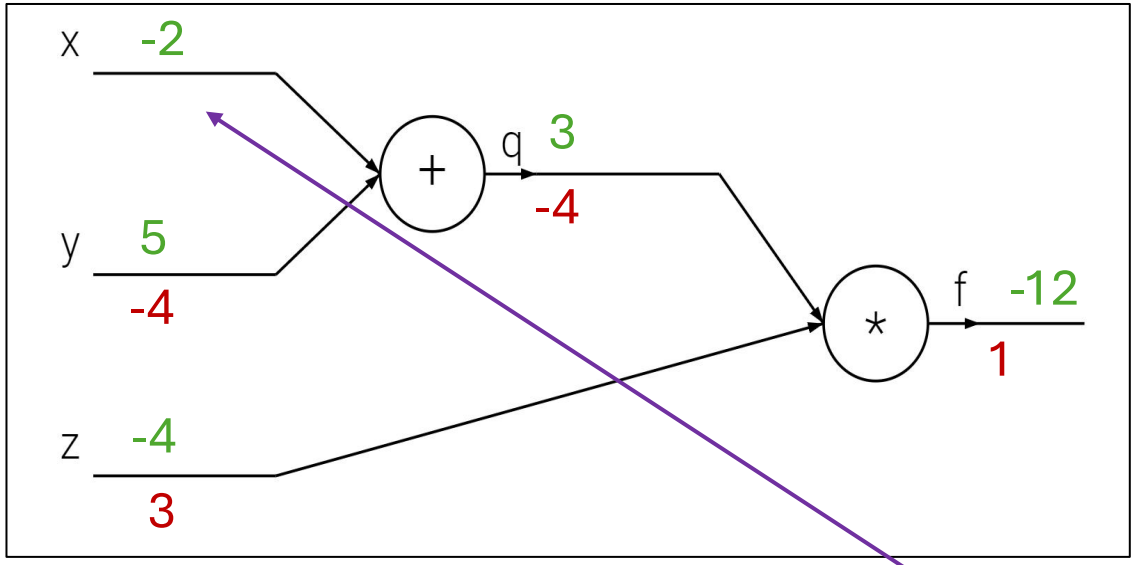
Upstream gradient

Local gradient

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4



- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

$\frac{\partial f}{\partial x}$

Chain rule:

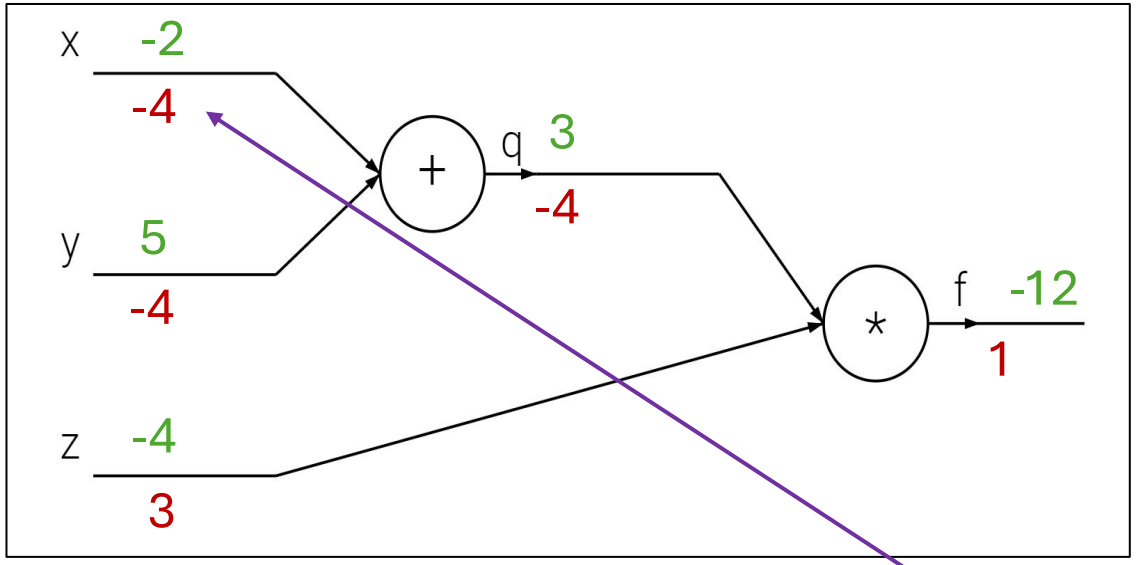
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

Upstream gradient

Local gradient

反向传播：简单示例

- $f(x, y, z) = (x + y)z$
  - e.g. x=-2, y=5, z=-4



- $q = x + y;\ \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$
- $f = qz;\quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$

$\frac{\partial f}{\partial x}$

Chain rule:

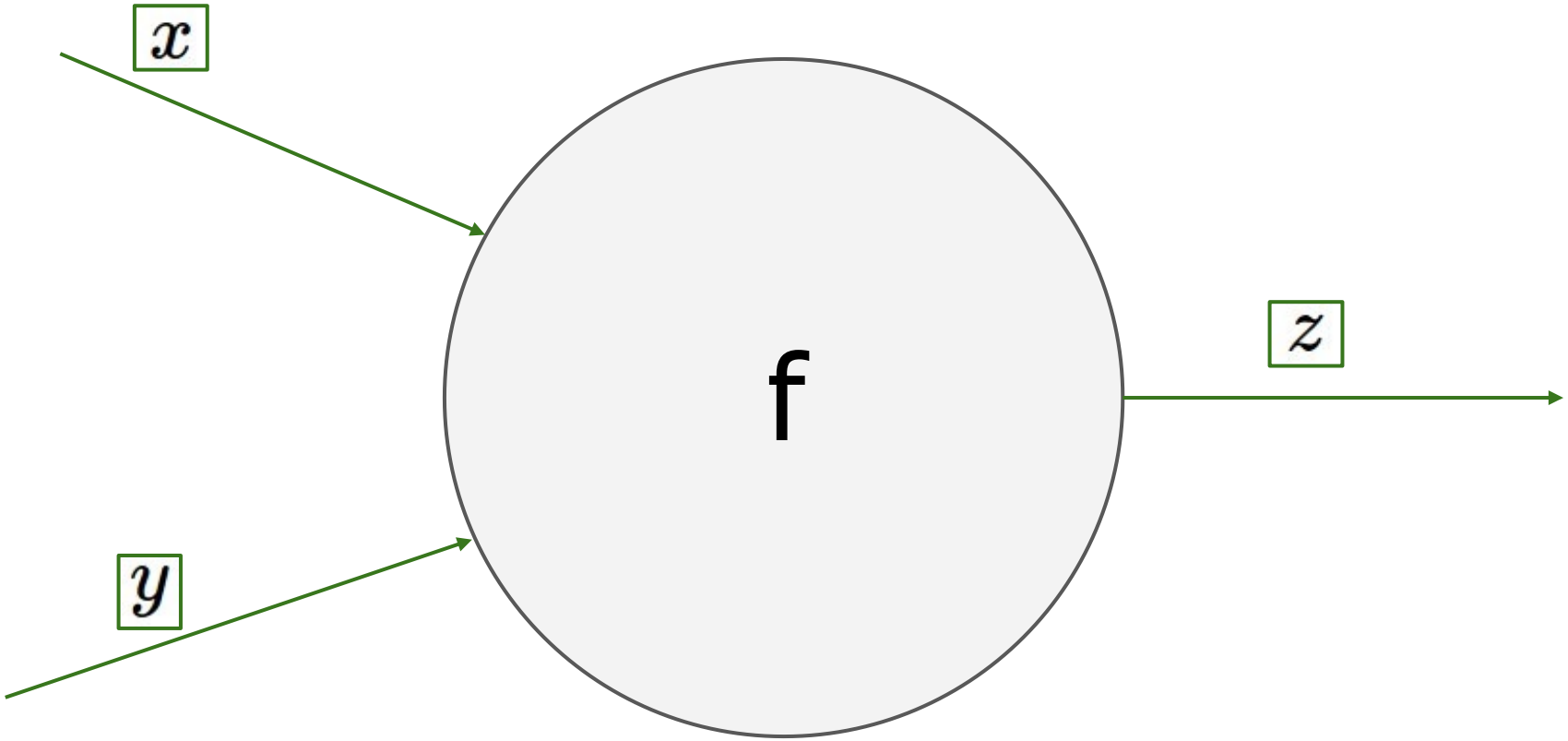
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

Upstream gradient

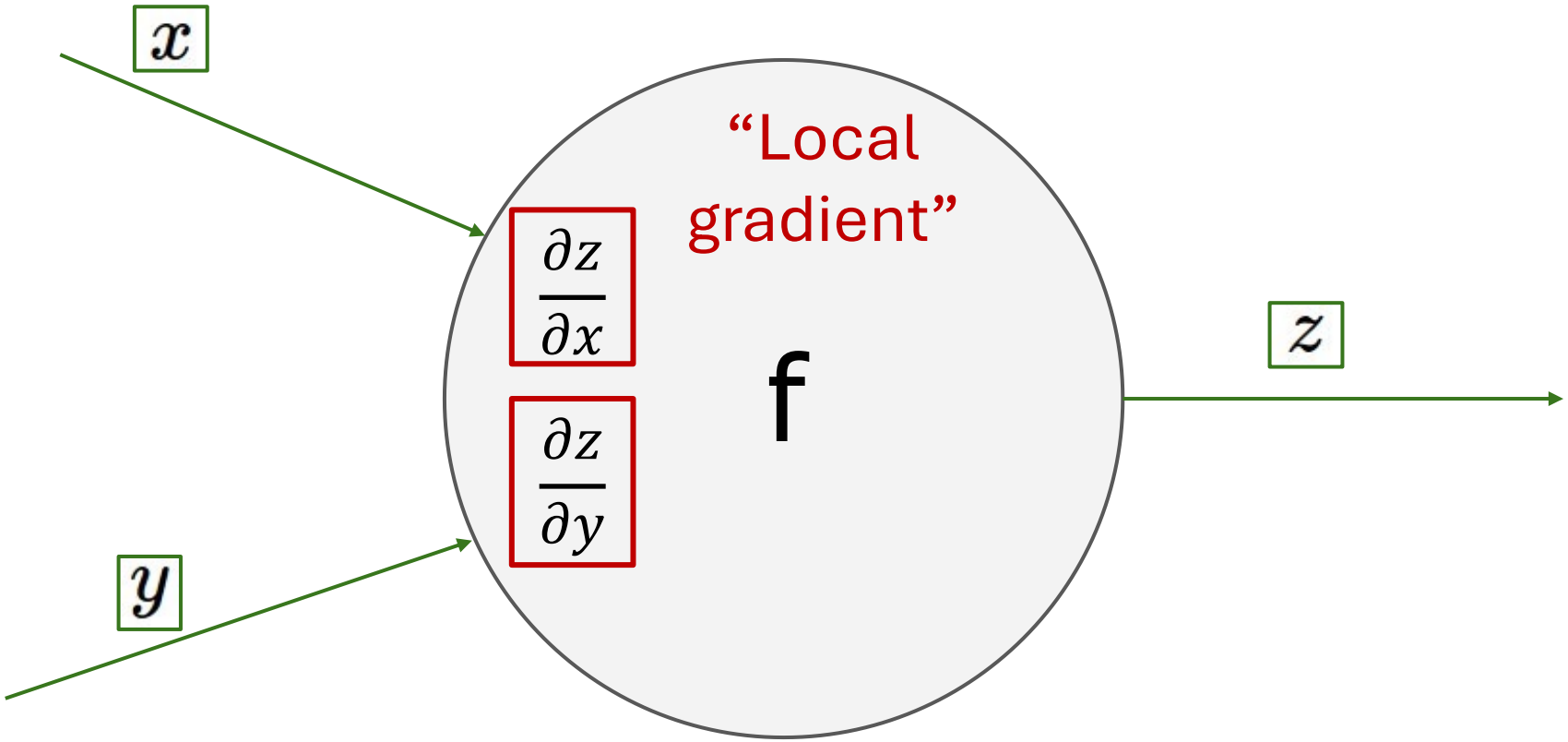
Local gradient

- Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

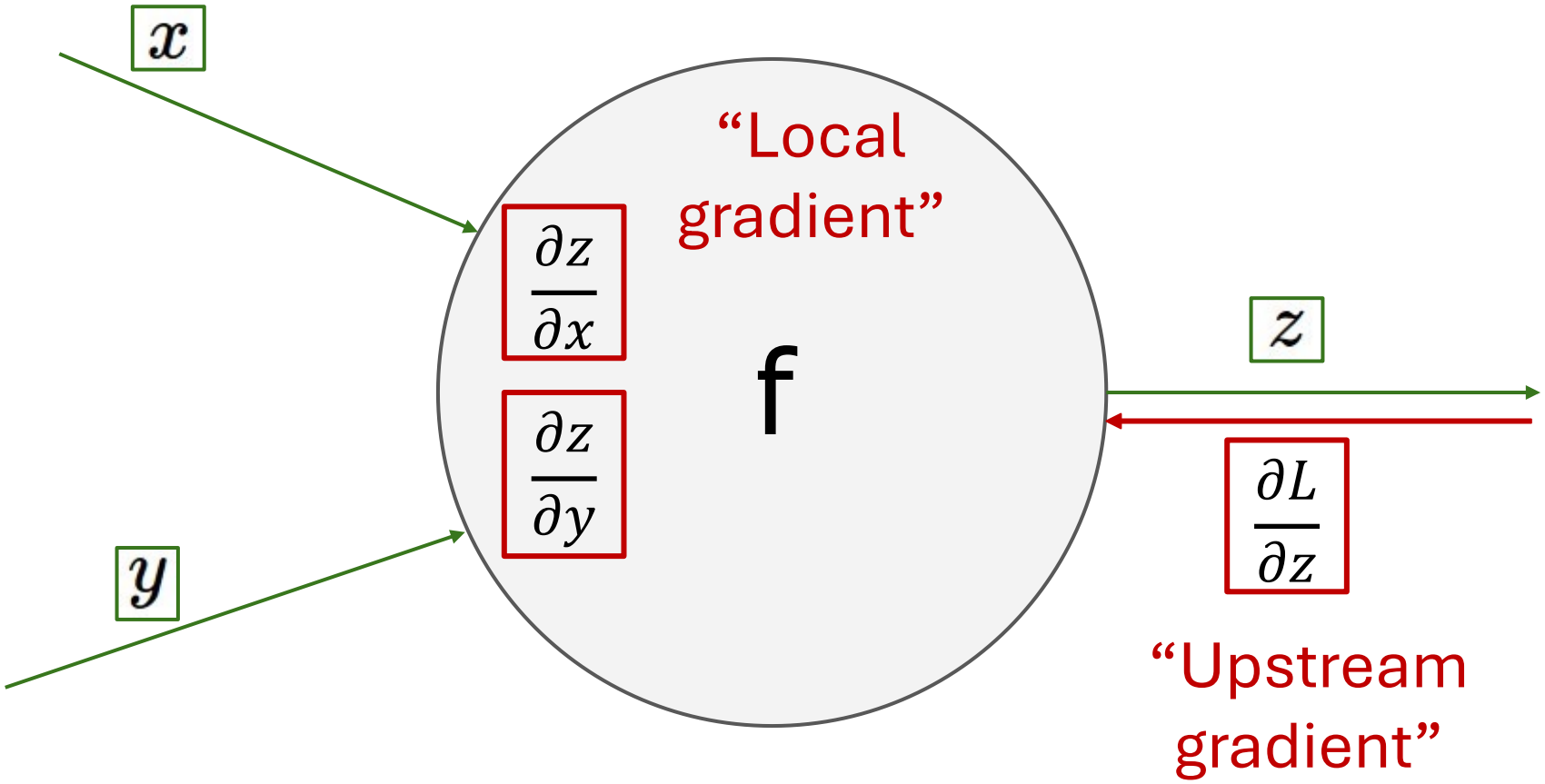
反向传播：链式法则



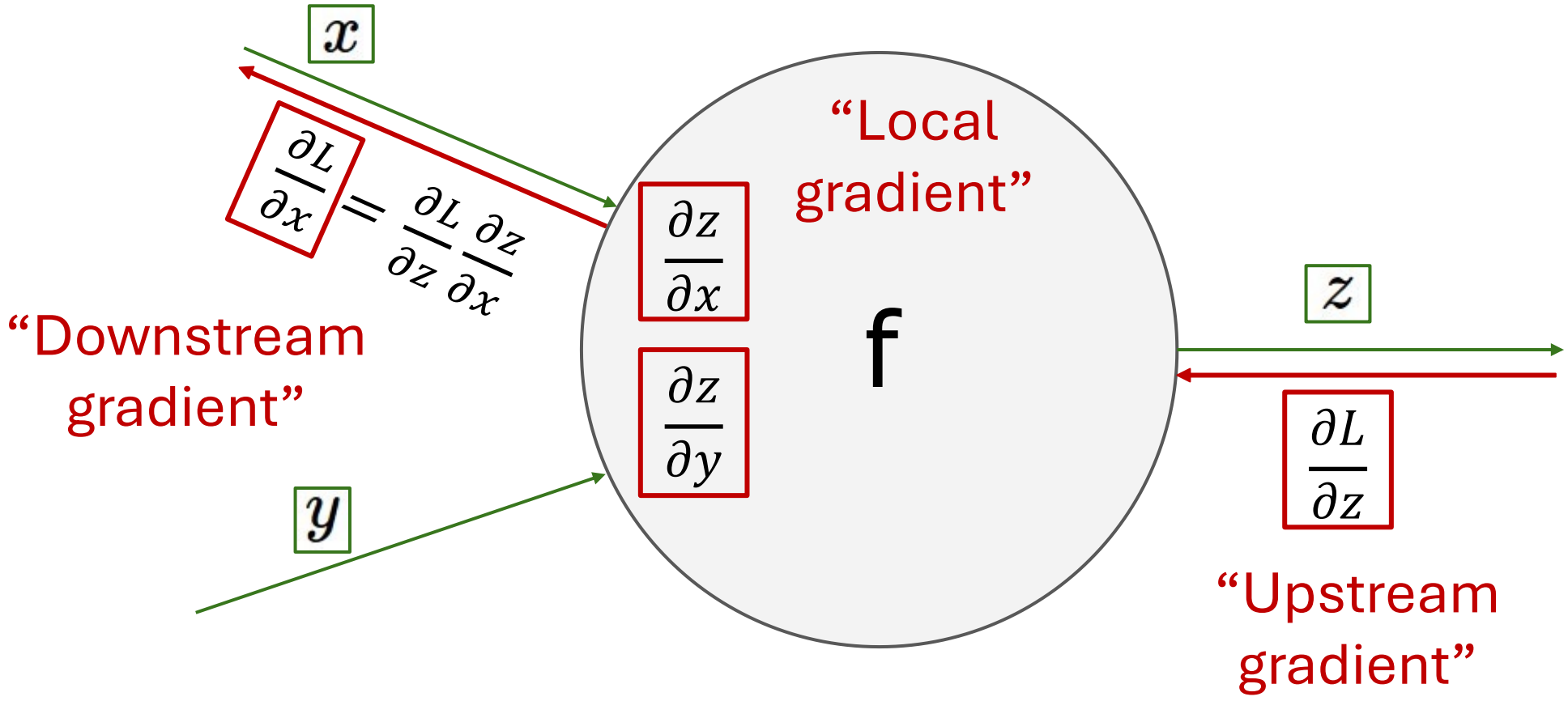
反向传播：链式法则


x
y
$\frac{\partial z}{\partial x}$
$\frac{\partial z}{\partial y}$
“Local gradient”
f
z

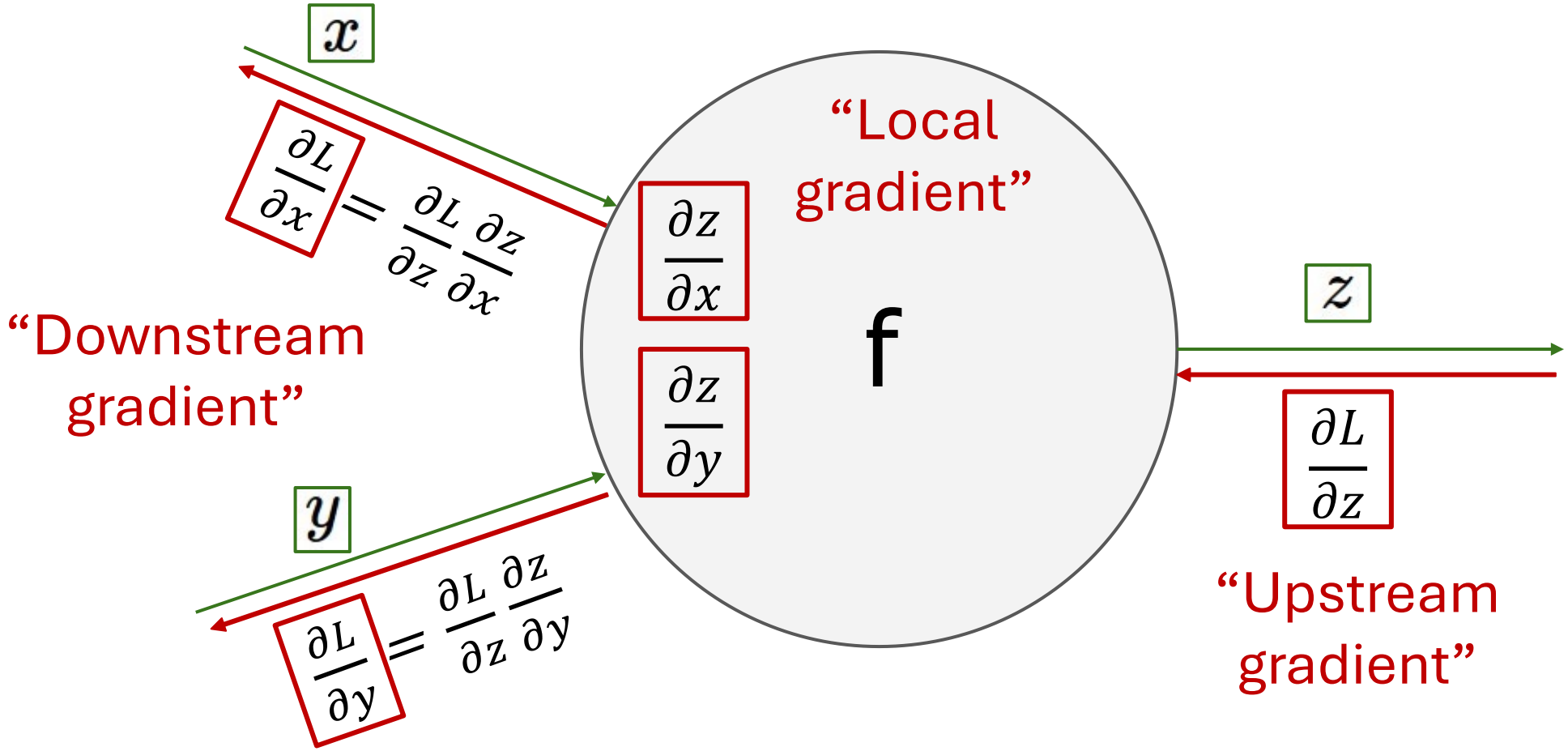
反向传播：链式法则


x
y
$\frac{\partial z}{\partial x}$
$\frac{\partial z}{\partial y}$
“Local gradient”
f
z
$\frac{\partial L}{\partial z}$
“Upstream gradient”

反向传播：链式法则


x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
“Downstream gradient”
y
“Local gradient”
$\frac{\partial z}{\partial x}$
$\frac{\partial z}{\partial y}$
f
z
$\frac{\partial L}{\partial z}$
“Upstream gradient”

反向传播：链式法则

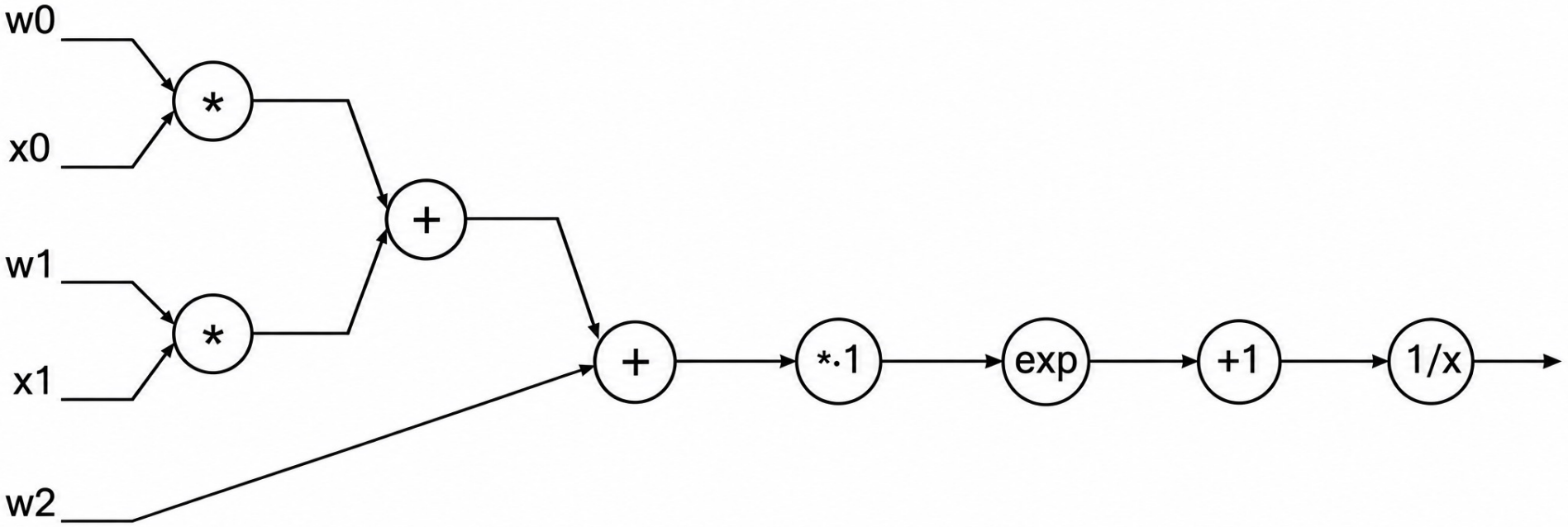

x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
“Downstream gradient”
y
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$
“Local gradient”
$\frac{\partial z}{\partial x}$
$\frac{\partial z}{\partial y}$
f
z
$\frac{\partial L}{\partial z}$
“Upstream gradient”

提纲

- **计算图与反向传播**
  - 标量计算图与链式法则
  - 常见节点的梯度规则
- 向量与张量的反向传播
- 神经网络训练中的挑战
- 数据流形与模型泛化

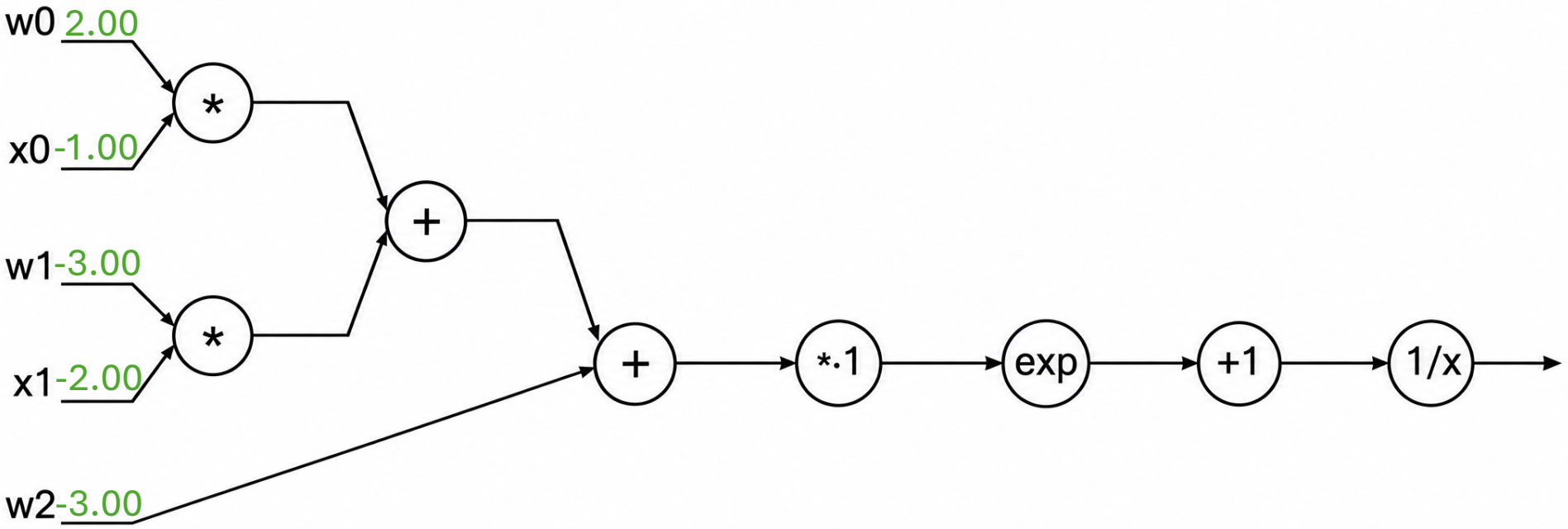
另一个例子

- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



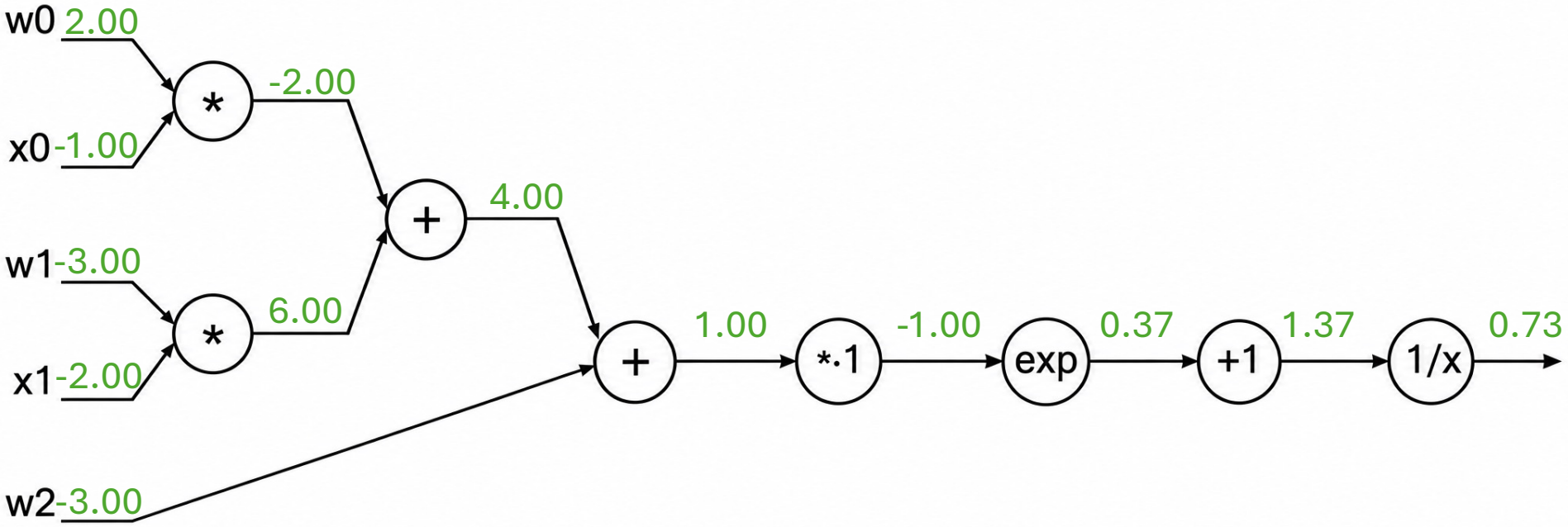
另一个例子

- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



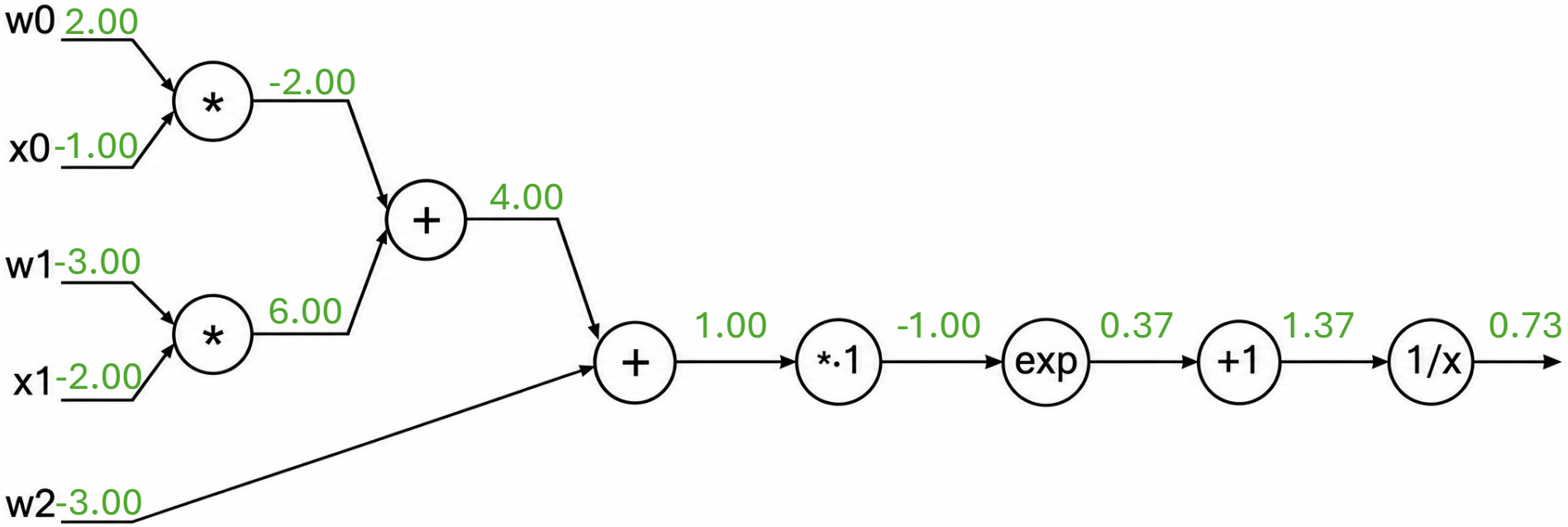
另一个例子

- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



另一个例子

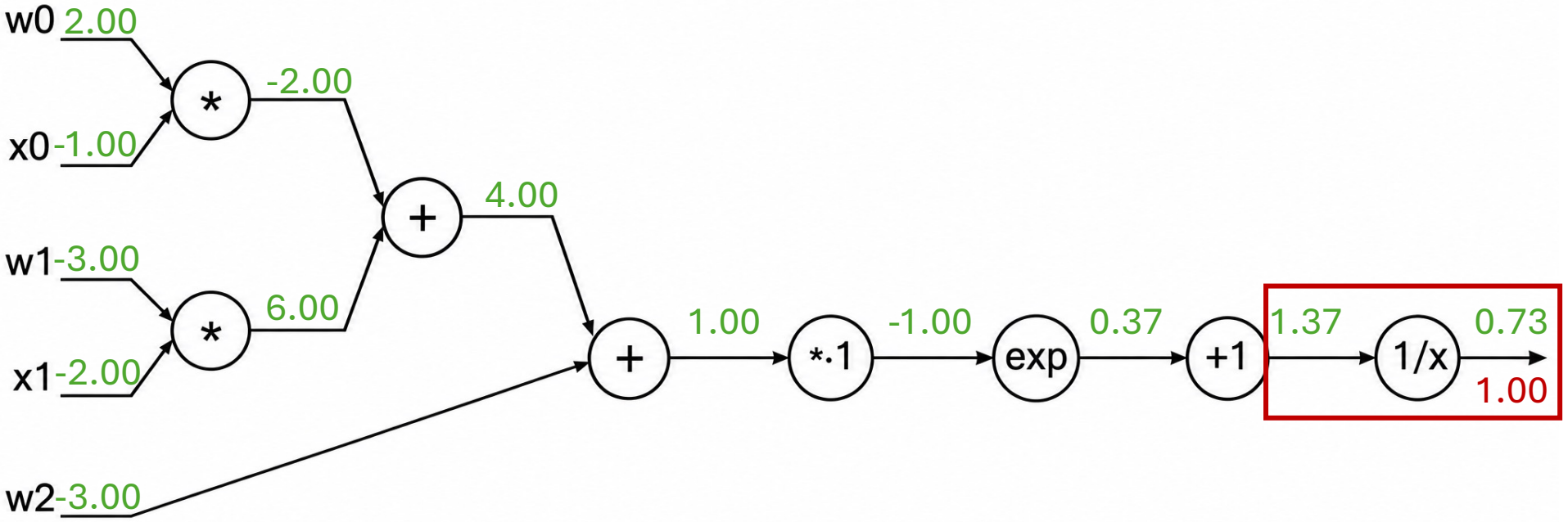
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

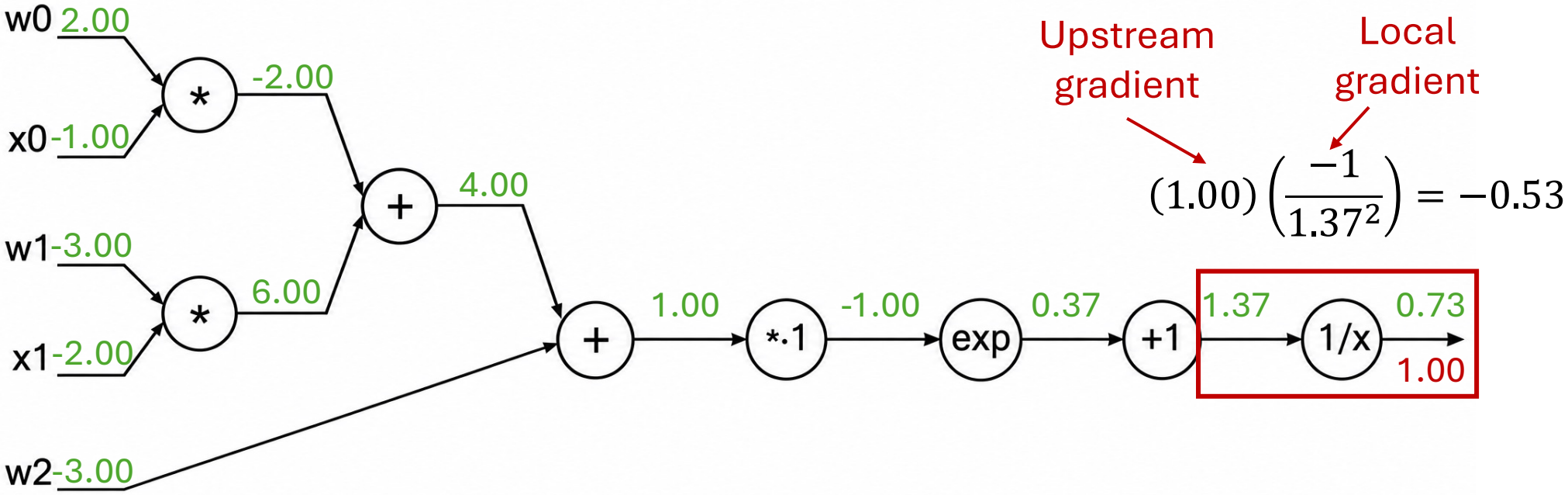
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

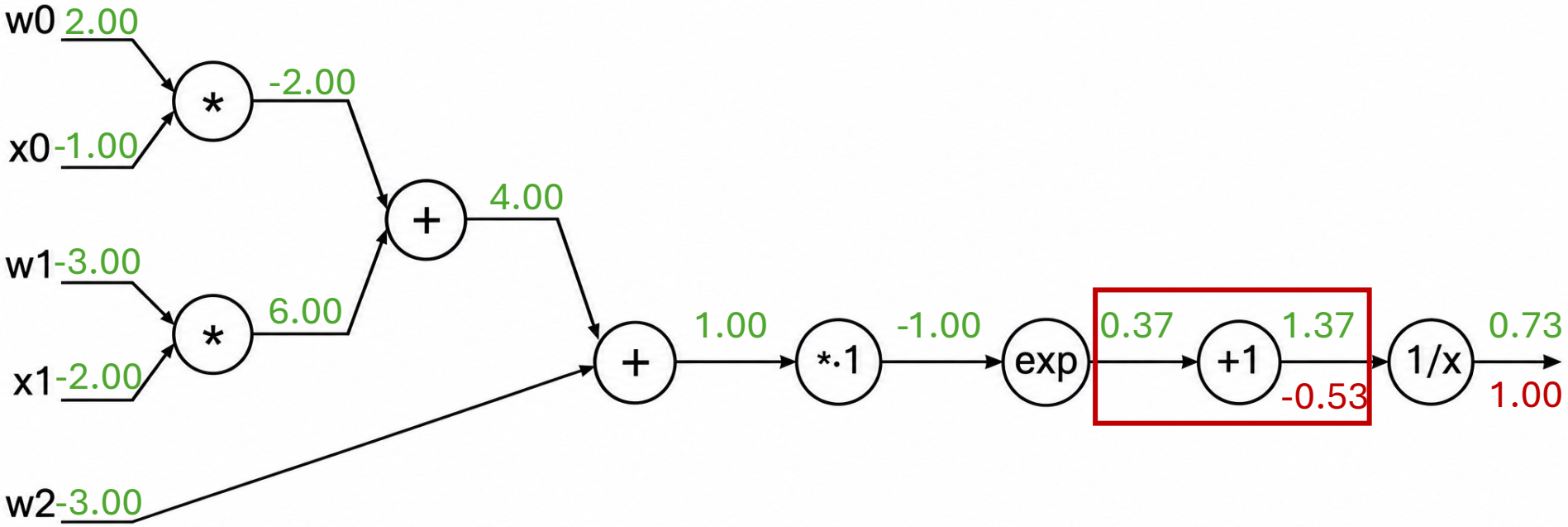
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$



$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$	$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$

另一个例子

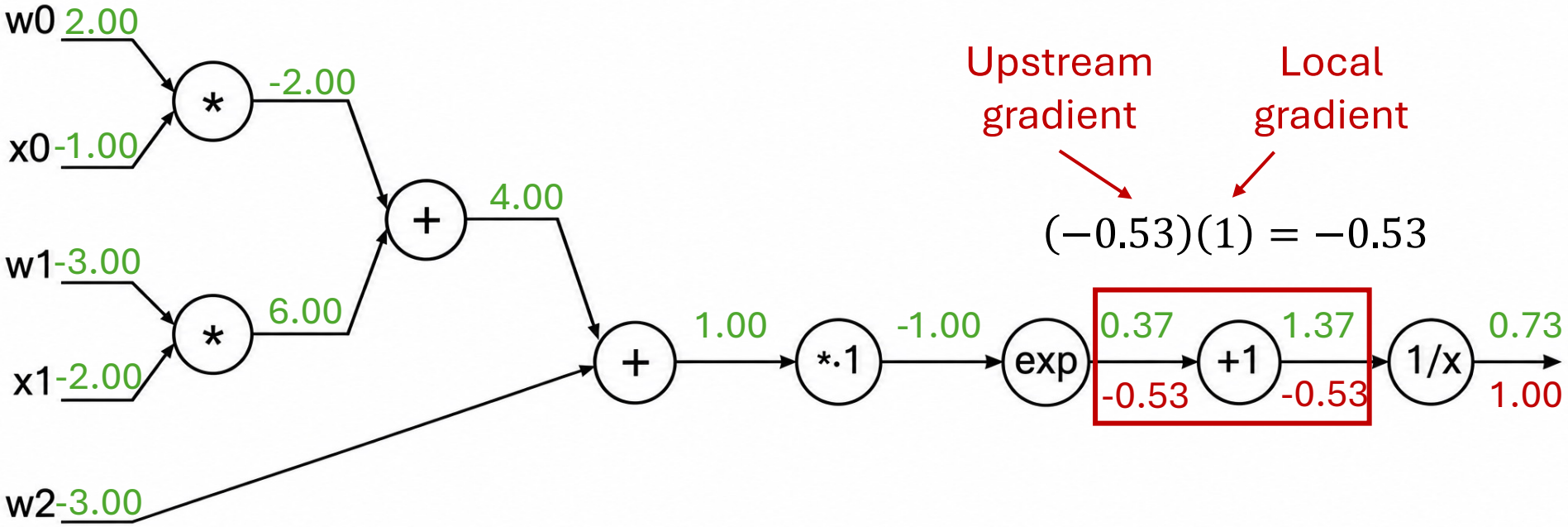
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

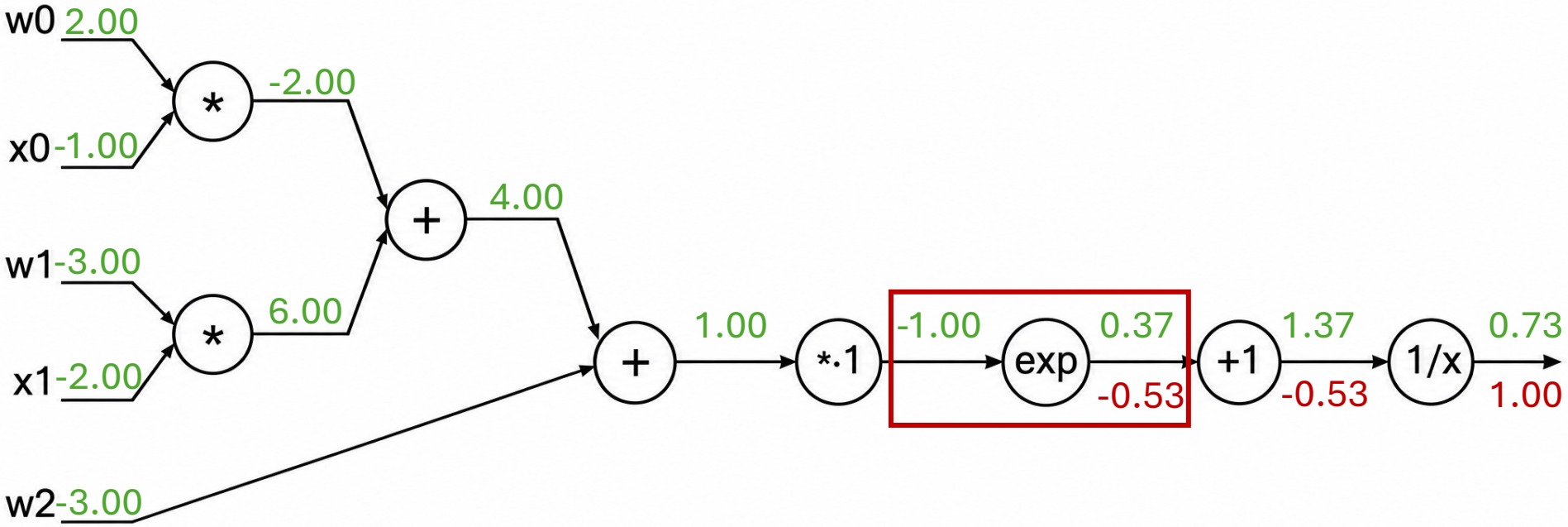
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

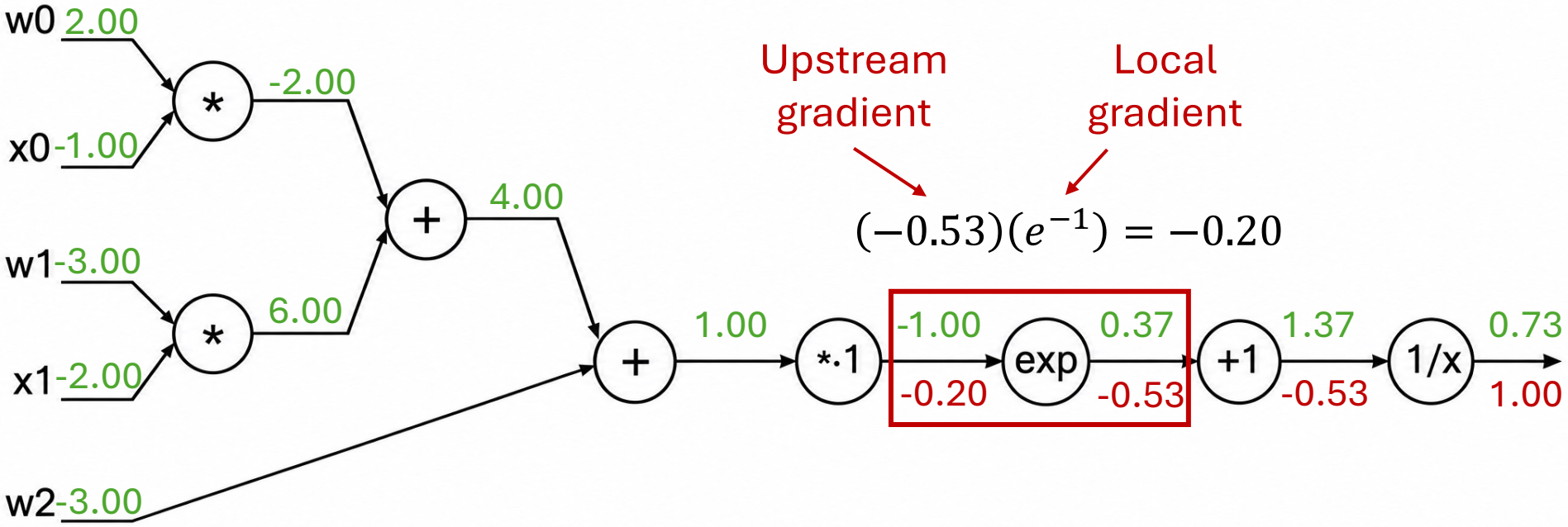
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

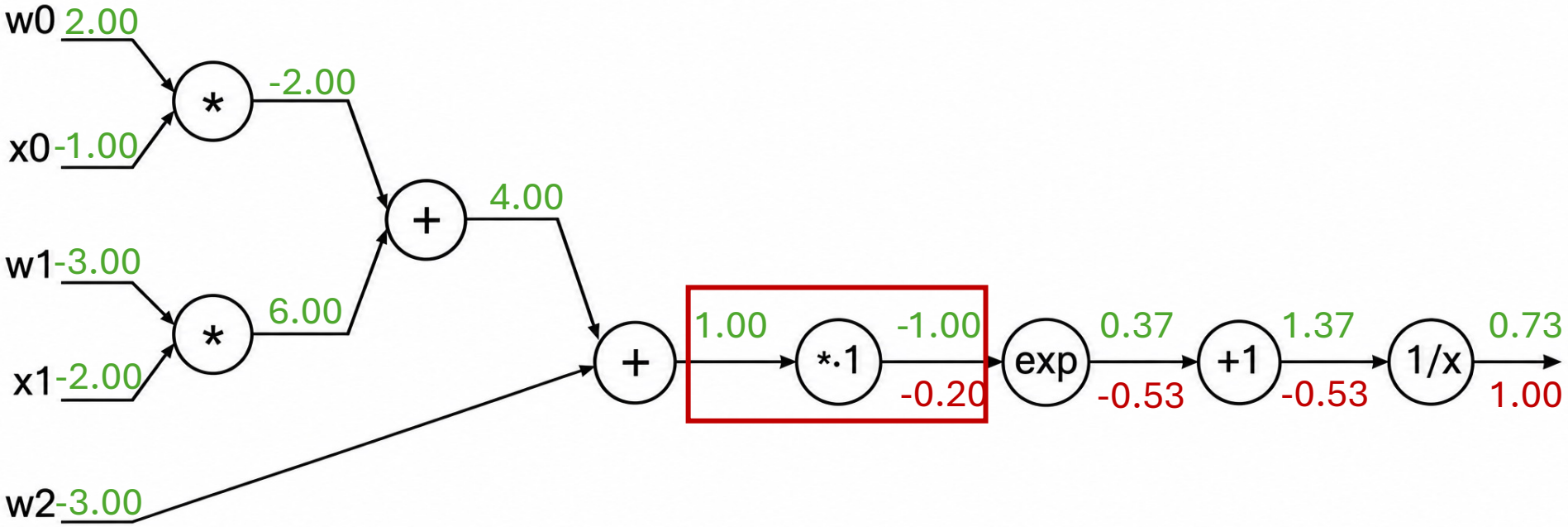
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

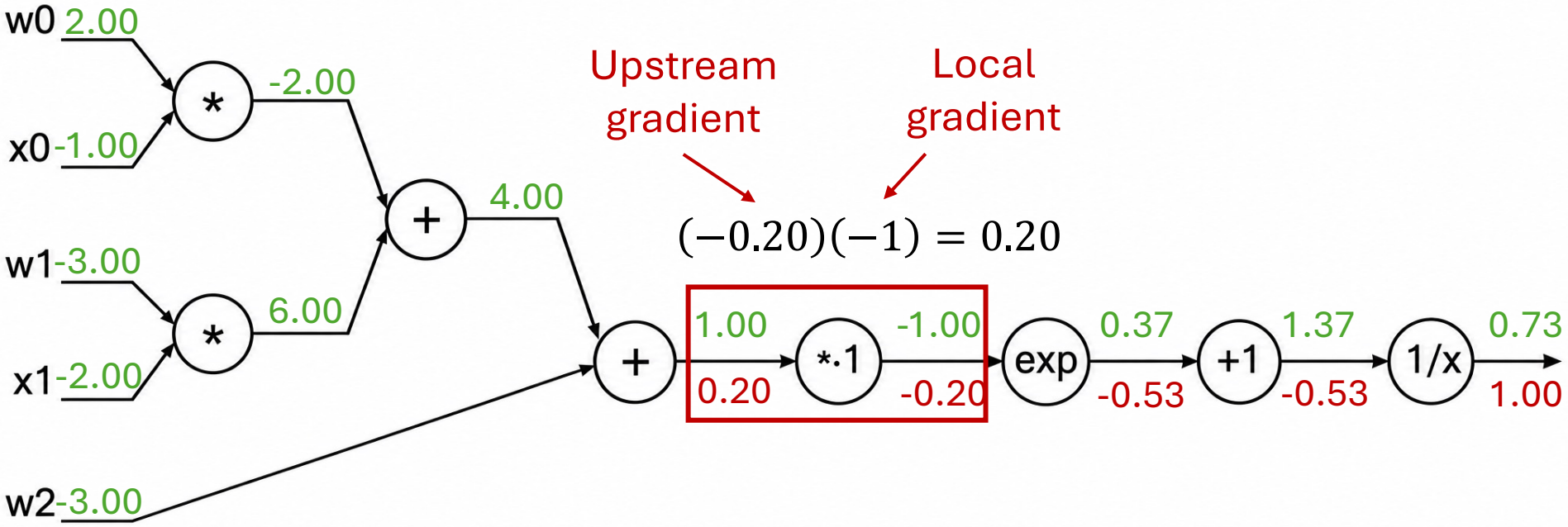
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

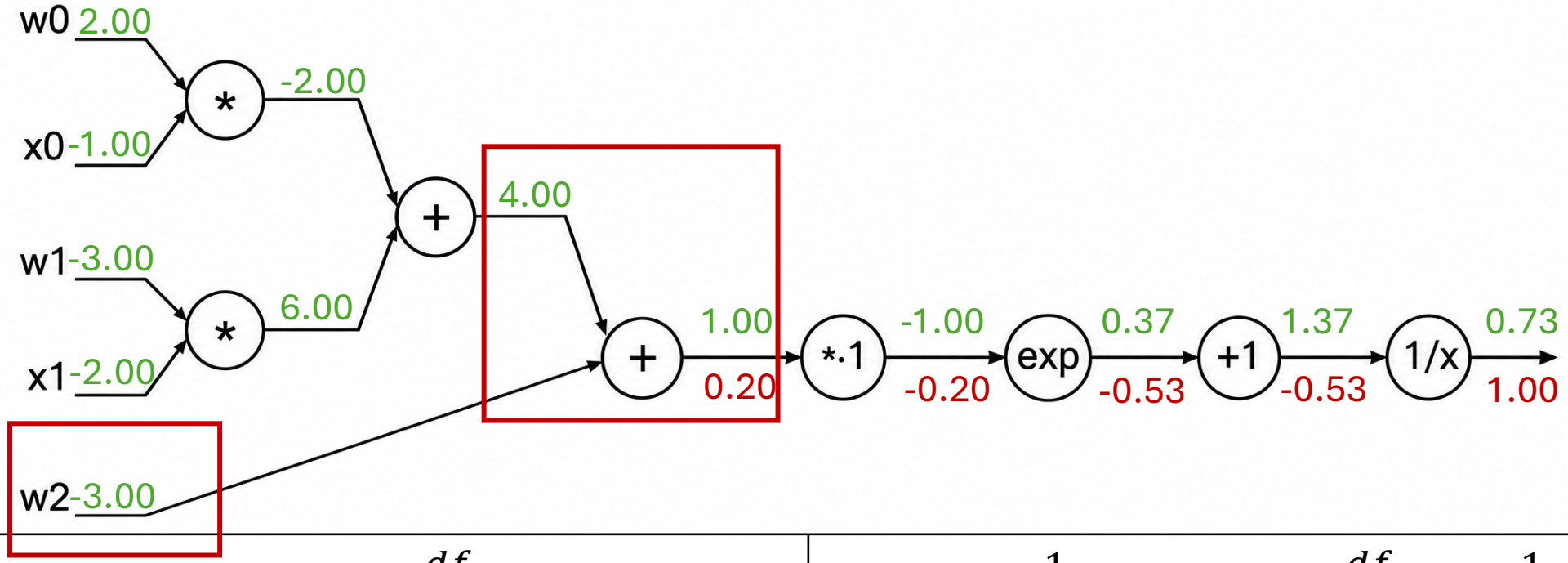
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

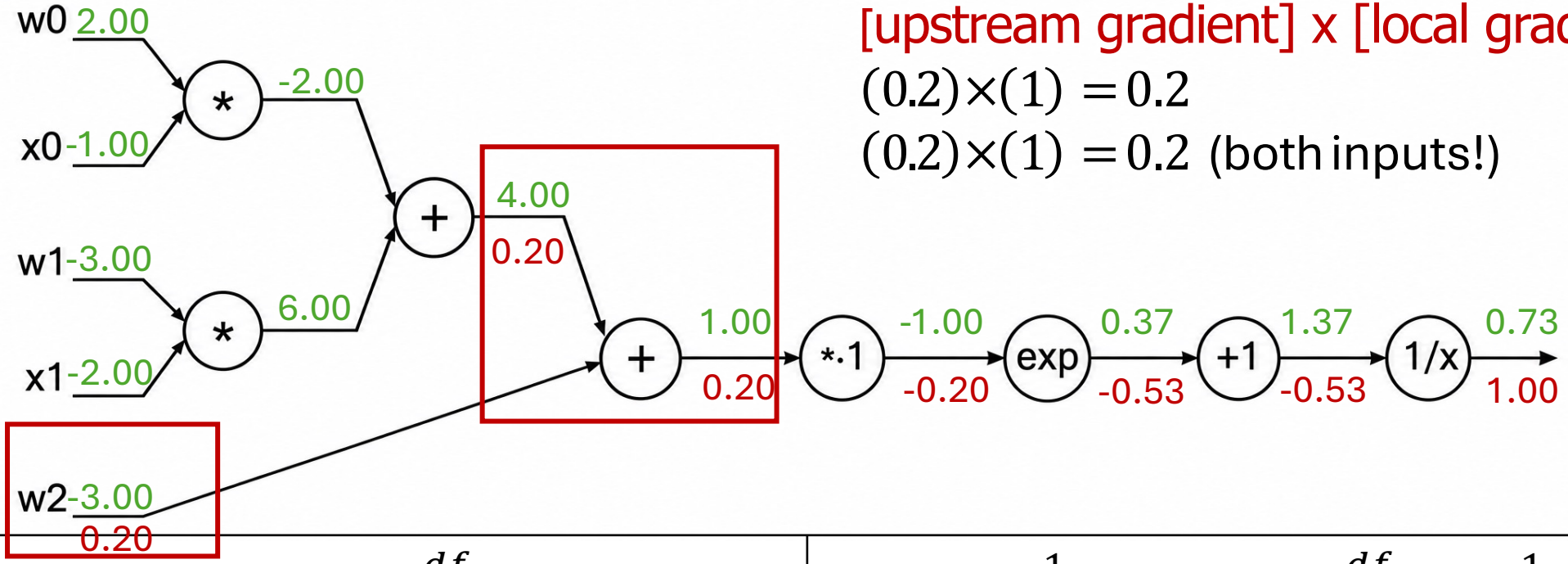
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



[upstream gradient] x [local gradient]

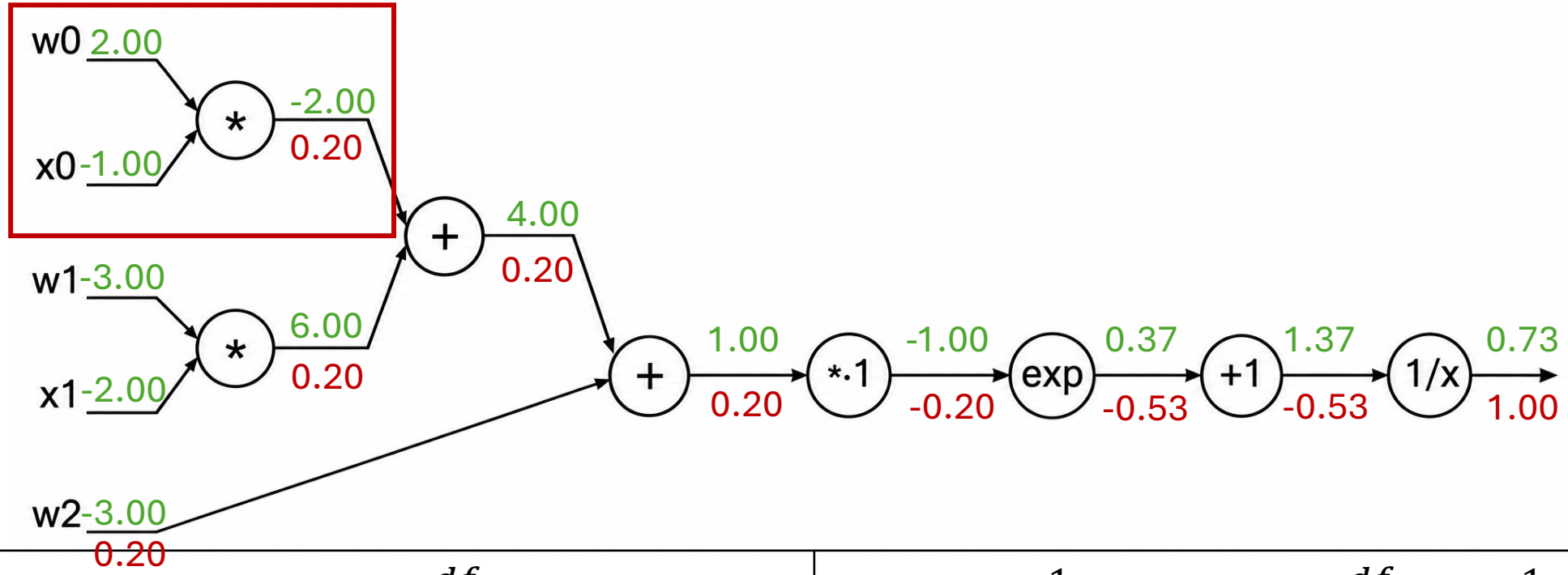
$(0.2) \times (1) = 0.2$

$(0.2) \times (1) = 0.2$ (both inputs!)

$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

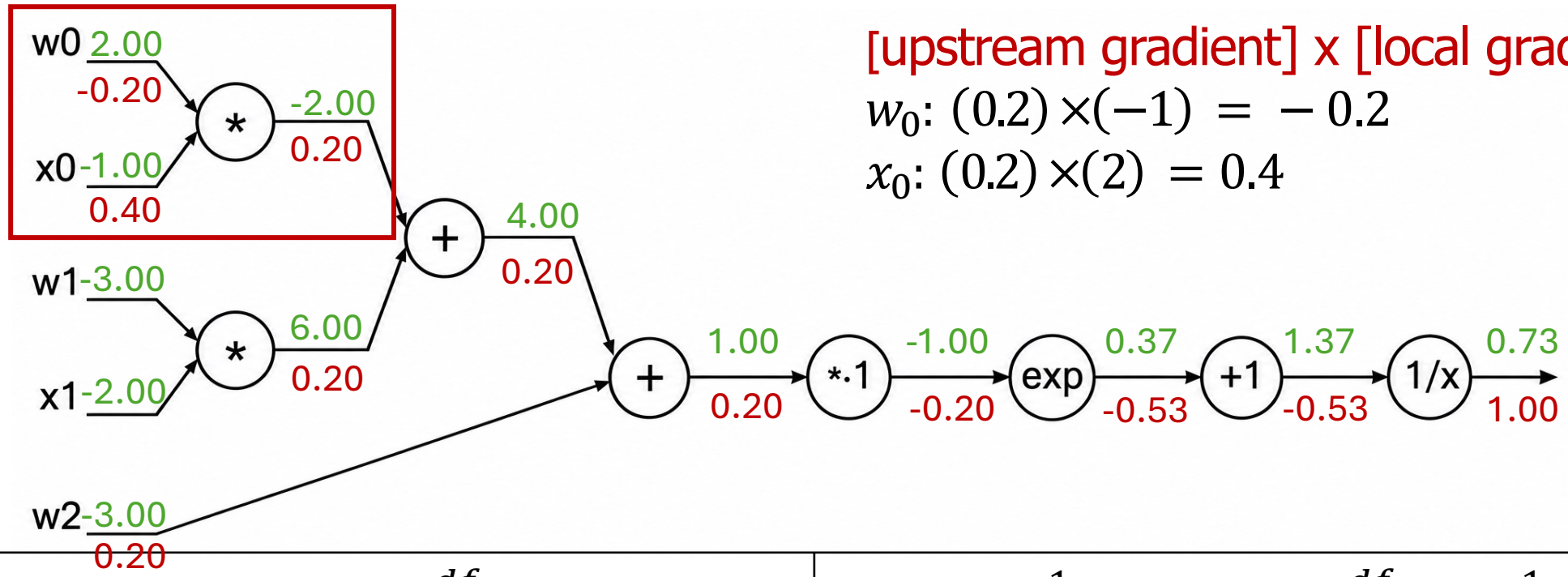
- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



[upstream gradient] x [local gradient]

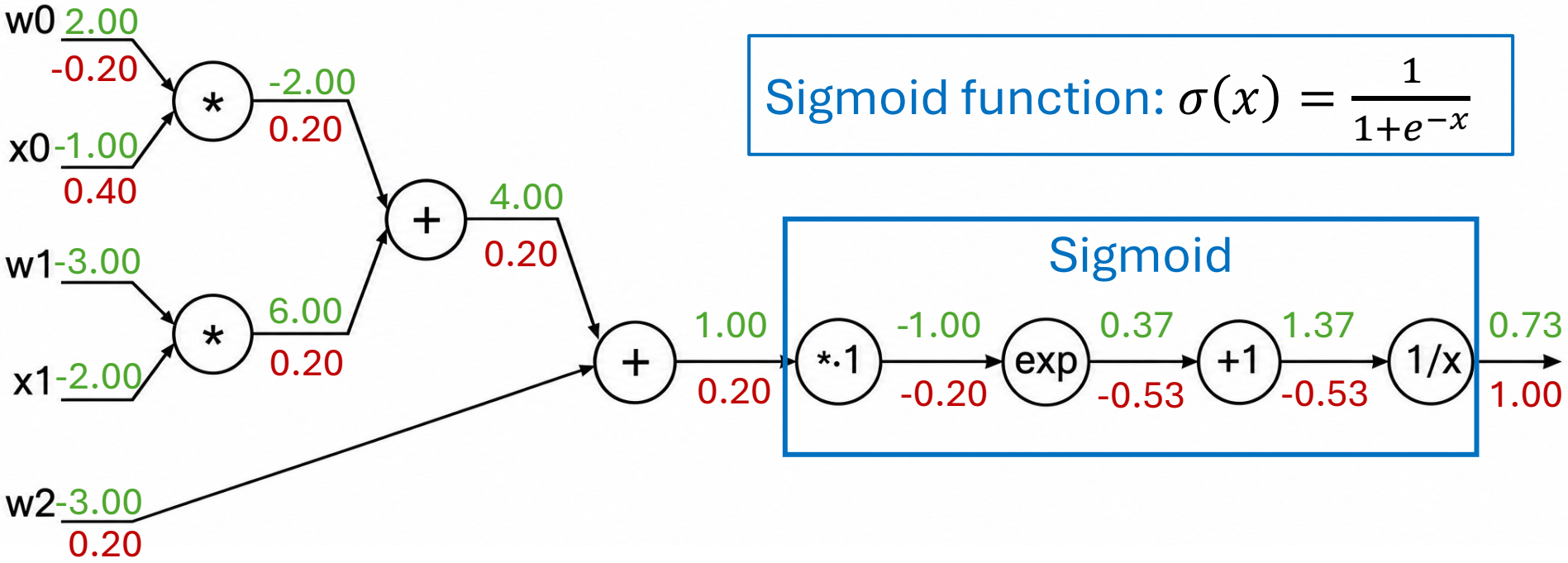
w_0: $(0.2) \times (-1) = -0.2$

x_0: $(0.2) \times (2) = 0.4$

$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$	$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -\frac{1}{x^2}$
$f_a(x) = ax \rightarrow \frac{df}{dx} = a$	$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

另一个例子

- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$

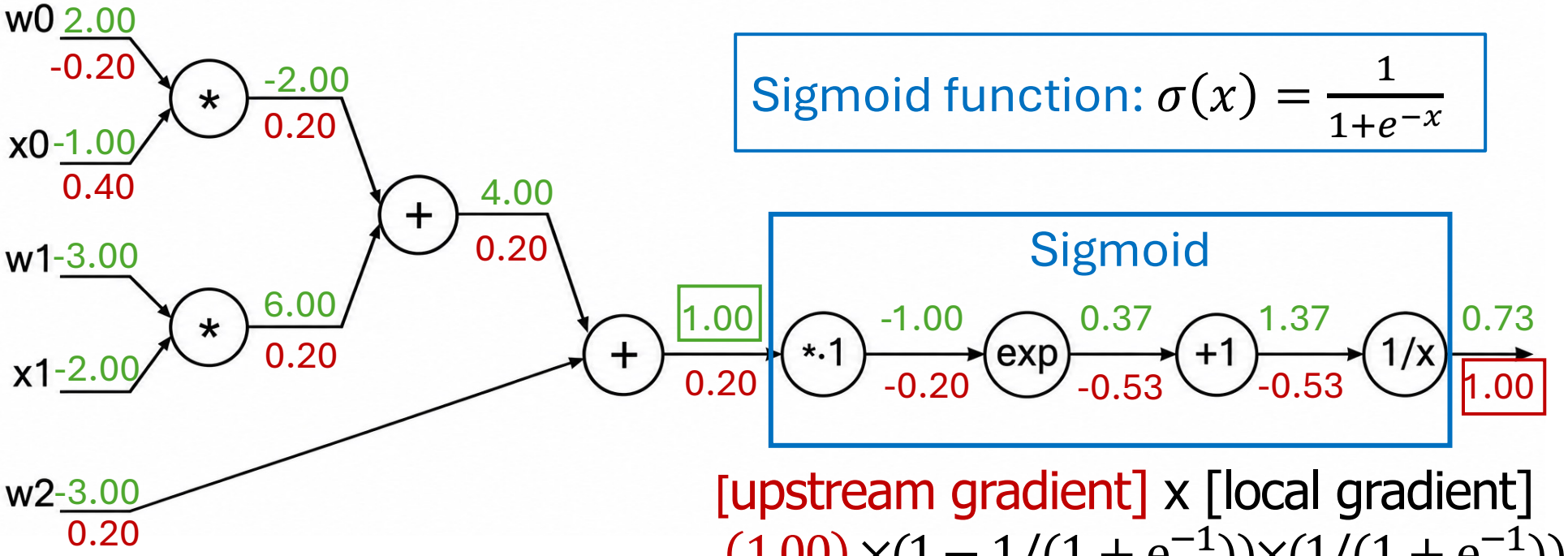


- 计算图的画法不是唯一的！我们要选择那种每个节点的局部导数很好算的拆分方式。

另一个例子

- $f(\mathbf{w}, x) = \frac{1}{1+e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$

计算图的画法不是唯一的！我们要选择那种每个节点的局部导数很好算的拆分方式。



Sigmoid function: $\sigma(x) = \frac{1}{1+e^{-x}}$

[upstream gradient] x [local gradient]

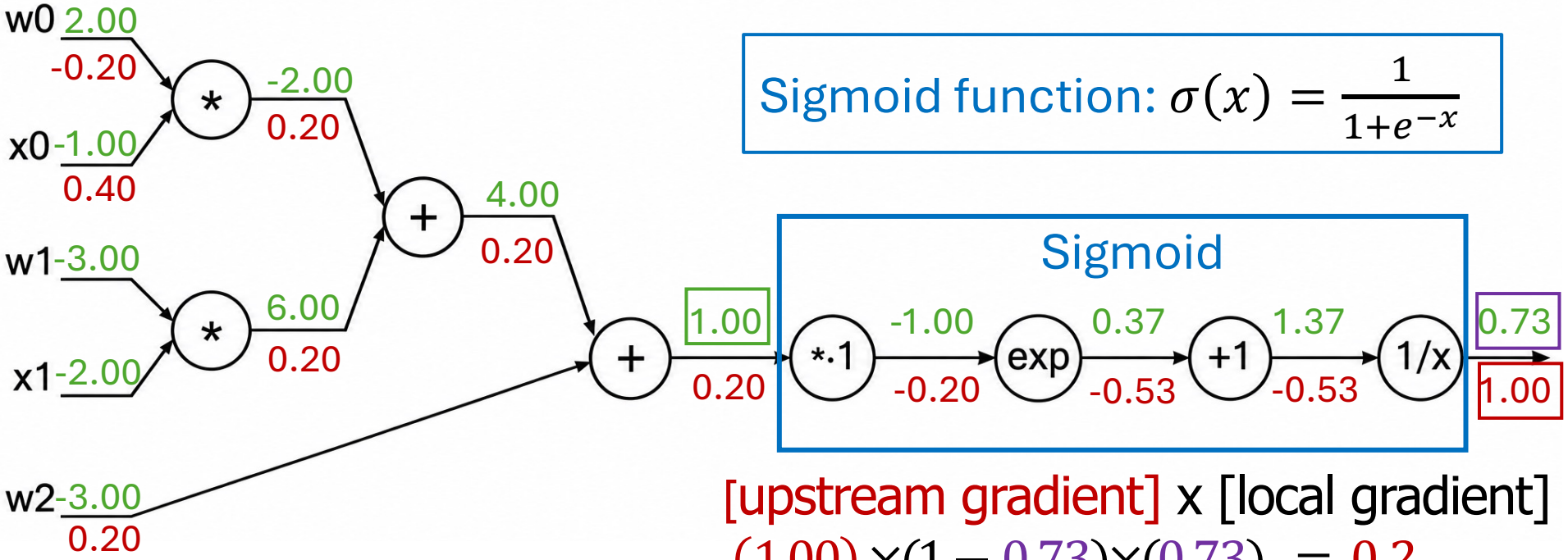
$(1.00) \times (1 - 1/(1 + e^{-1})) \times (1/(1 + e^{-1})) = 0.2$

Sigmoid local Gradient: $\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{(-x)})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\sigma(x)$

另一个例子

- $f(\mathbf{w}, \boldsymbol{x}) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$

计算图的画法不是唯一的！我们要选择那种每个节点的局部导数很好算的拆分方式。



Sigmoid function: $\sigma(x) = \frac{1}{1+e^{-x}}$

[upstream gradient] x [local gradient]

$(1.00) \times (1 - 0.73) \times (0.73) = 0.2$

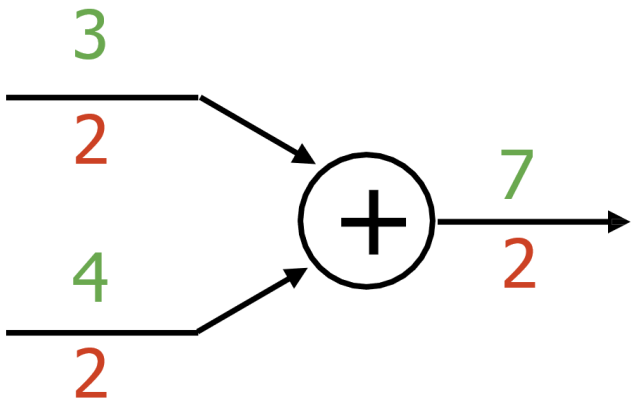
Sigmoid local Gradient: $\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{(-x)})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\sigma(x)$

提纲

- **计算图与反向传播**
  - 链式法则与局部梯度
  - 常见运算节点的梯度规则
- 反向传播的实现
- 向量与张量的反向传播
- 神经网络训练中的挑战
- 数据流形与模型泛化

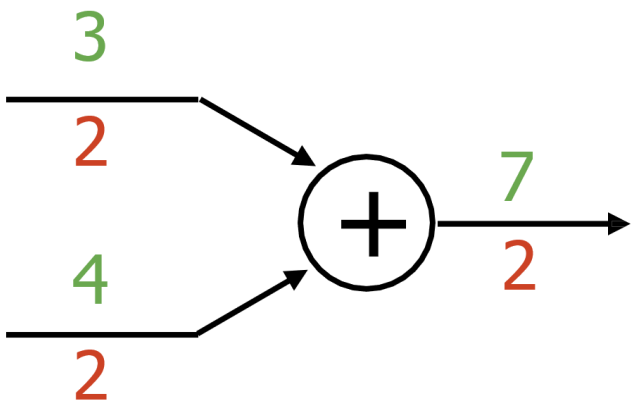
梯度流的模式

加法门：梯度分配器

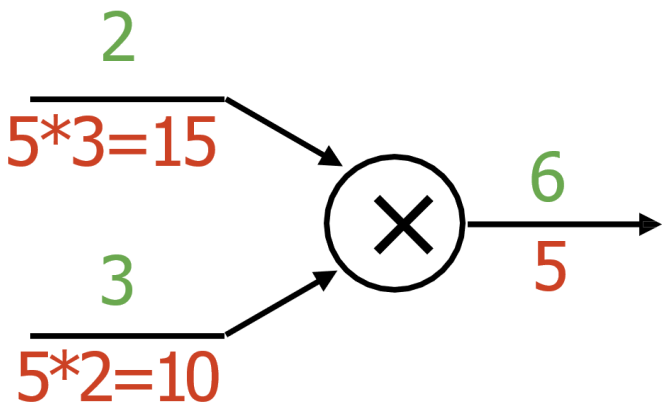


梯度流的模式

加法门：梯度分配器

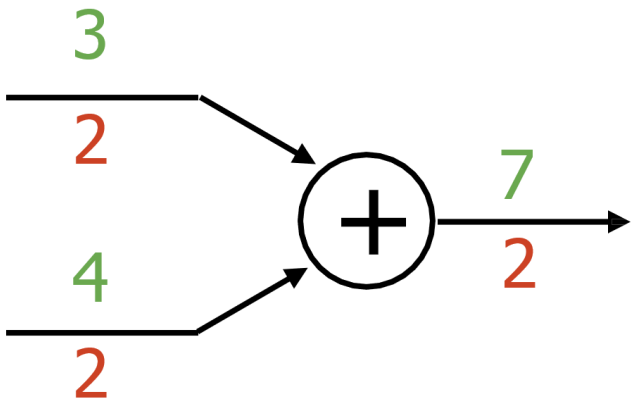


乘法门：“交换乘数”

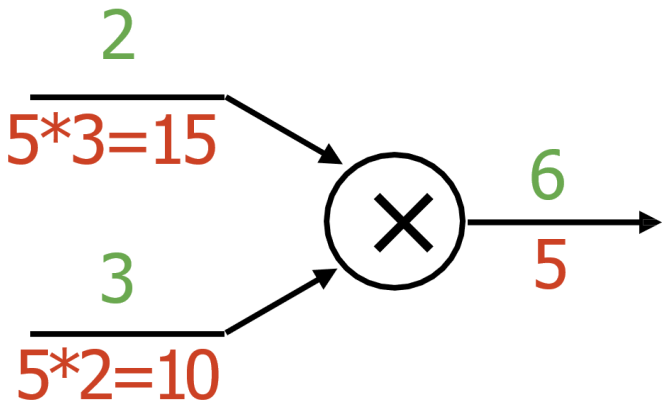


梯度流的模式

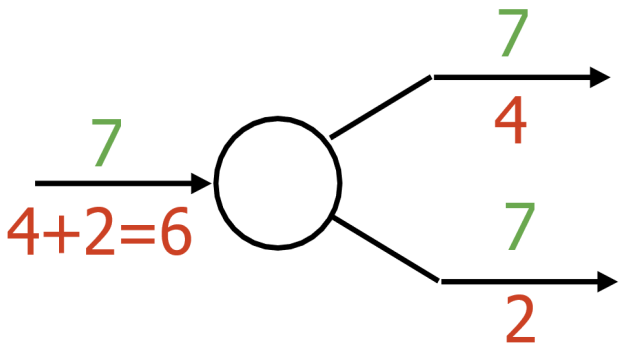
加法门：梯度分配器



乘法门：“交换乘数”

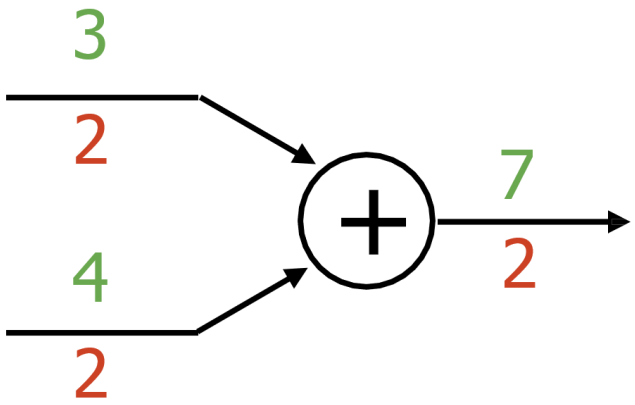


复制门：梯度加法器

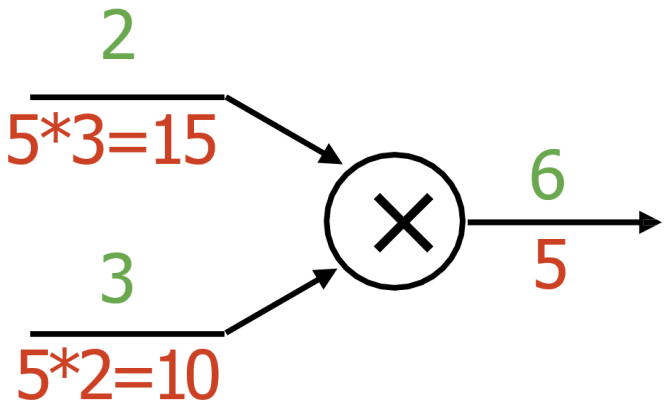


梯度流的模式

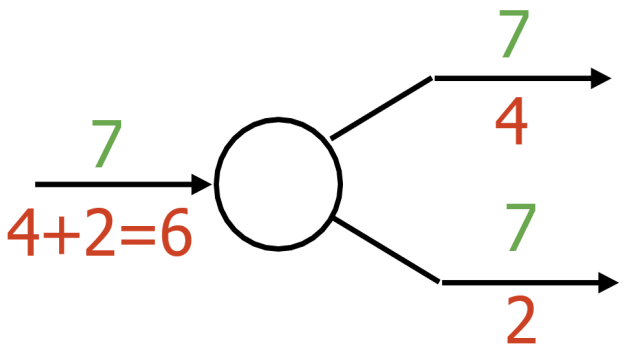
加法门：梯度分配器



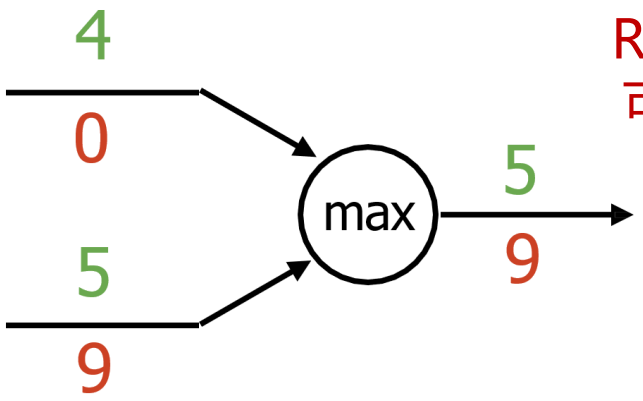
乘法门：“交换乘数”



复制门：梯度加法器

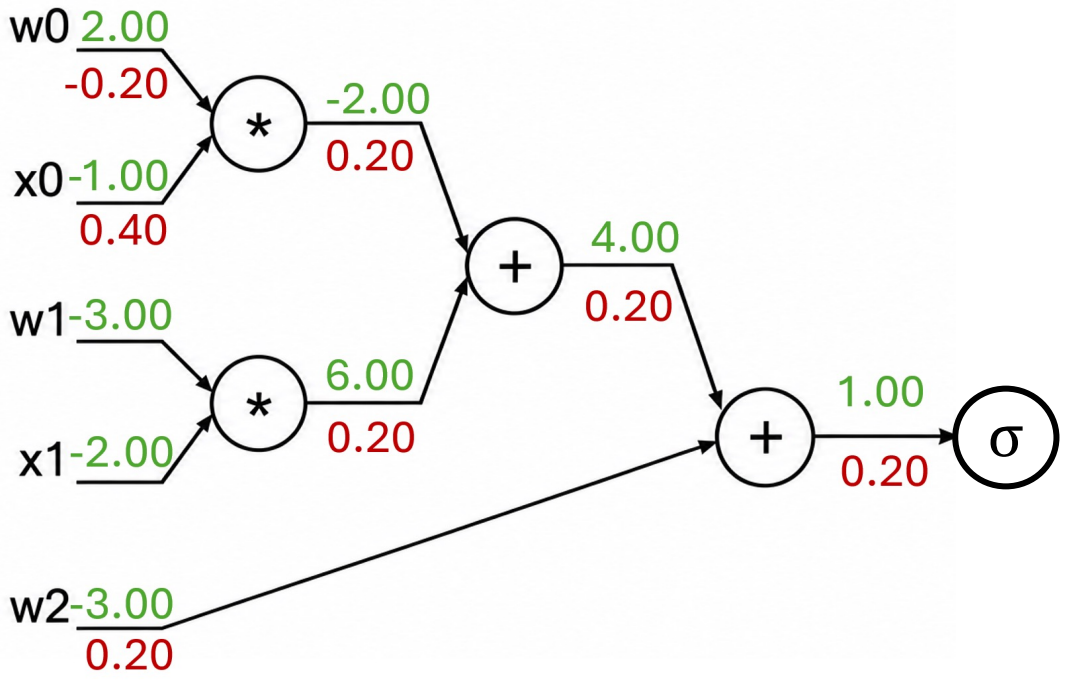


最大化门：梯度路由器



ReLU激活函数有什么可能存在的问题吗？

反向传播实现：“展开”代码



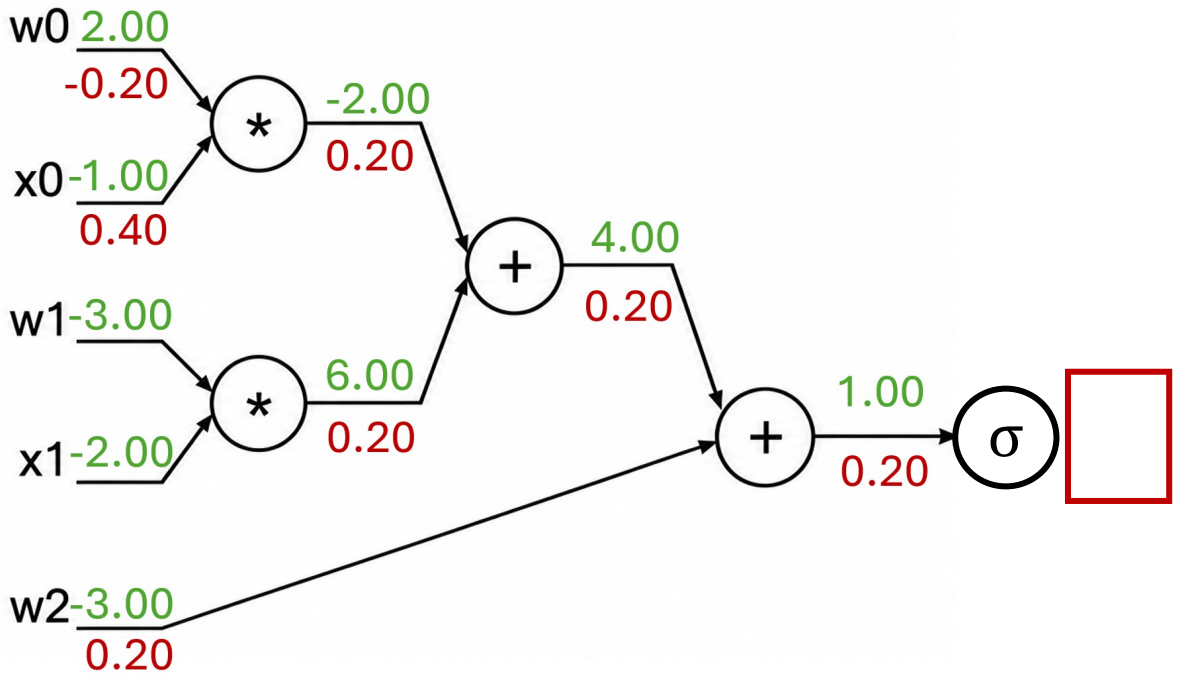
前向传播：
计算输出

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

反向传播：
计算梯度

```
grad_L = 1.0
grad_s3 = grad_L *(1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_s0 = grad_s2
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```

反向传播实现：“展开”代码



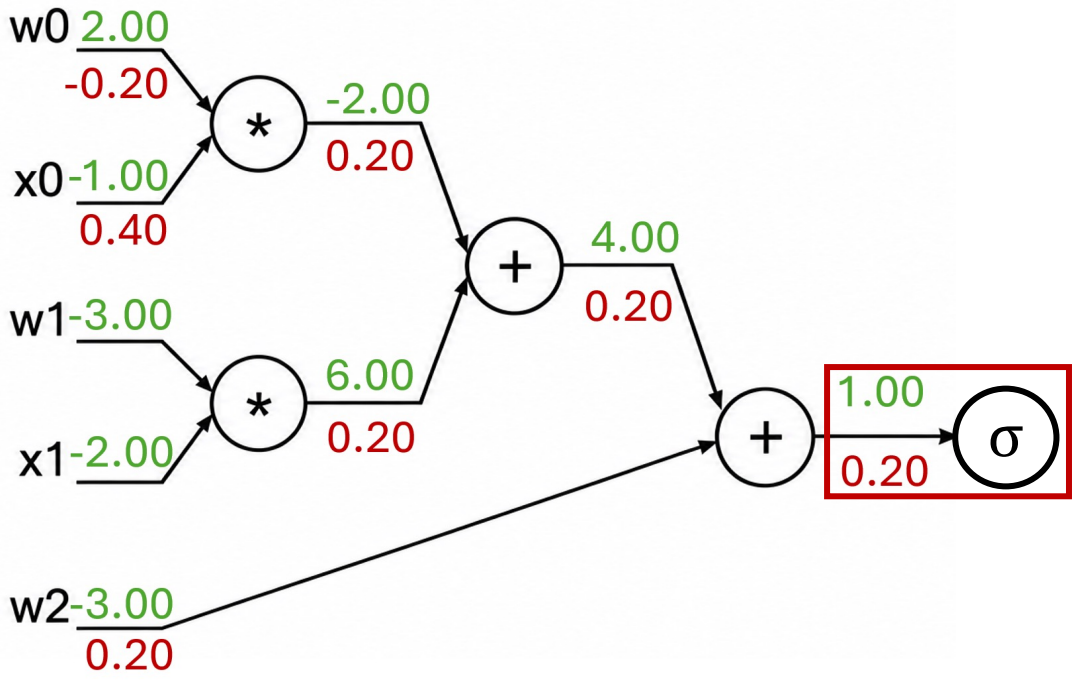
前向传播：
计算输出

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Base case

```
grad_L = 1.0
grad_s3 = grad_L * (1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_s0 = grad_s2
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```

反向传播实现：“展开”代码



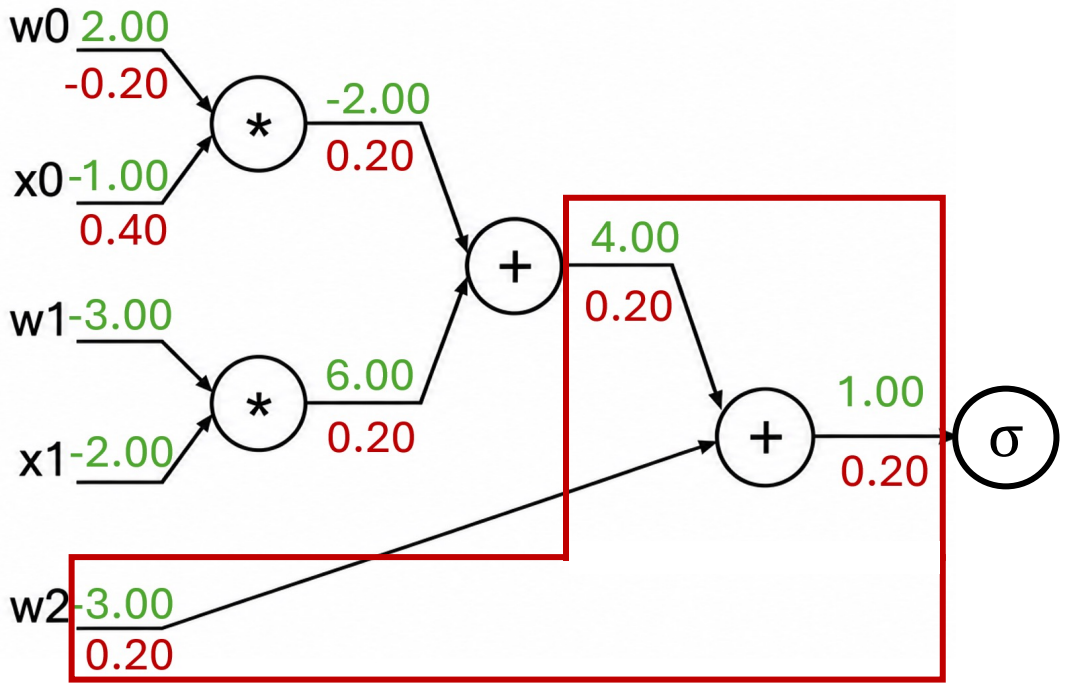
前向传播：
计算输出

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Sigmoid

```
  grad_L = 1.0
  grad_s3 = grad_L *(1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

反向传播实现：“展开”代码



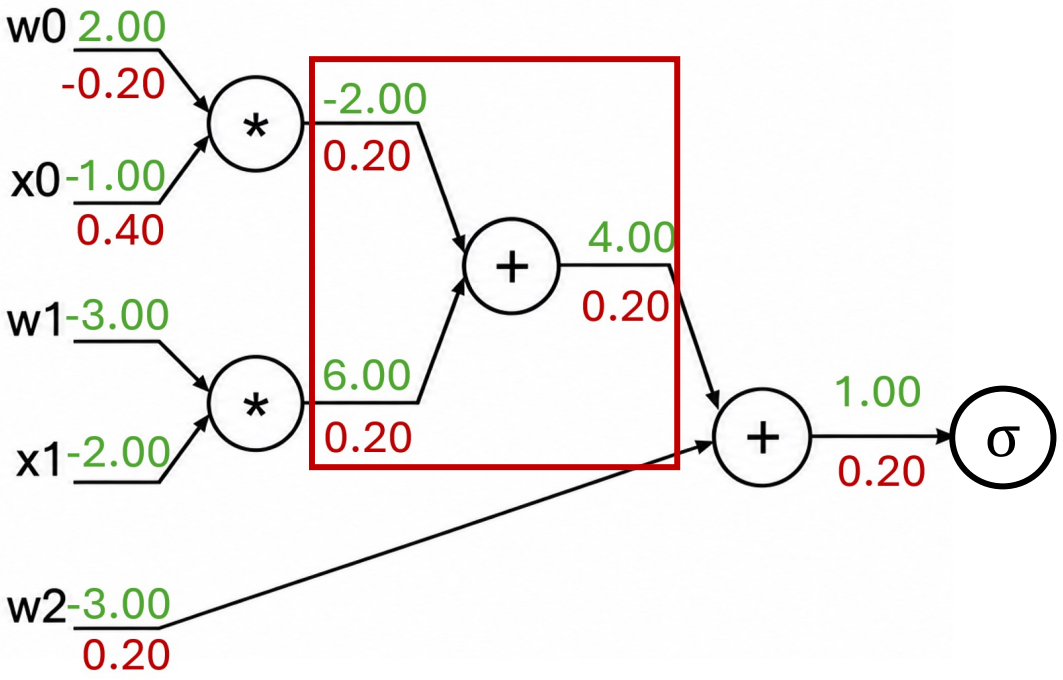
前向传播：
计算输出

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

加法门

```
grad_L = 1.0
grad_s3 = grad_L *(1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_s0 = grad_s2
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```

反向传播实现：“展开”代码



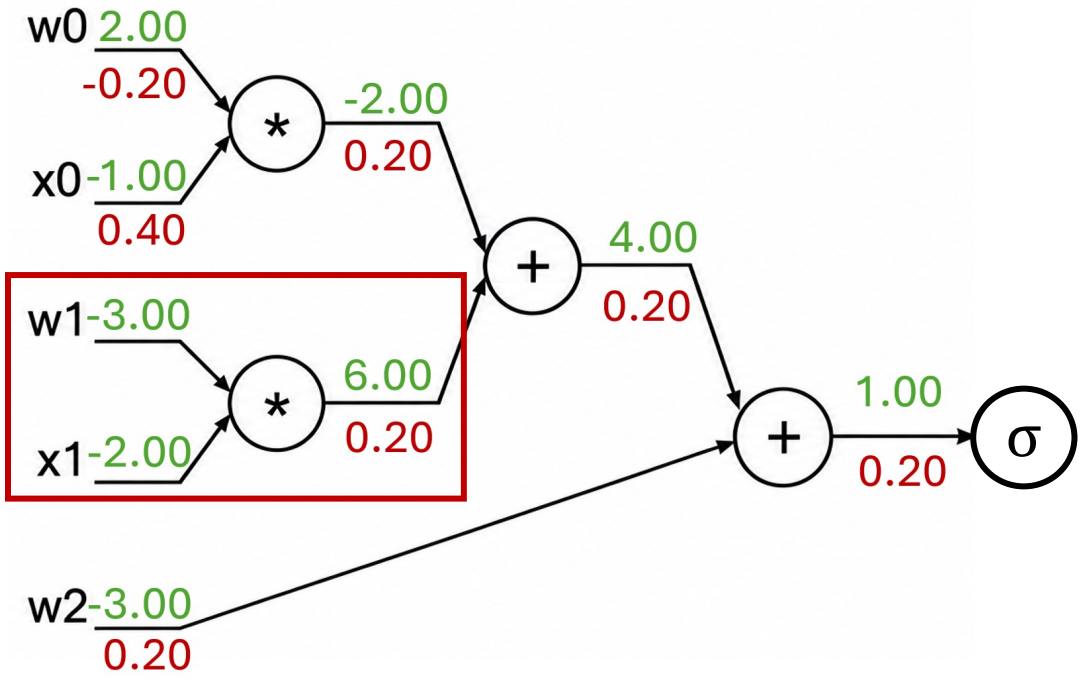
前向传播：
计算输出

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

加法门

```
grad_L = 1.0
grad_s3 = grad_L *(1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_s0 = grad_s2
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```

反向传播实现：“展开”代码



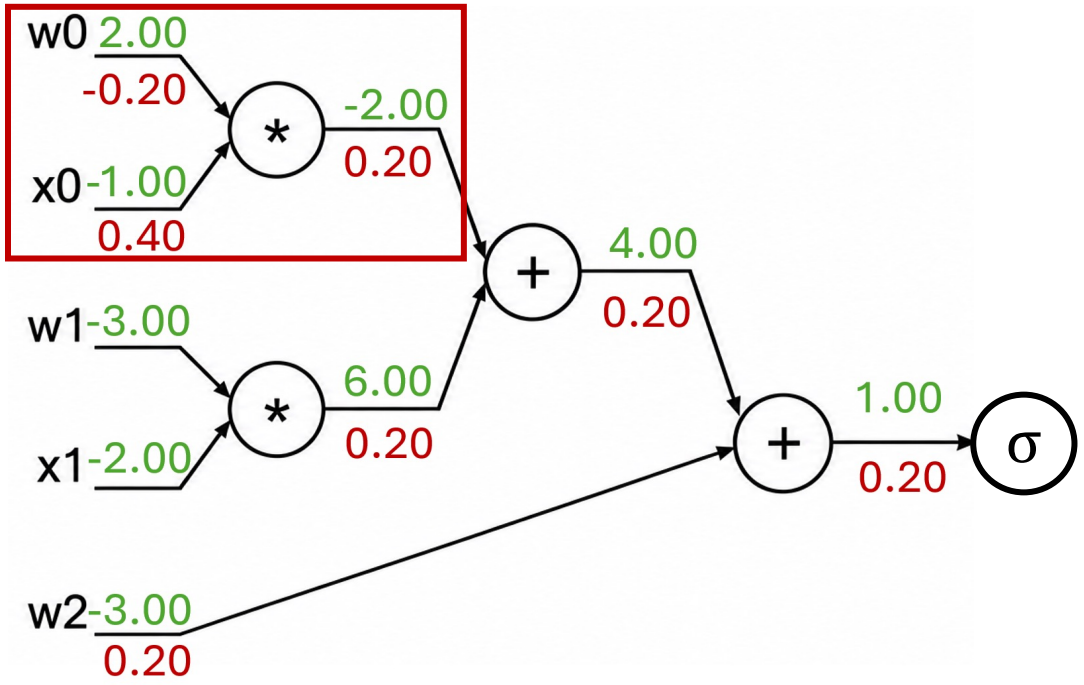
前向传播：
计算输出

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

乘法门

```
  grad_L = 1.0
  grad_s3 = grad_L *(1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

反向传播实现：“展开”代码



前向传播：
计算输出

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

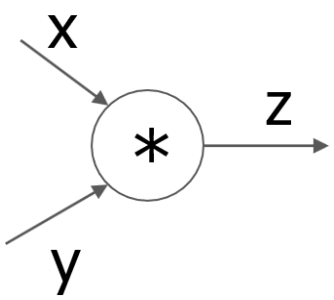
```
grad_L = 1.0
grad_s3 = grad_L *(1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_s0 = grad_s2
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```

乘法门

某个节点有多条下游分支汇入，该如何计算呢?

模块化实现：前向/反向API

- 实际 PyTorch 代码：乘法门的前向与反向实现



(x,y,z are scalars)

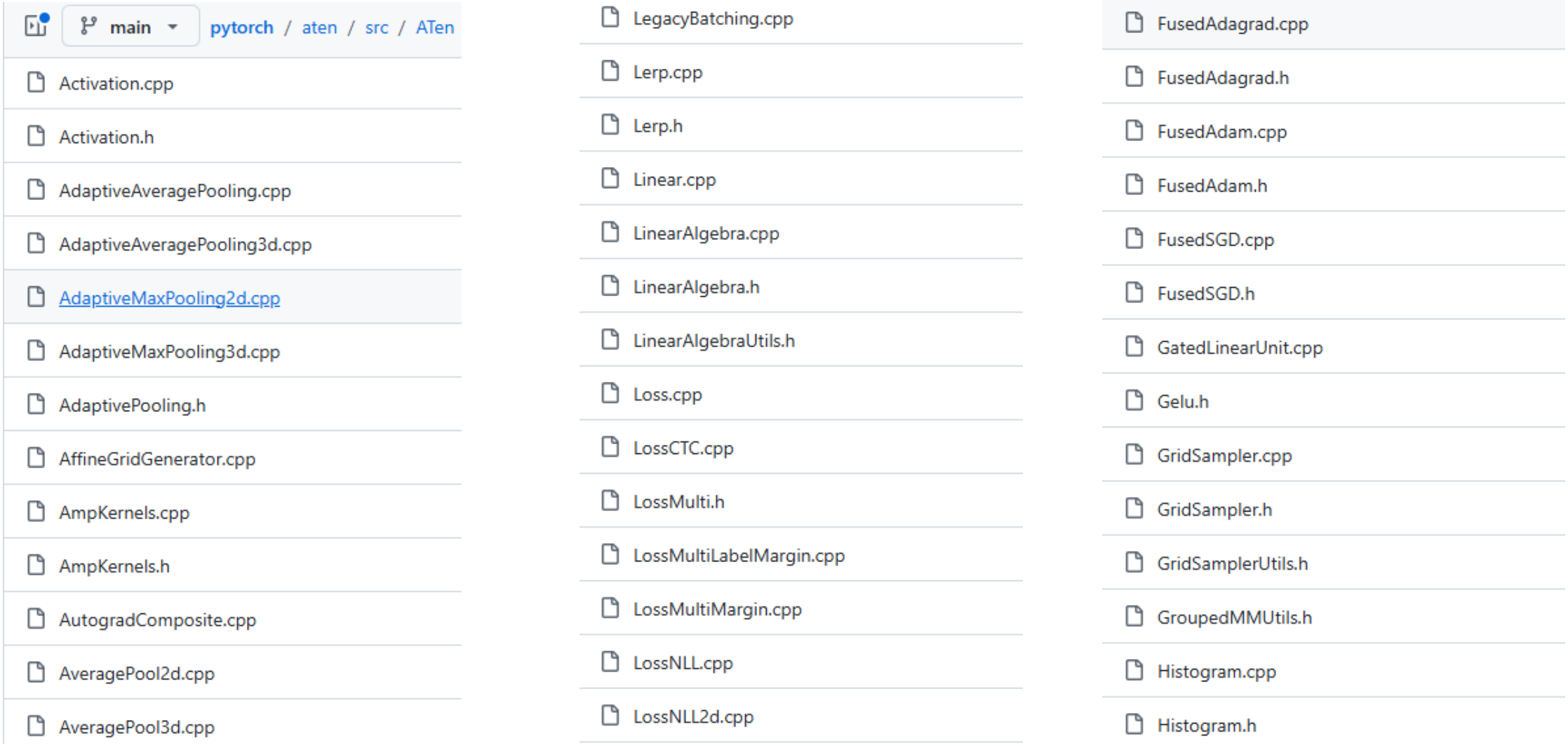
```
class Multiply(torch.autograd.Function):
  @staticmethod
  def forward(ctx, x, y):
    ctx.save_for_backward(x, y)
    z = x * y
    return z
  @staticmethod
  def backward(ctx, grad_z):
    x, y = ctx.saved_tensors
    grad_x = y * grad_z    # dz/dx * dL/dz
    grad_y = x * grad_z    # dz/dy * dL/dz
    return grad_x, grad_y
```

缓存前向计算值，用于反向传播

上游梯度

上游梯度 × 局部梯度

示例：PyTorch 运算符



pytorch/aten/src/ATen/native at main · pytorch/pytorch

PyTorch Sigmoid 层

```
#ifndef TH_GENERIC_FILE
#define TH_GENERIC_FILE "THNN/generic/Sigmoid.c"
#else

void THNN_(Sigmoid_updateOutput)(
          THNNState *state,
          THTensor *input,
          THTensor *output)
{
  THTensor_(sigmoid)(output, input);
}

void THNN_(Sigmoid_updateGradInput)(
          THNNState *state,
          THTensor *gradOutput,
          THTensor *gradInput,
          THTensor *output)
{
  THNN_CHECK_NELEMENT(output, gradOutput);
  THTensor_(resizeAs)(gradInput, output);
  TH_TENSOR_APPLY3(scalar_t, gradInput, scalar_t, gradOutput, scalar_t, output,
    scalar_t z = *output_data;
    *gradInput_data = *gradOutput_data * (1. - z) * z;
  );
}

#endif
```

Forward

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

PyTorch Sigmoid 层

```
#ifndef TH_GENERIC_FILE
#define TH_GENERIC_FILE "THNN/generic/Sigmoid.c"
#else

void THNN_(Sigmoid_updateOutput)(
          THNNState *state,
          THTensor *input,
          THTensor *output)
{
  THTensor_(sigmoid)(output, input);
}

void THNN_(Sigmoid_updateGradInput)(
          THNNState *state,
          THTensor *gradOutput,
          THTensor *gradInput,
          THTensor *output)
{
  THNN_CHECK_NELEMENT(output, gradOutput);
  THTensor_(resizeAs)(gradInput, output);
  TH_TENSOR_APPLY3(scalar_t, gradInput, scalar_t, gradOutput, scalar_t, output,
    scalar_t z = *output_data;
    *gradInput_data = *gradOutput_data * (1. - z) * z;
  );
}

#endif
```

Forward

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

```
static void sigmoid_kernel(TensorIterator& iter) {
  AT_DISPATCH_FLOATING_TYPES(iter.dtype(), "sigmoid_cpu", [&]() {
    unary_kernel_vec(
        iter,
        [=](scalar_t a) -> scalar_t { return (1 / (1 + std::exp((-a)))); },
        [=](Vec256<scalar_t> a) {
          a = Vec256<scalar_t>((scalar_t(0)) - a;
          a = a.exp();
          a = Vec256<scalar_t>((scalar_t(1)) + a;
          a = a.reciprocal();
          return a;
        });
    });
}
```

```
return (1 / (1 + std::exp((-a))))
```

PyTorch Sigmoid 层

```
#ifndef TH_GENERIC_FILE
#define TH_GENERIC_FILE "THNN/generic/Sigmoid.c"
#else

void THNN_(Sigmoid_updateOutput)(
          THNNState *state,
          THTensor *input,
          THTensor *output)
{
  THTensor_(sigmoid)(output, input);
}

void THNN_(Sigmoid_updateGradInput)(
          THNNState *state,
          THTensor *gradOutput,
          THTensor *gradInput,
          THTensor *output)
{
  THNN_CHECK_NELEMENT(output, gradOutput);
  THTensor_(resizeAs)(gradInput, output);
  TH_TENSOR_APPLY3(scalar_t, gradInput, scalar_t, gradOutput, scalar_t, output,
    scalar_t z = *output_data;
    *gradInput_data = *gradOutput_data * (1. - z) * z;
  );
}

#endif
```

Forward

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

```
static void sigmoid_kernel(TensorIterator& iter) {
  AT_DISPATCH_FLOATING_TYPES(iter.dtype(), "sigmoid_cpu", [&]() {
    unary_kernel_vec(
        iter,
        [=](scalar_t a) -> scalar_t { return (1 / (1 + std::exp((-a)))); },
        [=](Vec256<scalar_t> a) {
          a = Vec256<scalar_t>((scalar_t(0)) - a;
          a = a.exp();
          a = Vec256<scalar_t>((scalar_t(1)) + a;
          a = a.reciprocal();
          return a;
        });
    });
}
```

backward

$$(1 - \sigma(x))\sigma(x)$$

提纲

- 计算图与反向传播
- **向量与张量的反向传播**
- 神经网络训练中的挑战
- 数据流形与模型泛化

向量场景下的梯度计算

- 到目前为止：我们讨论的都是标量的反向传播
- 那输出是向量时，梯度怎么计算?

回顾：导数

标量到标量

- 输入与输出均为标量

$$x \in \mathbb{R}, y \in \mathbb{R}$$

- 导数

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

- 直观理解

如果 x 发生微小变化，
y会变化多少？

回顾：导数

标量到标量

- 输入与输出均为标量
  $x \in \mathbb{R}, y \in \mathbb{R}$
- 导数
  $$\frac{\partial y}{\partial x} \in \mathbb{R}$$
- 直观理解

如果 x 发生微小变化，y会变化多少？

向量到标量

- 向量输入，标量输出
  $\boldsymbol{x} \in \mathbb{R}^N, y \in \mathbb{R}$
- 梯度
  $$\frac{\partial y}{\partial \boldsymbol{x}} \in \mathbb{R}^N, \left(\frac{\partial y}{\partial \boldsymbol{x}}\right)_n = \frac{\partial y}{\partial x_n}$$
- 直观理解

对于 $\boldsymbol{x}$ 的每个元素，如果它发生微小变化，y 会变化多少？

回顾：导数

标量到标量

- 输入与输出均为标量
  $x \in \mathbb{R}, y \in \mathbb{R}$
- 导数
  $$\frac{\partial y}{\partial x} \in \mathbb{R}$$
- 直观理解

如果 x 发生微小变化，y会变化多少？

向量到标量

- 向量输入，标量输出
  $\boldsymbol{x} \in \mathbb{R}^N, y \in \mathbb{R}$
- 梯度
  $$\frac{\partial y}{\partial \boldsymbol{x}} \in \mathbb{R}^N, \left(\frac{\partial y}{\partial \boldsymbol{x}}\right)_n = \frac{\partial y}{\partial x_n}$$
- 直观理解

对于 $\boldsymbol{x}$ 的每个元素，如果它发生微小变化，y 会变化多少？

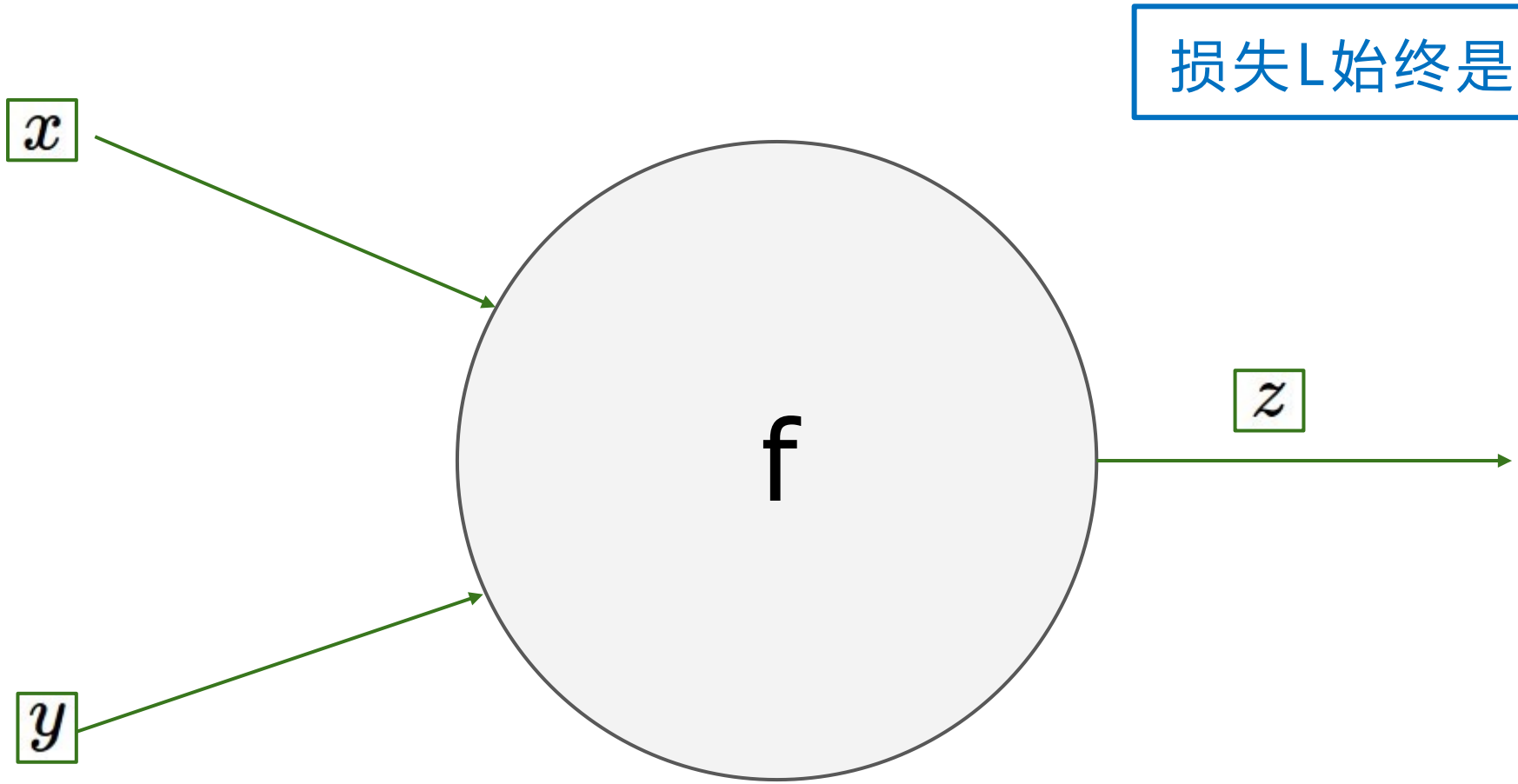
向量到向量

- 输入与输出均为向量
  $\boldsymbol{x} \in \mathbb{R}^N, \boldsymbol{y} \in \mathbb{R}^M$
- 雅可比矩阵
  $$\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} \in \mathbb{R}^{M \times N}, \left(\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}}\right)_{m,n} = \frac{\partial y_m}{\partial x_n}$$
- 直观理解

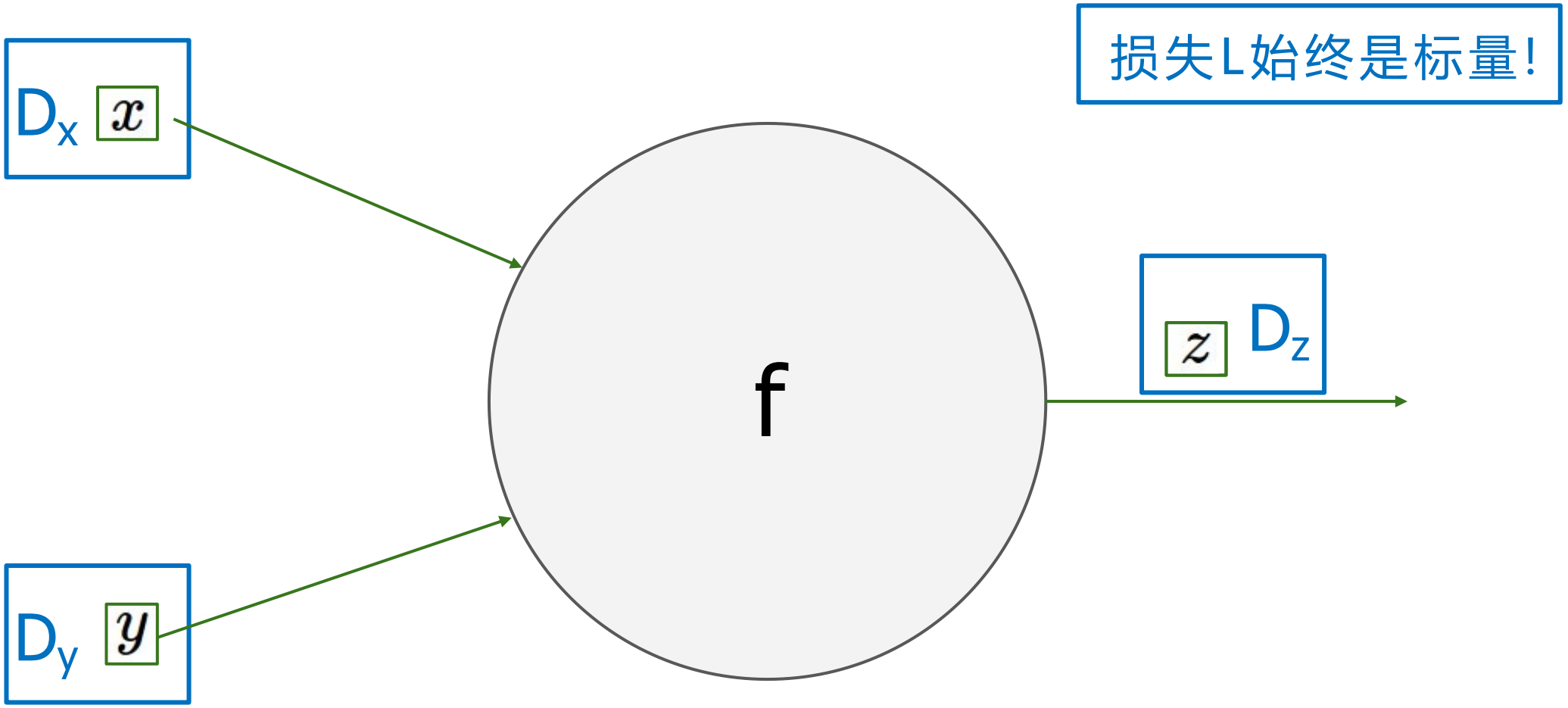
对于 $\boldsymbol{x}$ 的每个元素，如果它发生微小变化，$\boldsymbol{y}$ 的每个元素会变化多少？

The Matrix Cookbook. https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf

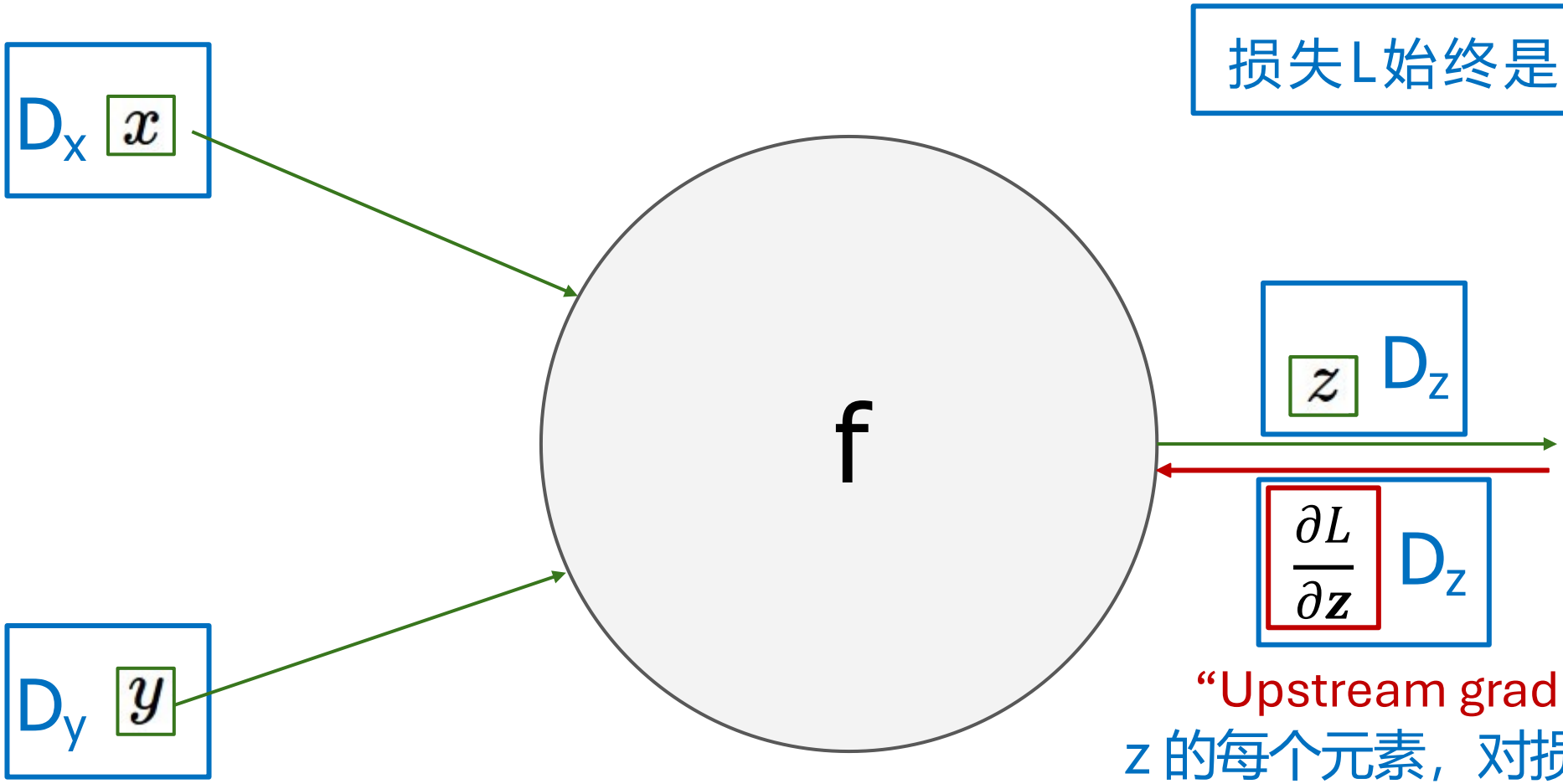
向量的反向传播



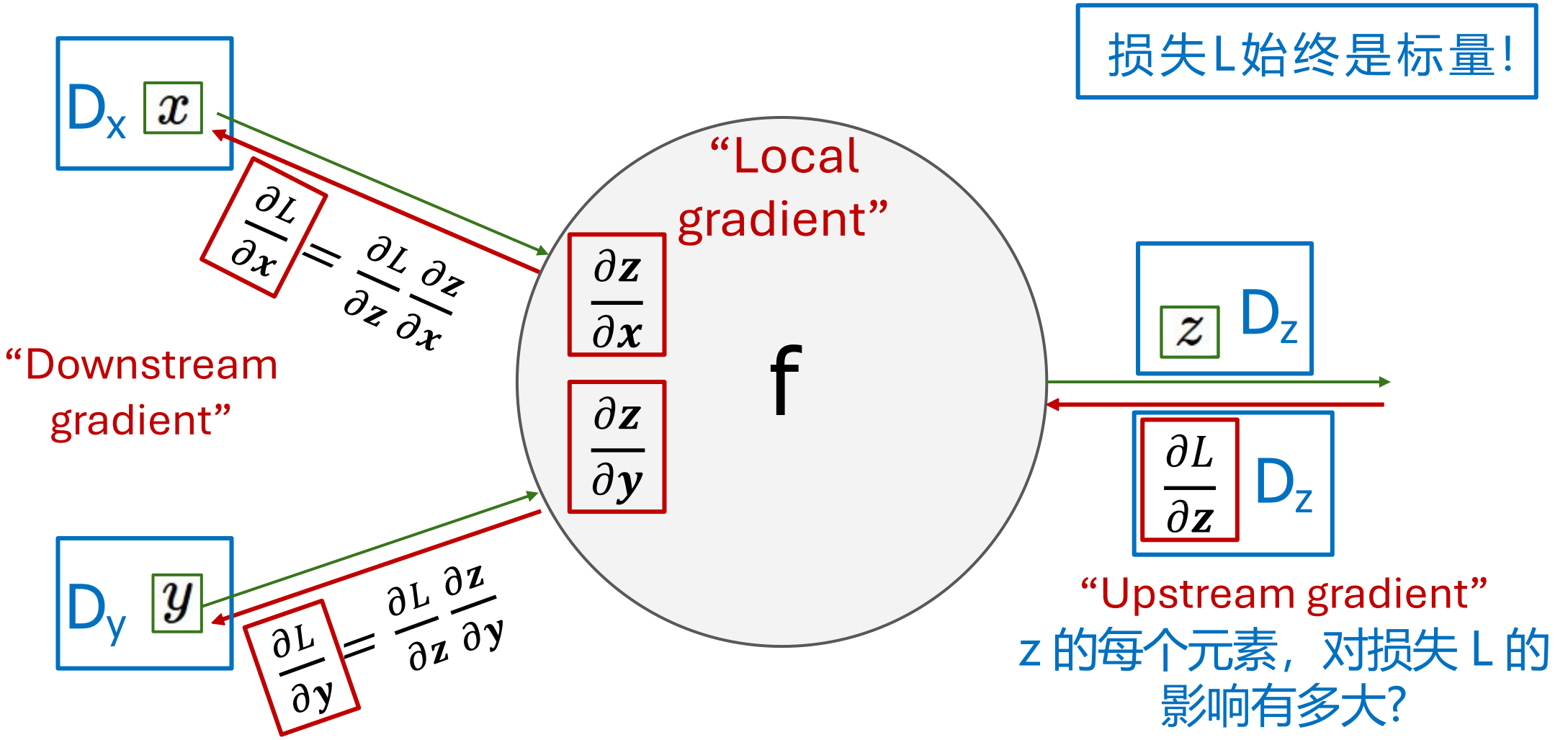
向量的反向传播



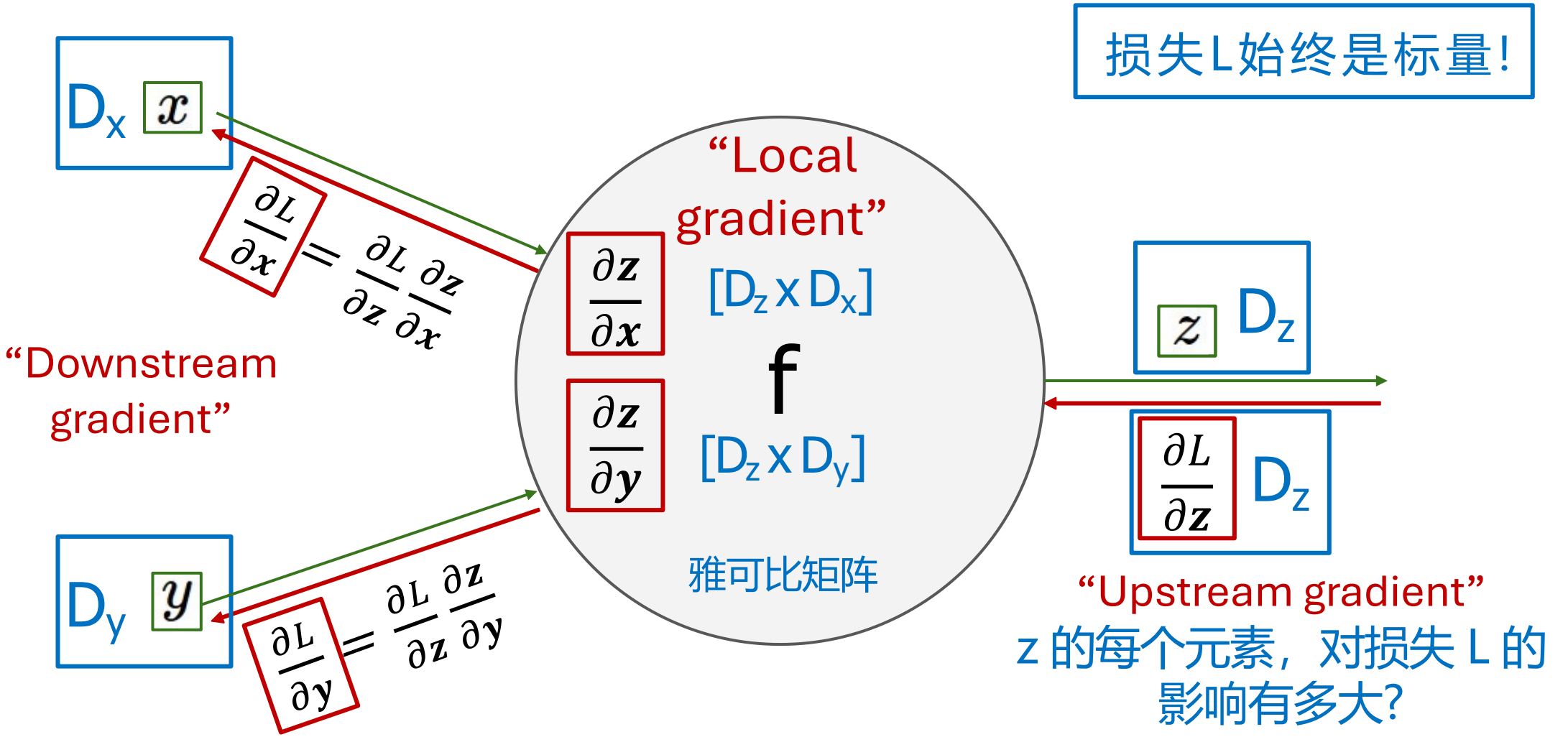
向量的反向传播


损失L始终是标量!
D_x x
D_y y
f
z D_z
$\frac{\partial L}{\partial z}$ D_z
“Upstream gradient”
z 的每个元素，对损失 L 的影响有多大?

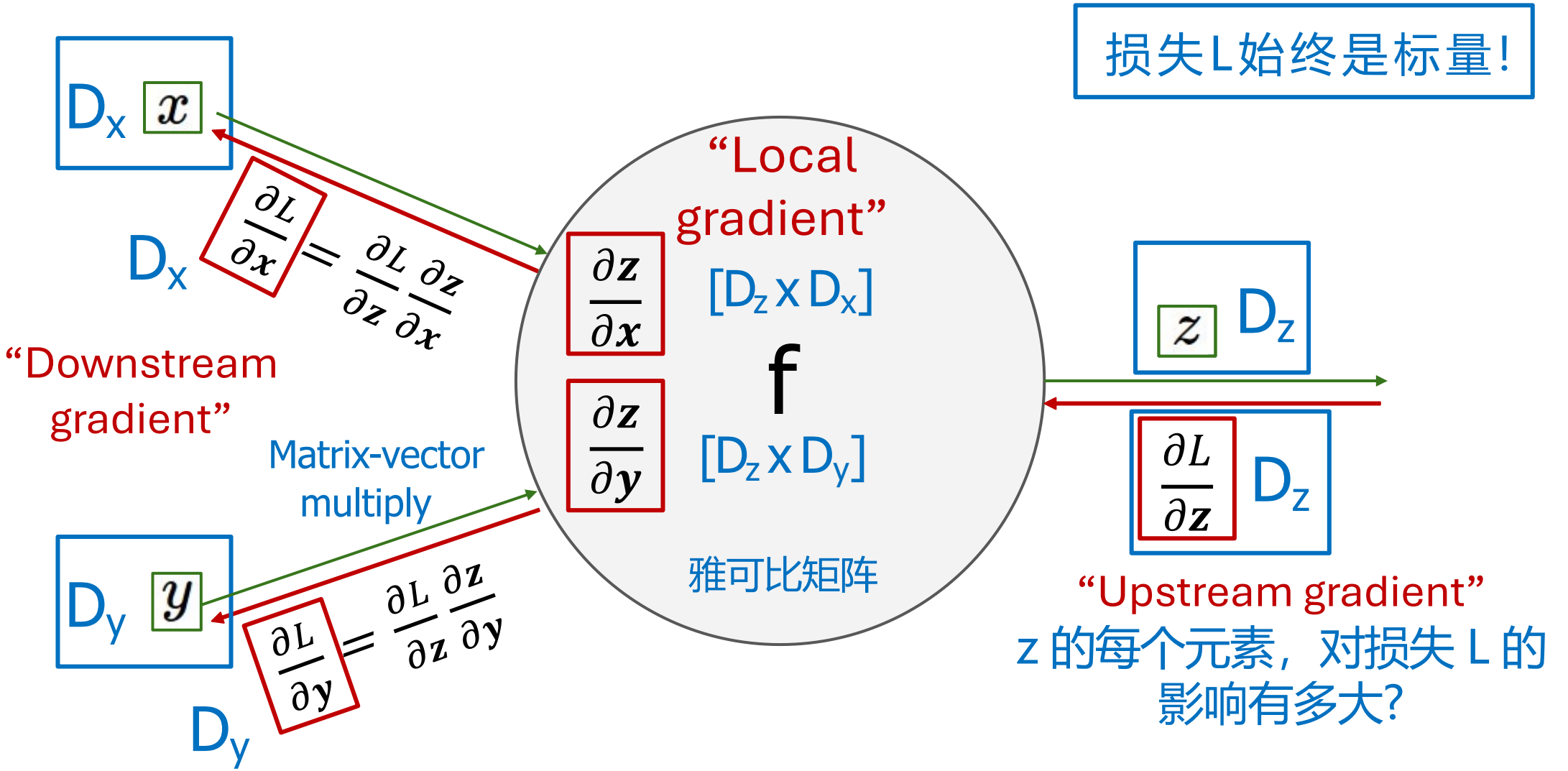
向量的反向传播


损失L始终是标量!
D_x x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
“Downstream gradient”
“Local gradient”
$\frac{\partial z}{\partial x}$
$\frac{\partial z}{\partial y}$
f
z D_z
$\frac{\partial L}{\partial z}$ D_z
D_y y
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$
“Upstream gradient”
z 的每个元素，对损失 L 的影响有多大?

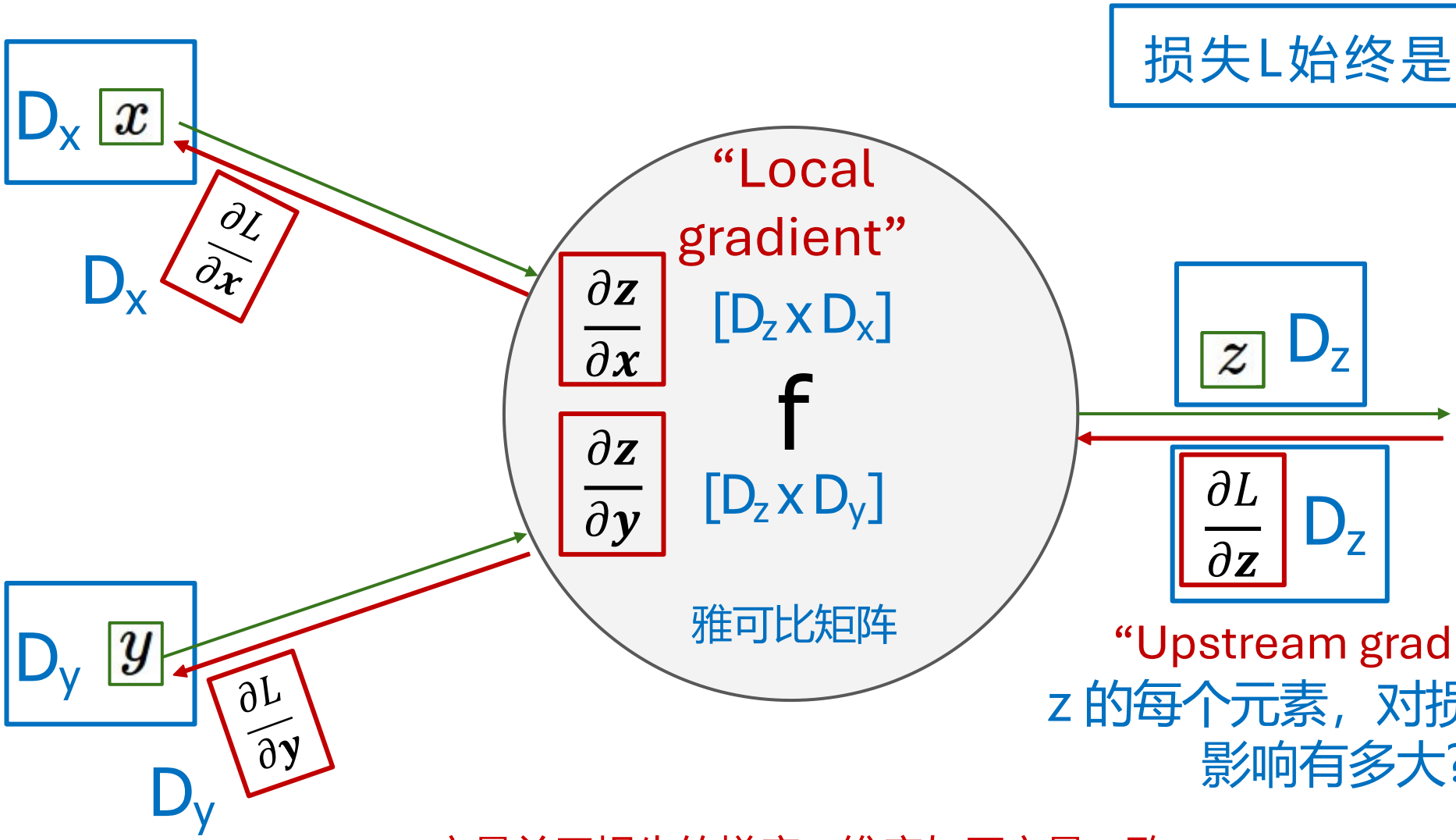
向量的反向传播


损失L始终是标量!
D_x x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
"Downstream gradient"
"Local gradient"
$\frac{\partial z}{\partial x}$
$[D_z \times D_x]$
f
$\frac{\partial z}{\partial y}$
$[D_z \times D_y]$
雅可比矩阵
z D_z
$\frac{\partial L}{\partial z}$ D_z
"Upstream gradient"
z 的每个元素，对损失 L 的影响有多大?
D_y y
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$

向量的反向传播

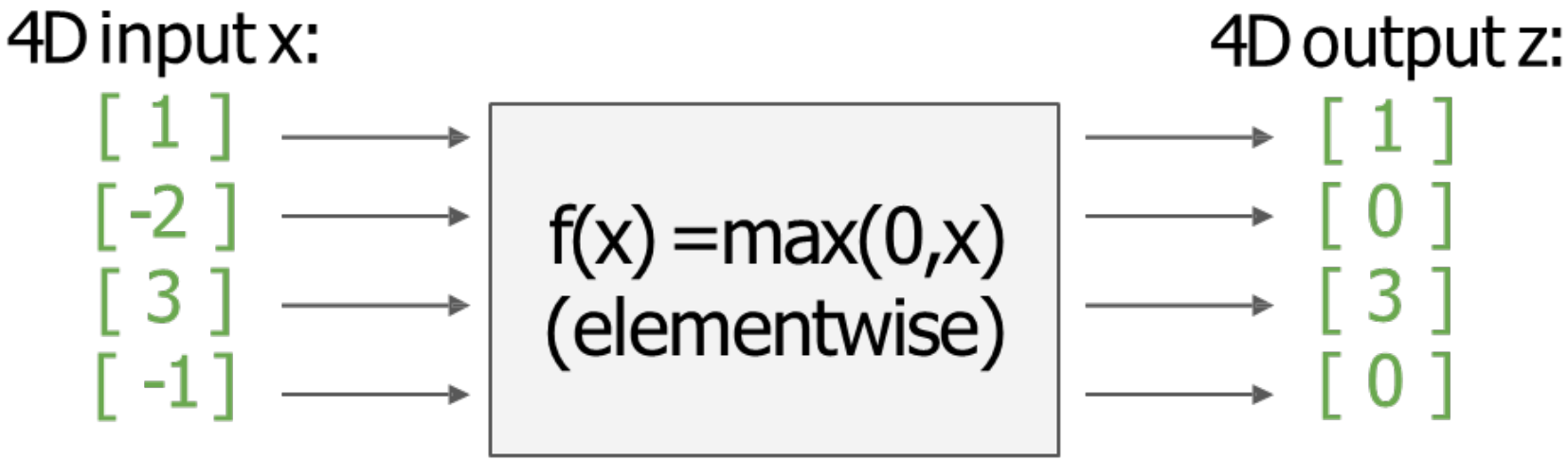

损失L始终是标量!
D_x x
D_x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
“Downstream gradient”
“Local gradient”
$\frac{\partial z}{\partial x}$
$[D_z \times D_x]$
f
$\frac{\partial z}{\partial y}$
$[D_z \times D_y]$
雅可比矩阵
Matrix-vector multiply
D_y y
D_y
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$
z D_z
$\frac{\partial L}{\partial z}$ D_z
“Upstream gradient”
z 的每个元素，对损失 L 的影响有多大?

向量的反向传播

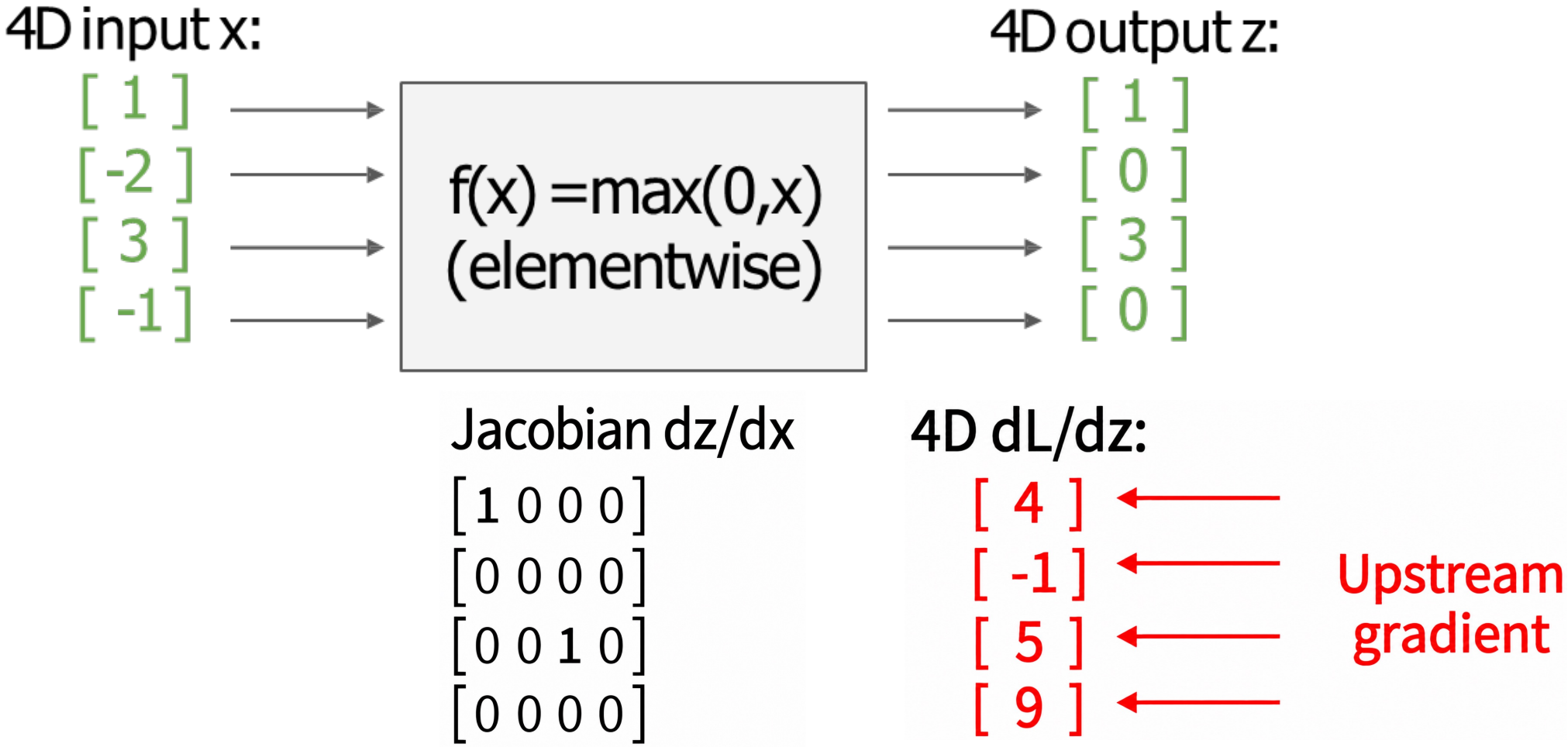


变量关于损失的梯度，维度与原变量一致

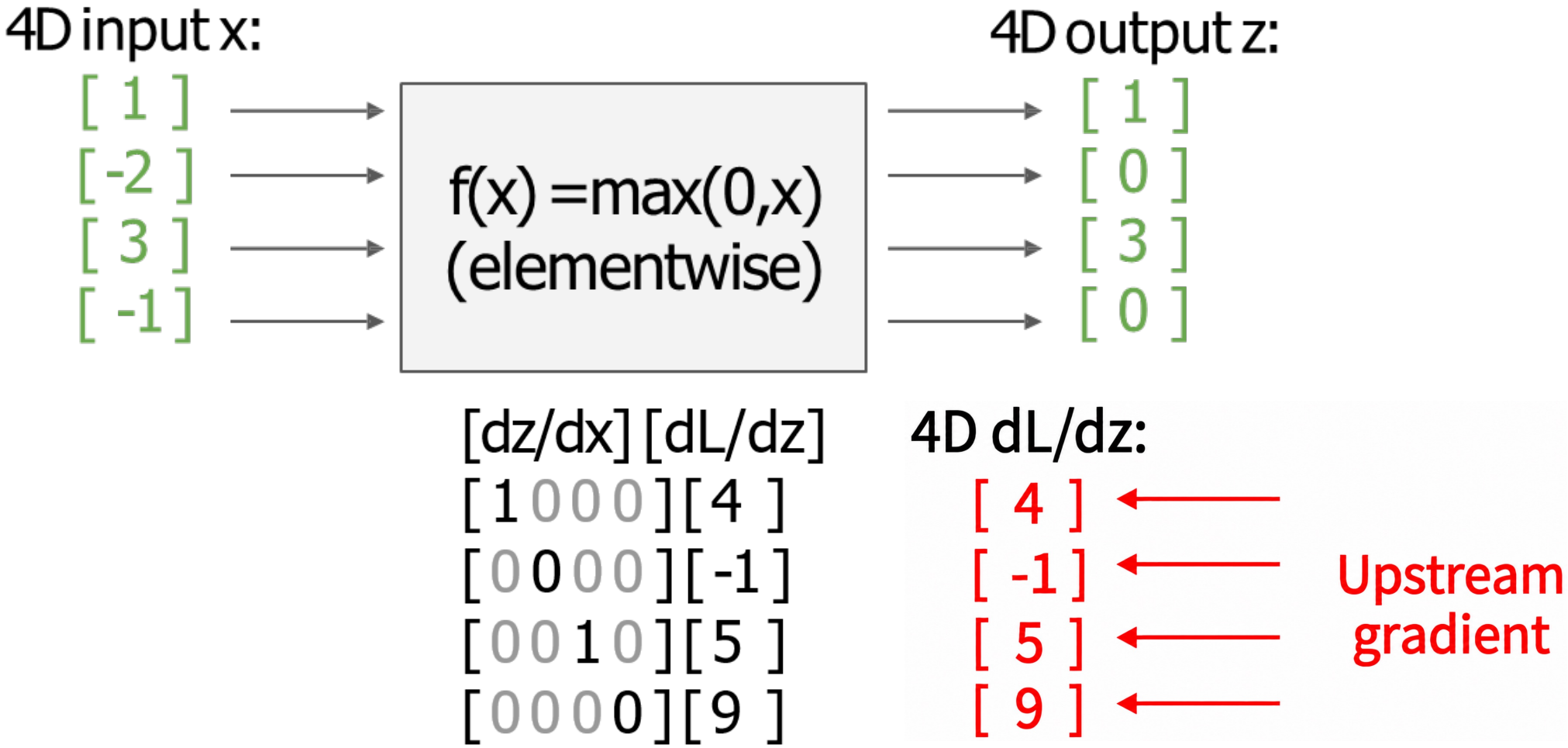
向量的反向传播



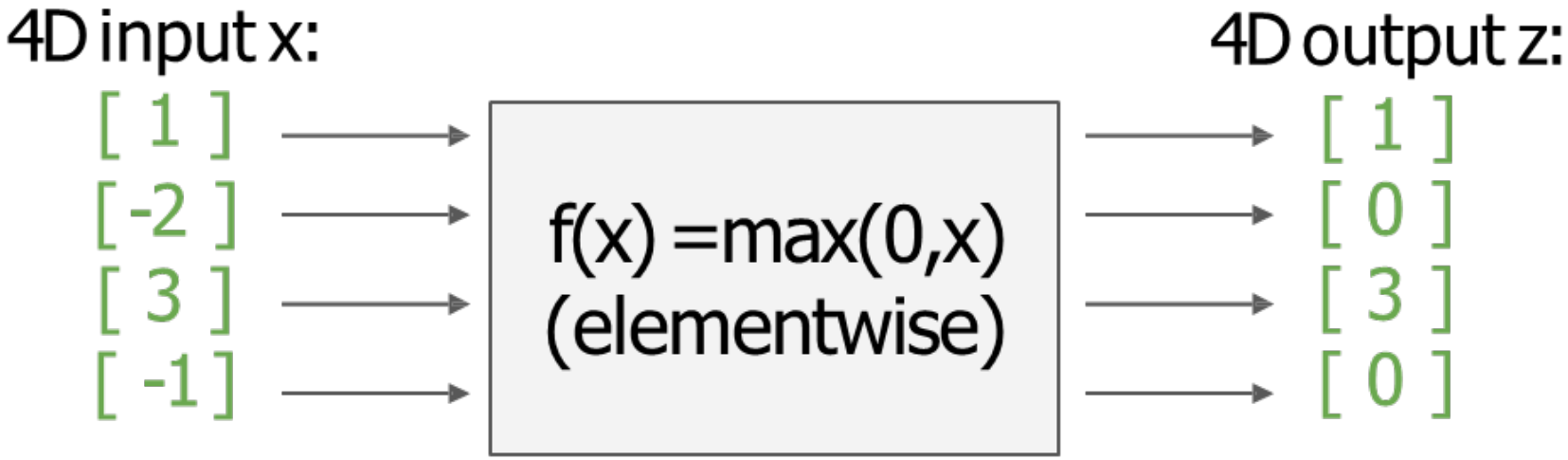
向量的反向传播


4D input x:
[1]
[-2]
[3]
[-1]
f(x) =max(0,x)
(elementwise)
4D output z:
[1]
[0]
[3]
[0]
Jacobian dz/dx
[1 0 0 0]
[0 0 0 0]
[0 0 1 0]
[0 0 0 0]
4D dL/dz:
[4]
[-1]
[5]
[9]
Upstream gradient

向量的反向传播


4D input x:
[1]
[-2]
[3]
[-1]
f(x) =max(0,x)
(elementwise)
4D output z:
[1]
[0]
[3]
[0]
[dz/dx] [dL/dz]
[1000][4]
[0000][-1]
[0010][5]
[0000][9]
4D dL/dz:
[4]
[-1]
[5]
[9]
Upstream gradient

向量的反向传播



4D dL/dx:

[4] ←
[0] ←
[5] ←
[0] ←

[dz/dx] [dL/dz]

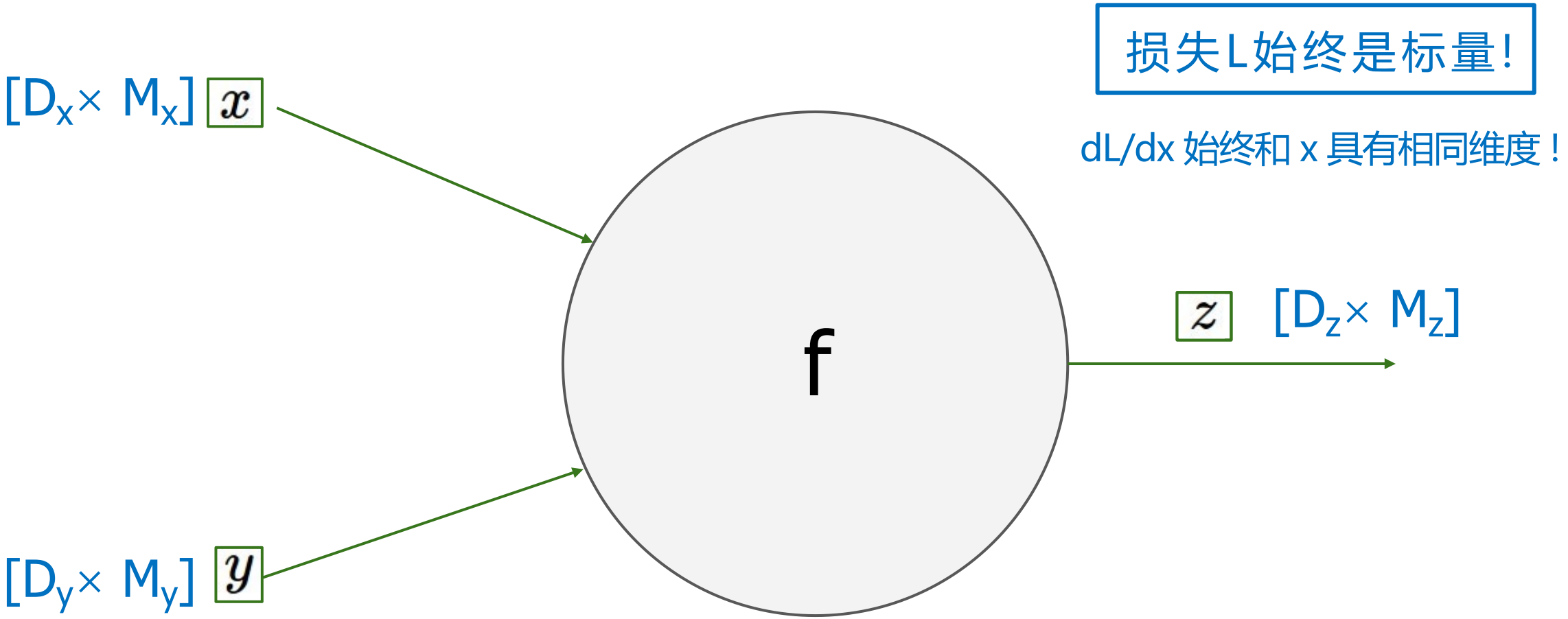
$$\left(\frac{\partial L}{\partial x}\right)_i = \begin{cases} \left(\frac{\partial L}{\partial z}\right)_i & \text{if } x_i > 0 \\ 0 & \text{otherwise} \end{cases}$$

4D dL/dz:

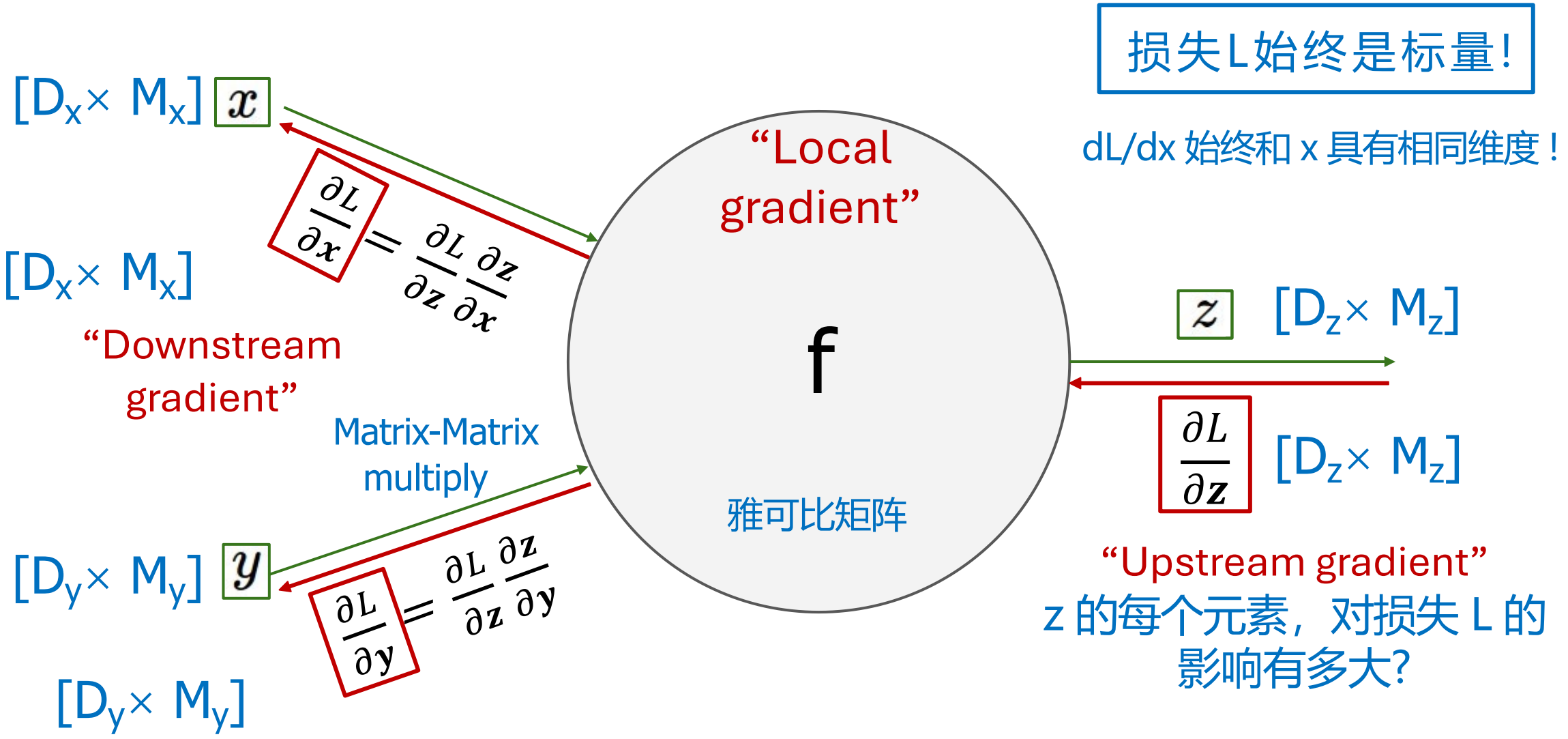
[4] ←
[-1] ←
[5] ←
[9] ←

Upstream gradient

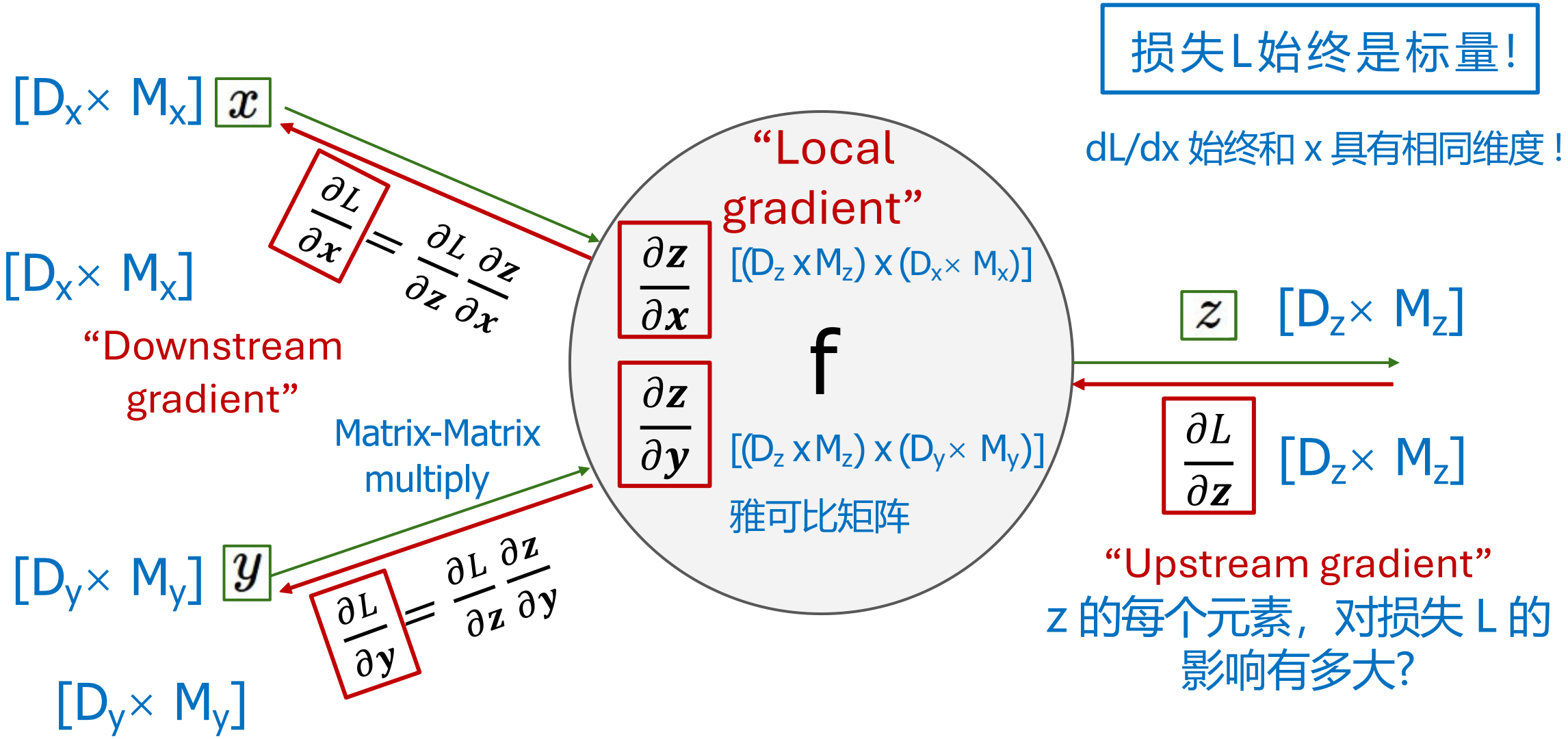
矩阵 (张量) 的反向传播



矩阵 (张量) 的反向传播


损失L始终是标量!
dL/dx 始终和 x 具有相同维度 !
[Dx× Mx]
x
[Dx× Mx]
“Downstream gradient”
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
“Local gradient”
f
雅可比矩阵
z
[Dz× Mz]
$\frac{\partial L}{\partial z}$
[Dz× Mz]
“Upstream gradient”
z 的每个元素，对损失 L 的影响有多大?
Matrix-Matrix multiply
[Dy× My]
y
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$
[Dy× My]

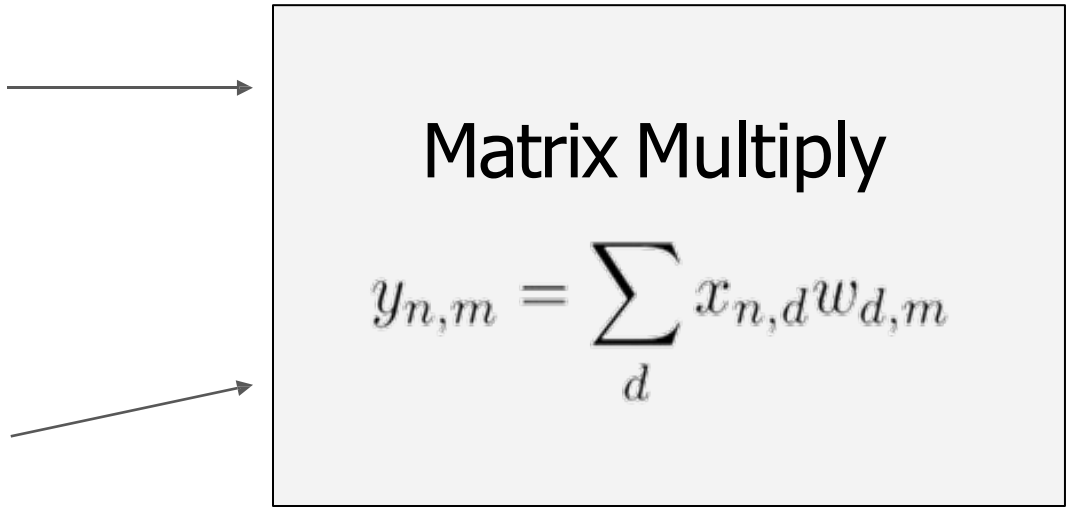
矩阵 (张量) 的反向传播


损失L始终是标量!
dL/dx 始终和 x 具有相同维度 !
[Dx× Mx] x
$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial x}$
[Dx× Mx]
“Downstream gradient”
“Local gradient”
$\frac{\partial z}{\partial x}$ [(Dz x Mz) x (Dx× Mx)]
f
$\frac{\partial z}{\partial y}$ [(Dz x Mz) x (Dy× My)]
雅可比矩阵
z [Dz× Mz]
$\frac{\partial L}{\partial z}$ [Dz× Mz]
“Upstream gradient”
z 的每个元素，对损失 L 的影响有多大?
Matrix-Matrix multiply
[Dy× My] y
$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial y}$
[Dy× My]

矩阵乘法的反向传播

x: [N×D]
[2 -1 3]
[-3 4 2]

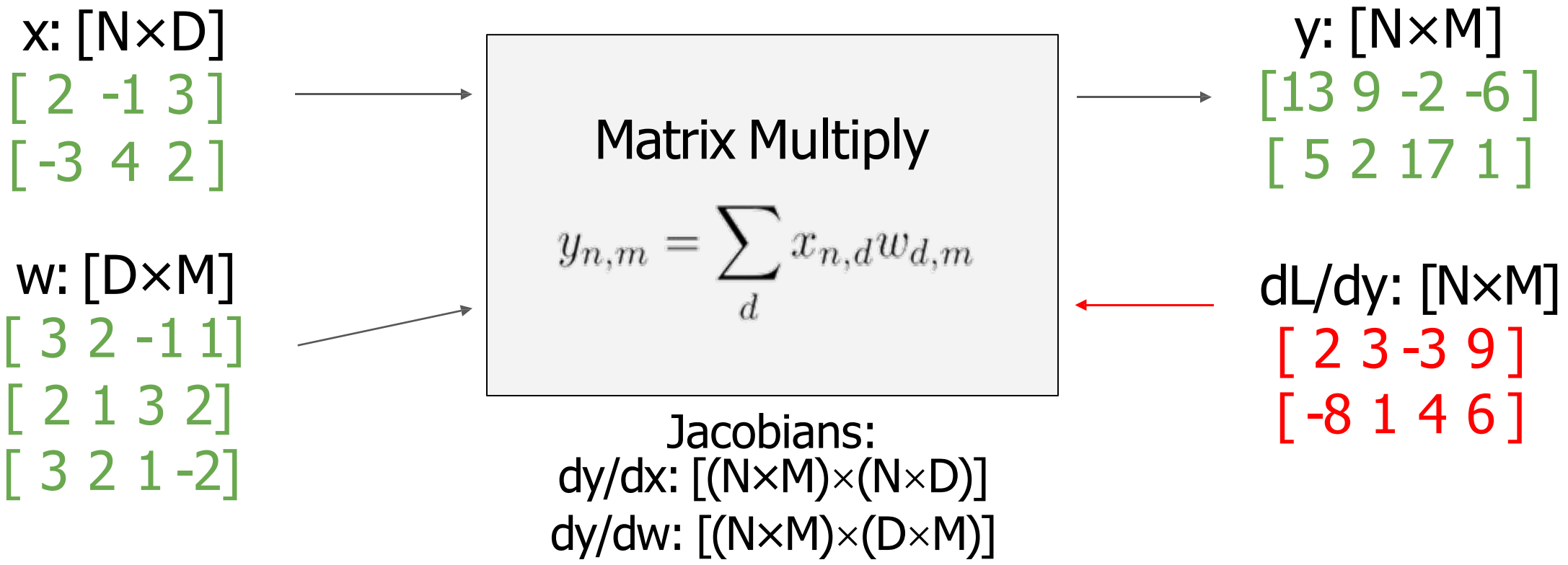
w: [D×M]
[3 2 -1 1]
[2 1 3 2]
[3 2 1 -2]



y: [N×M]
[13 9 -2 -6]
[5 2 17 1]

dL/dy: [N×M]
[2 3 -3 9]
[-8 1 4 6]

矩阵乘法的反向传播

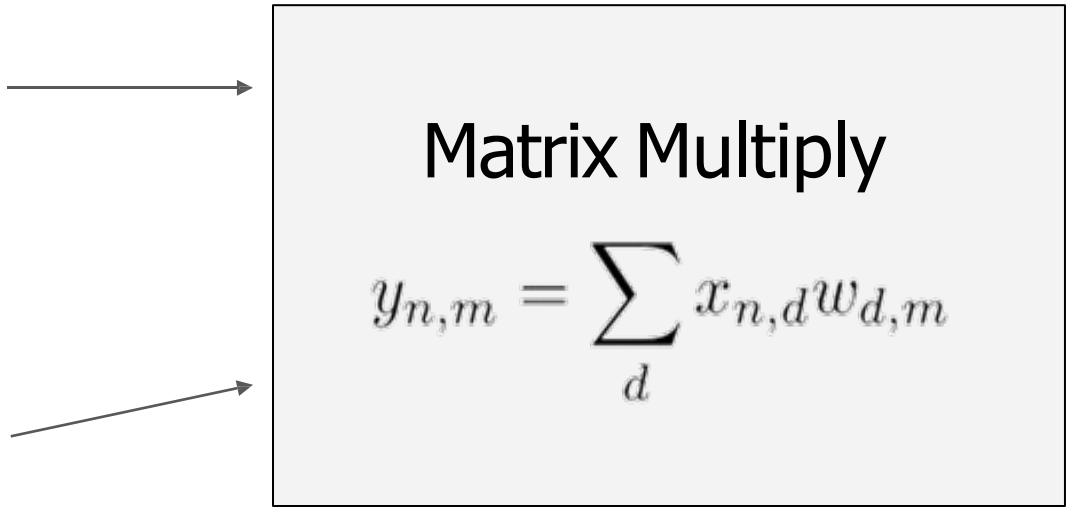


- 对于一个神经网络，我们可能有 N=64, D=M=4096， dy/dx需要约 256 GB 的内存， $dy/d\mathbf{w}$需要约 16 TB 的内存!
- 必须以隐式方式处理它们!

矩阵乘法的反向传播

x: [N×D]
[2 -1 3]
[-3 4 2]

w: [D×M]
[3 2 -1 1]
[2 1 3 2]
[3 2 1 -2]



y: [N×M]
[13 9 -2 -6]
[5 2 17 1]

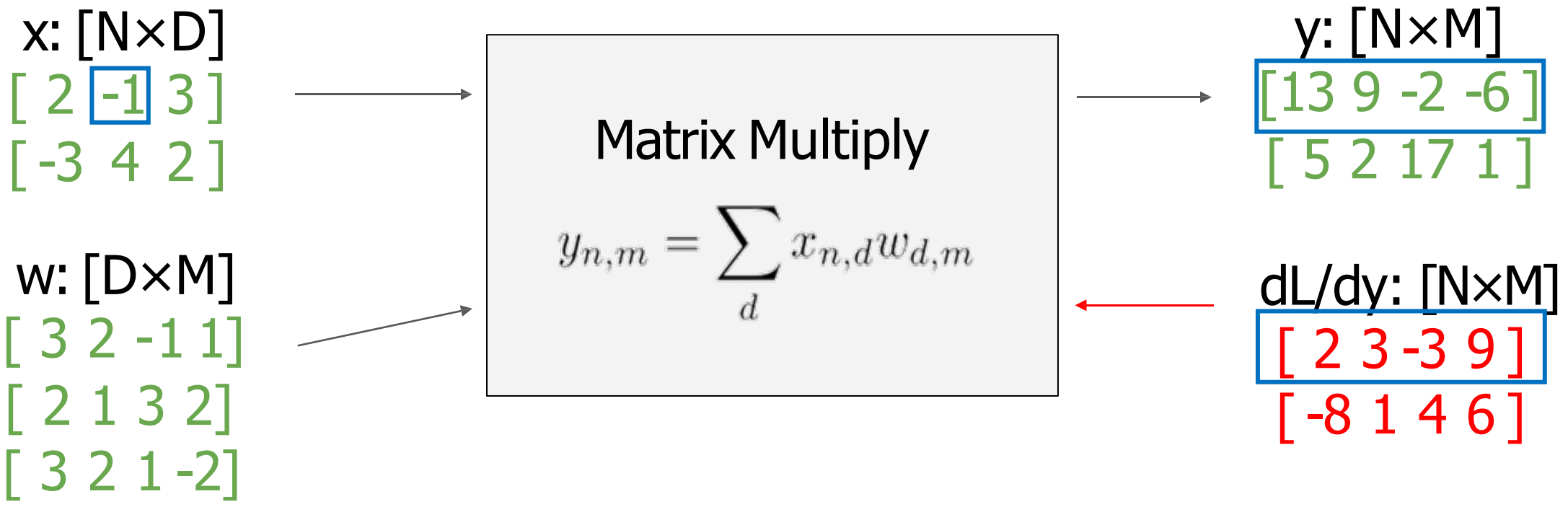
dL/dy: [N×M]
[2 3 -3 9]
[-8 1 4 6]

问：y 的哪些部分会受到 x 中某个元素的影响?

答：$x_{n,d}$ 影响整行 $y_{n,\cdot}$

$$\frac{\partial L}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} \frac{\partial y_{n,m}}{\partial x_{n,d}}$$

矩阵乘法的反向传播



问：$\boldsymbol{y}$ 的哪些部分会受到 $\boldsymbol{x}$ 中某个元素的影响?

答：$x_{n,d}$ 影响整行 $y_{n,\cdot}$

$$\frac{\partial L}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} \frac{\partial y_{n,m}}{\partial x_{n,d}}$$

矩阵乘法的反向传播

x: [N×D]

[2 -1 3]

[-3 4 2]

w: [D×M]

[3 2 -1 1]

[2 1 3 2]

[3 2 1 -2]

Matrix Multiply

$$y_{n,m} = \sum_d x_{n,d} w_{d,m}$$

y: [N×M]

[13 9 -2 -6]

[5 2 17 1]

dL/dy: [N×M]

[2 3 -3 9]

[-8 1 4 6]

问：y 的哪些部分会受到 x 中某个元素的影响？

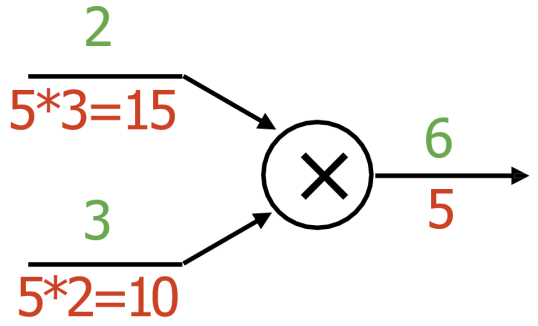
答：$x_{n,d}$ 影响整行 $y_{n,\cdot}$

$x_{n,d}$ 对 $y_{n,m}$ 影响是多少？

答：$w_{d,m}$

$$\frac{\partial L}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} \frac{\partial y_{n,m}}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} w_{d,m}$$

乘法门：“交换乘数”



矩阵乘法的反向传播

x: [N×D]

[2 -1 3]

[-3 4 2]

w: [D×M]

[3 2 -1 1]

[2 1 3 2]

[3 2 1 -2]

Matrix Multiply

$$y_{n,m} = \sum_d x_{n,d} w_{d,m}$$

y: [N×M]

[13 9 -2 -6]

[5 2 17 1]

dL/dy: [N×M]

[2 3 -3 9]

[-8 1 4 6]

问：y 的哪些部分会受到 x 中某个元素的影响？

答：$x_{n,d}$ 影响整行 $y_{n,\cdot}$

$x_{n,d}$ 对 $y_{n,m}$ 影响是多少？

答：$w_{d,m}$

[N×D] [N×M] [M×D]

$$\frac{\partial L}{\partial x} = \left(\frac{\partial L}{\partial y}\right) w^T$$

$$\frac{\partial L}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} \frac{\partial y_{n,m}}{\partial x_{n,d}} = \sum_m \frac{\partial L}{\partial y_{n,m}} w_{d,m}$$

矩阵乘法的反向传播

x: [N×D]

[2 -1 3]

[-3 4 2]

w: [D×M]

[3 2 -1 1]

[2 1 3 2]

[3 2 1 -2]

Matrix Multiply

$$y_{n,m} = \sum_d x_{n,d} w_{d,m}$$

y: [N×M]

[13 9 -2 -6]

[5 2 17 1]

dL/dy: [N×M]

[2 3 -3 9]

[-8 1 4 6]

这些公式很容易记住：它们是使形状匹配的唯一方式!

By similar logic:

[D×M] [D×N] [N×M]

$$\frac{\partial L}{\partial w} = x^T \left(\frac{\partial L}{\partial y} \right)$$

[N×D] [N×M] [M×D]

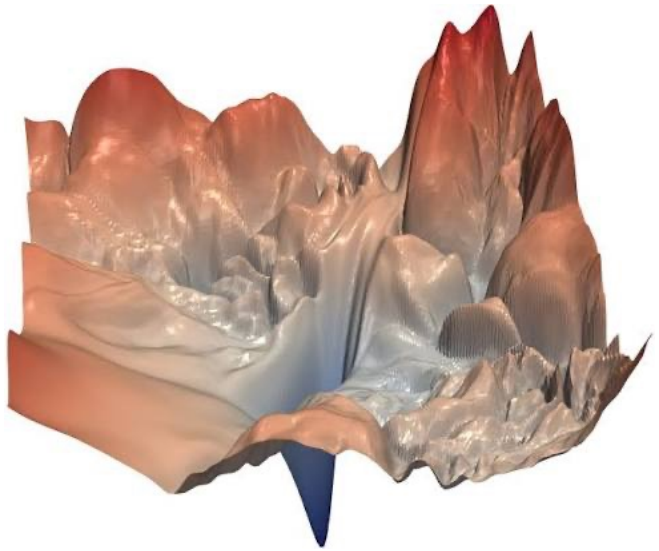
$$\frac{\partial L}{\partial x} = \left(\frac{\partial L}{\partial y} \right) w^T$$

反向传播总结

- 全连接神经网络是线性函数和非线性激活函数的堆叠；它们具有比线性分类器更强的表示能力
- 反向传播 = 沿计算图递归应用链式法则，以计算所有输入/参数/中间变量的梯度
- 实现维护了一个图结构，其中节点实现了forward() / backward() API
- 前向：计算操作的结果，并将梯度计算中需要的任何中间变量保存在内存中；
- 反向：应用链式法则，计算损失函数相对于输入的梯度

提纲

- 计算图与反向传播
- 向量与张量的反向传播
- **神经网络训练中的挑战**
  - 局部最小值与优化算法
  - 梯度消失
- 数据流形与模型泛化

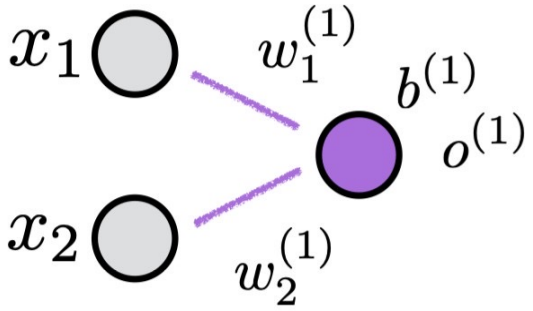


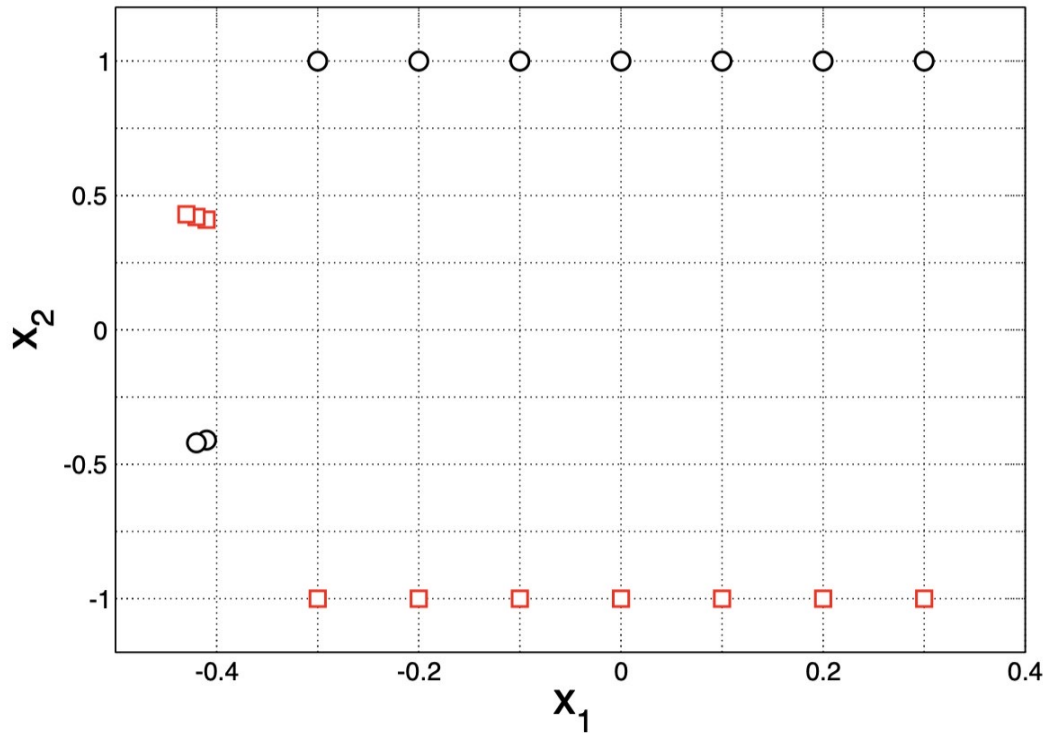
局部最优

$$o_i = \sigma(w_1 x_1 + w_2 x_2) \text{ with } w_2 = 1$$

$$\sigma(z) = \frac{1}{1+\exp(-z)}$$

$$C(w_1) = \frac{1}{n}\sum_i (y_i - o_i)^2$$



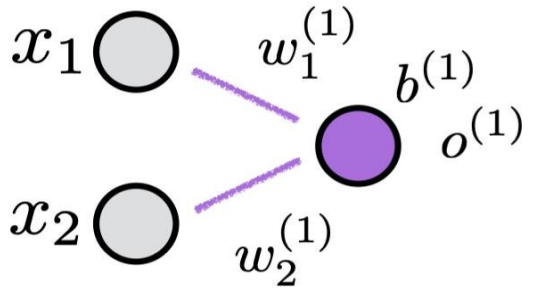


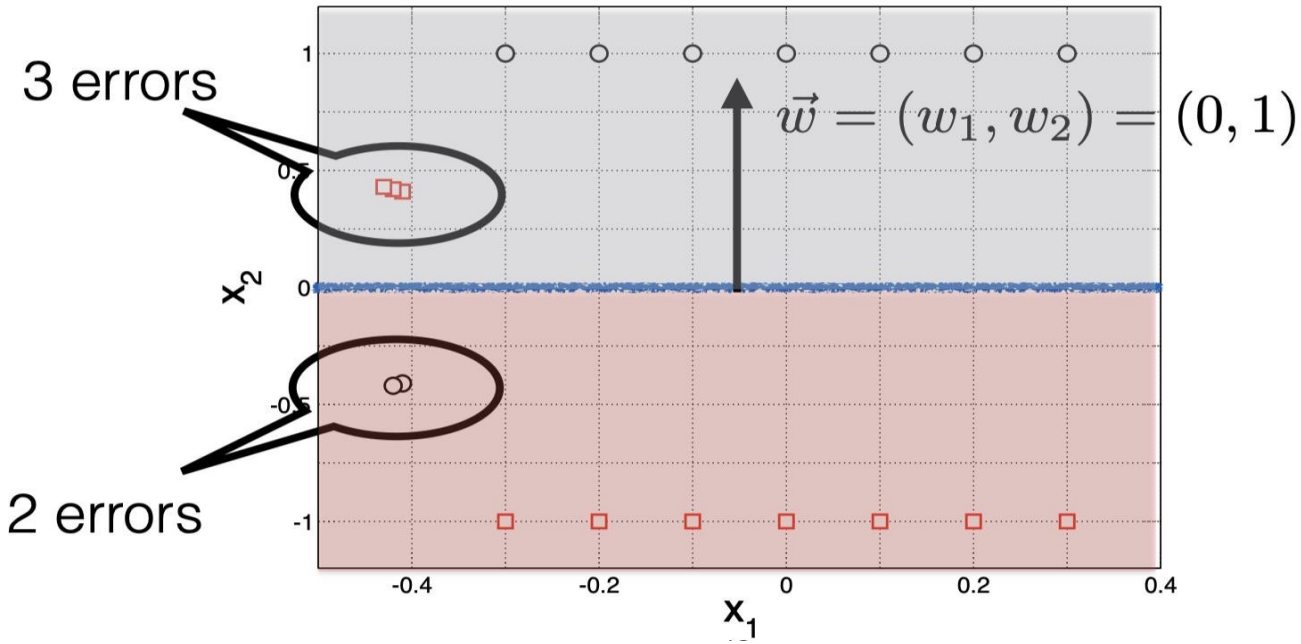
局部最优

$$o_i = \sigma(w_1 x_1 + w_2 x_2) \text{ with } w_2 = 1$$

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

$$C(w_1) = \frac{1}{n} \sum_i (y_i - o_i)^2$$

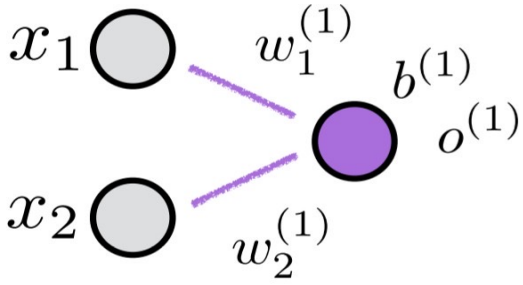




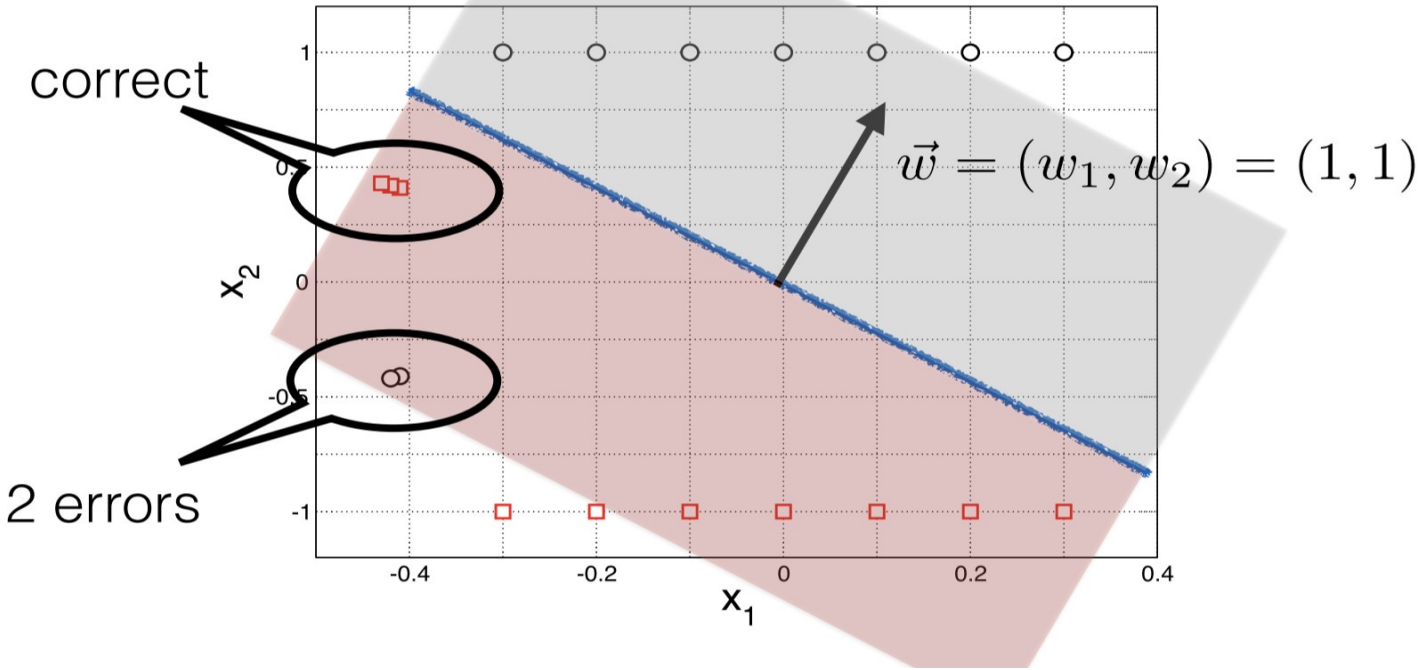
局部最优

$$o_i = \sigma(w_1 x_1 + w_2 x_2) \text{ with } w_2 = 1$$

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$



$$C(w_1) = \frac{1}{n} \sum_i (y_i - o_i)^2$$

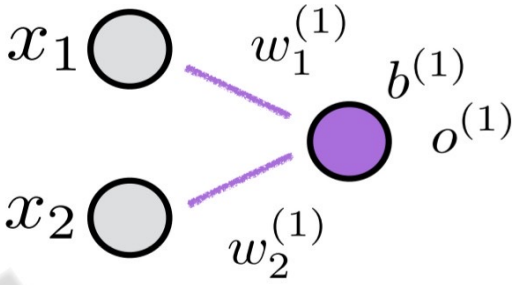


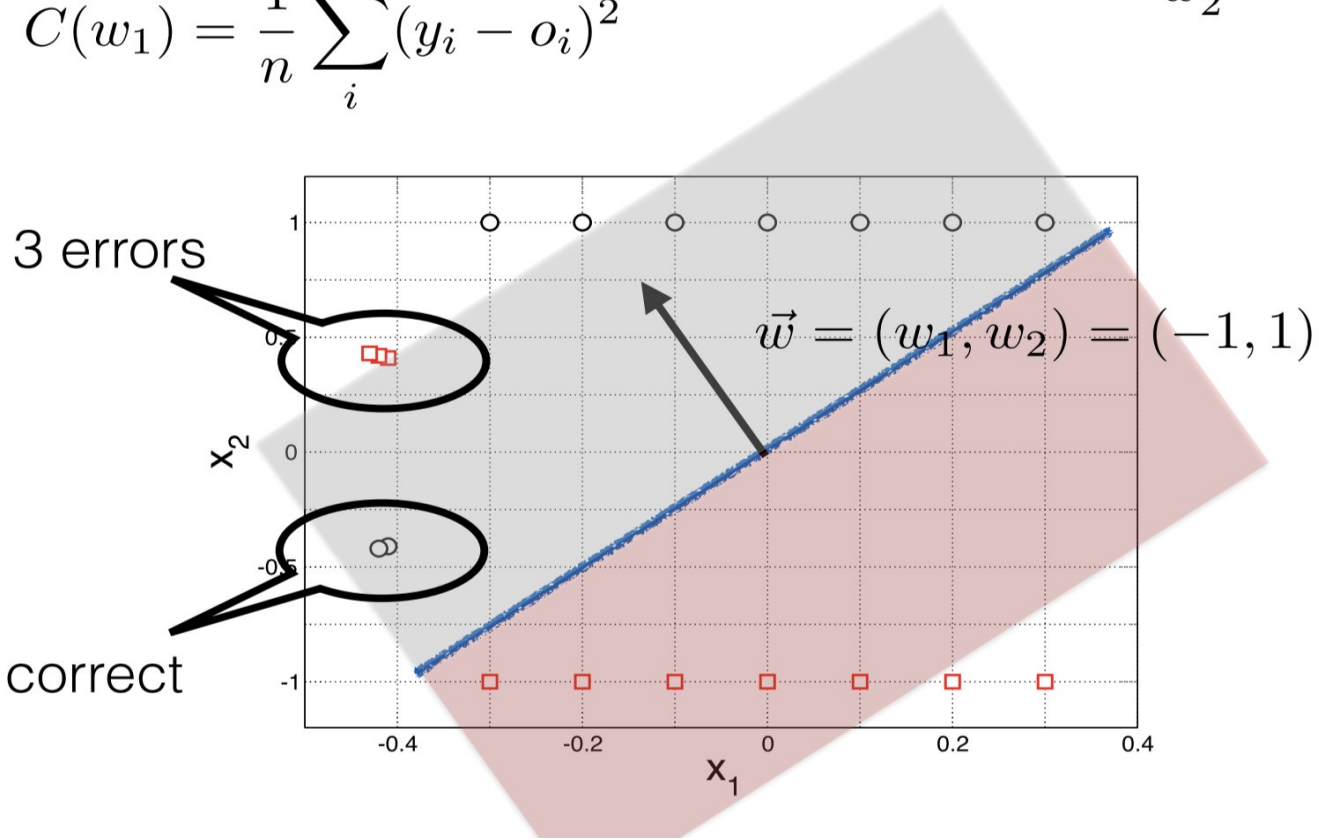
局部最优

$$o_i = \sigma(w_1 x_1 + w_2 x_2) \text{ with } w_2 = 1$$

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

$$C(w_1) = \frac{1}{n} \sum_i (y_i - o_i)^2$$



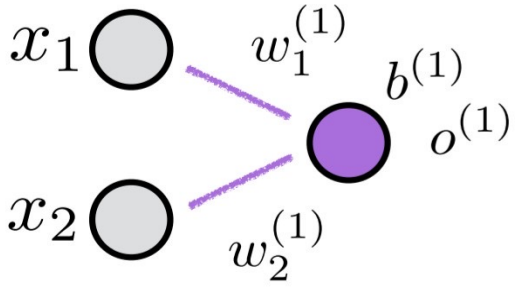


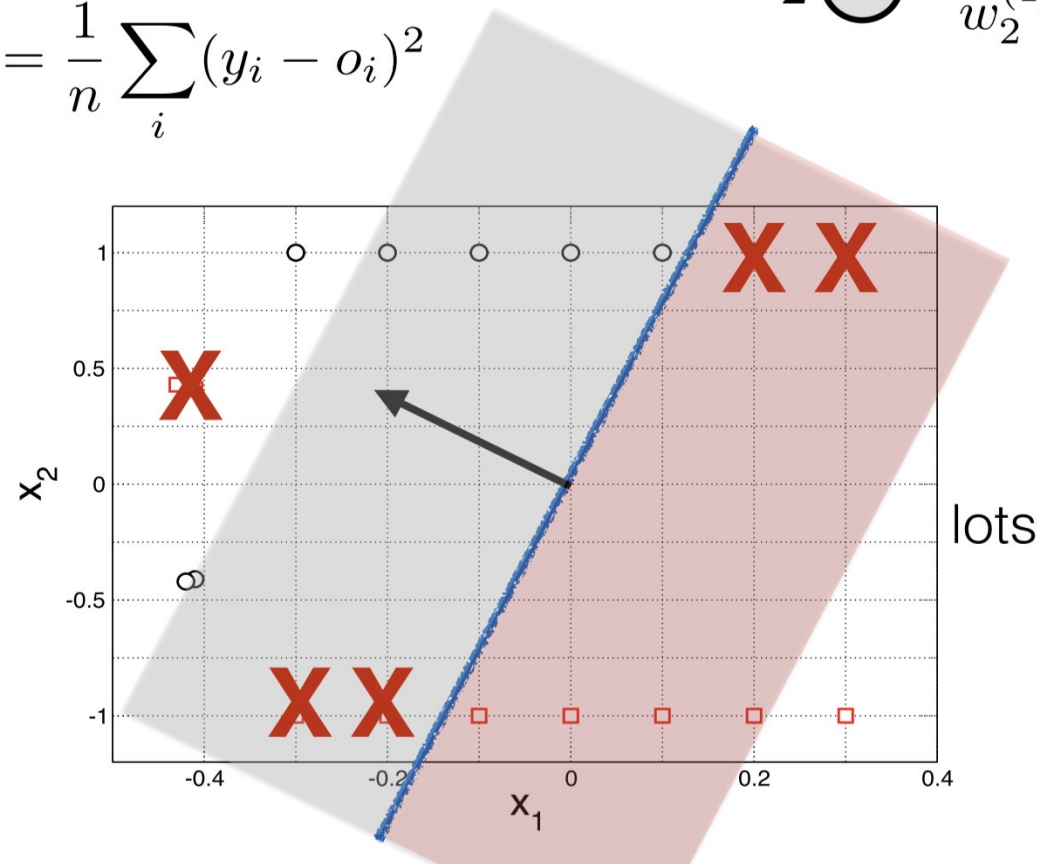
局部最优

$$o_i = \sigma(w_1 x_1 + w_2 x_2) \text{ with } w_2 = 1$$

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

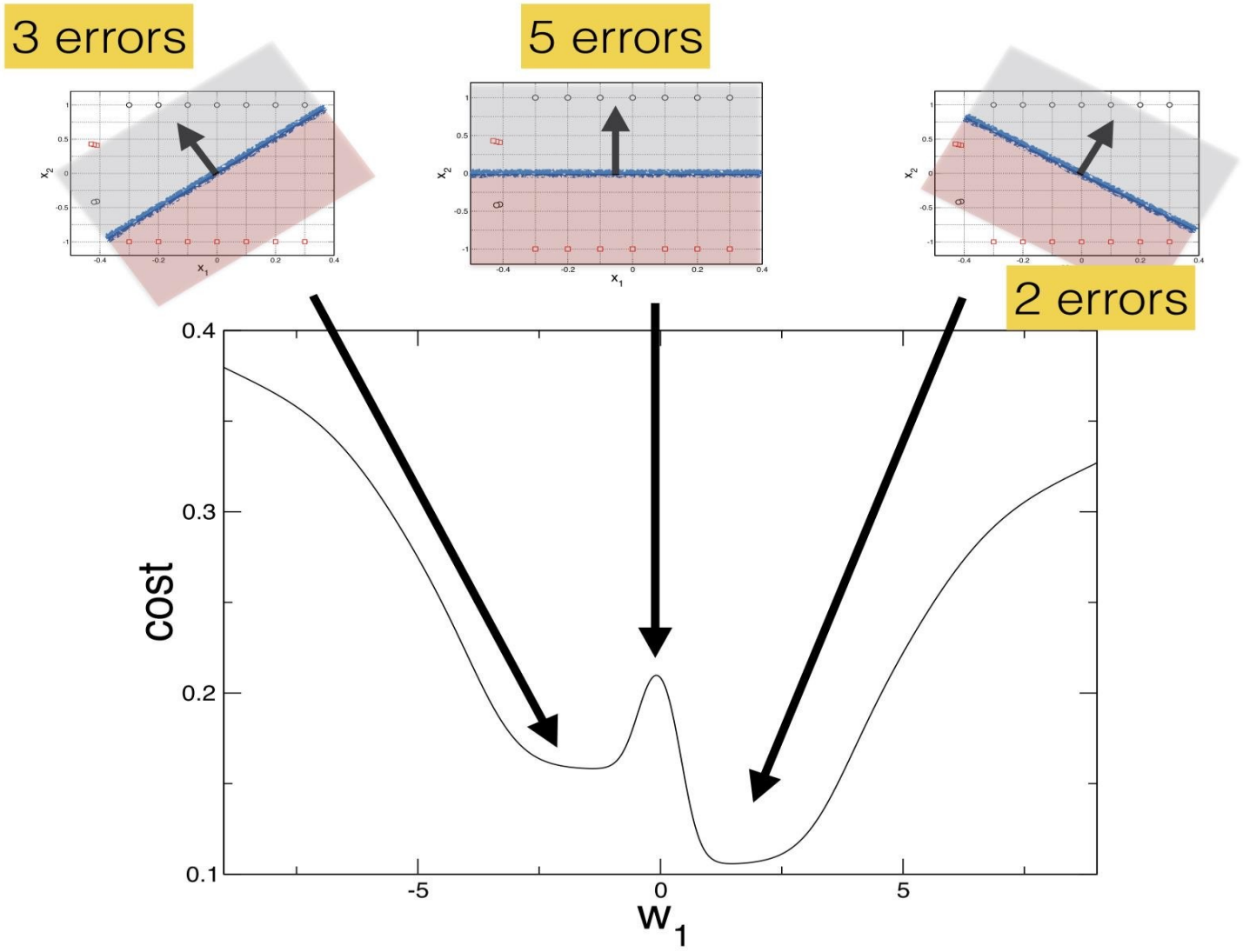
$$C(w_1) = \frac{1}{n} \sum_i (y_i - o_i)^2$$



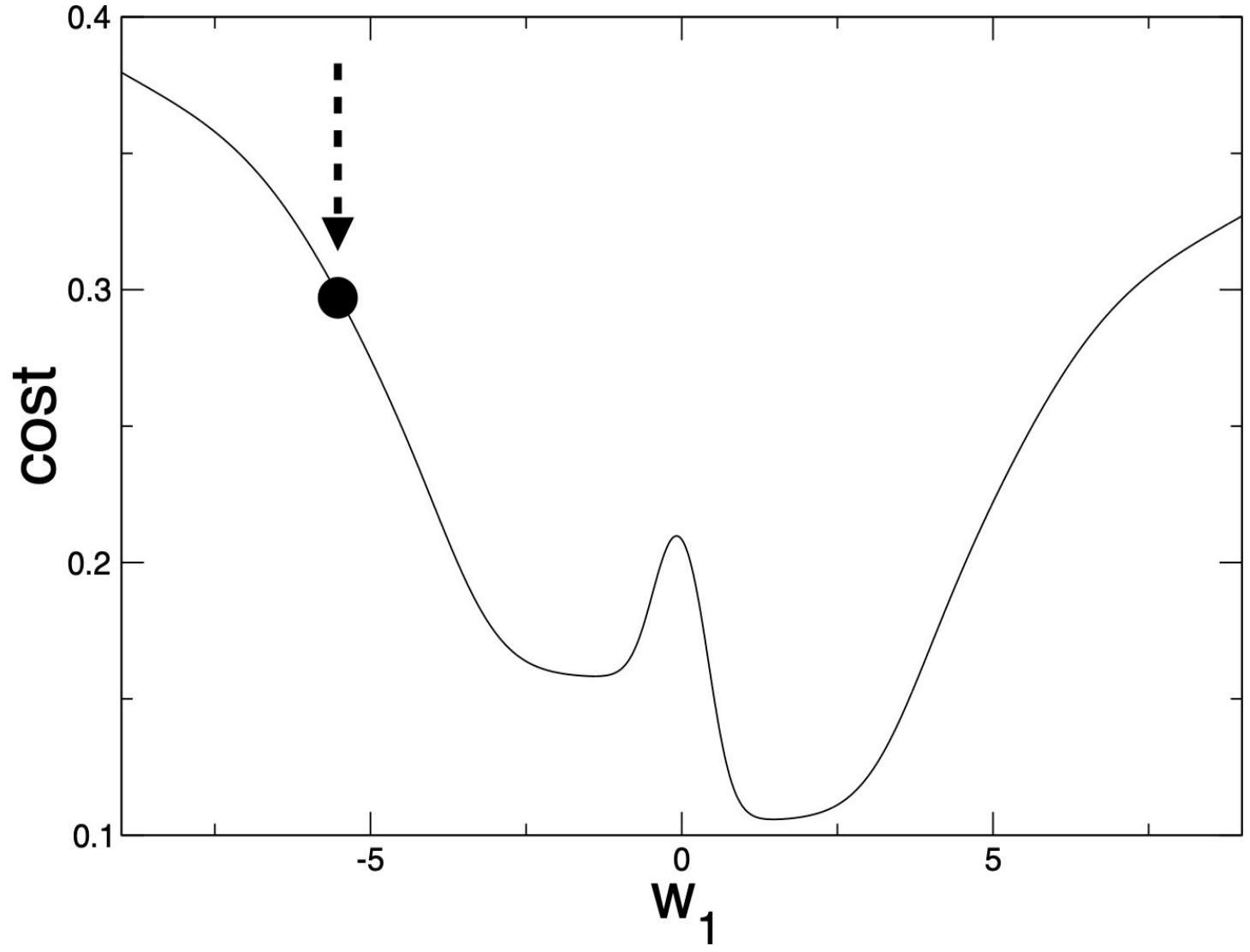


lots of errors!

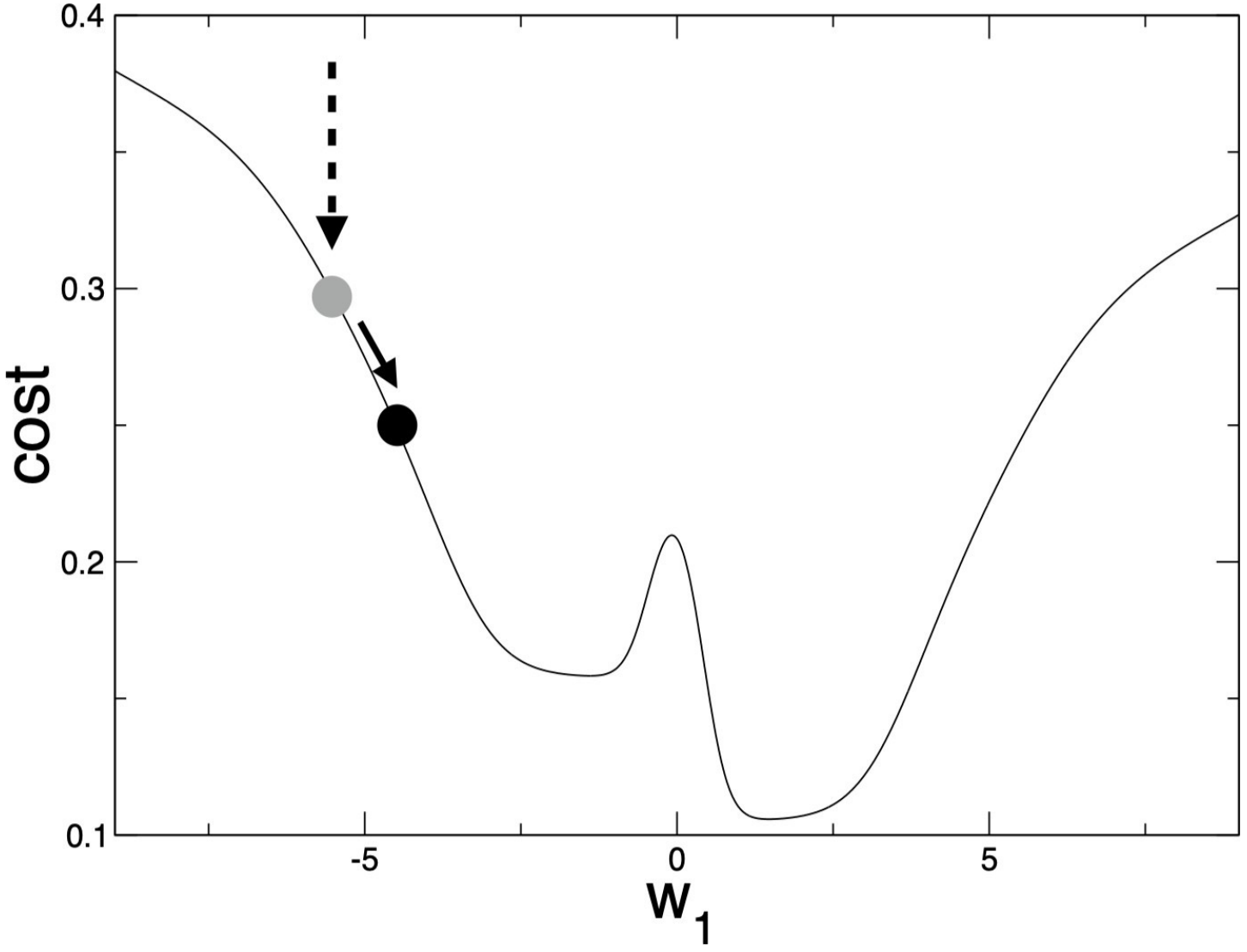
局部最优



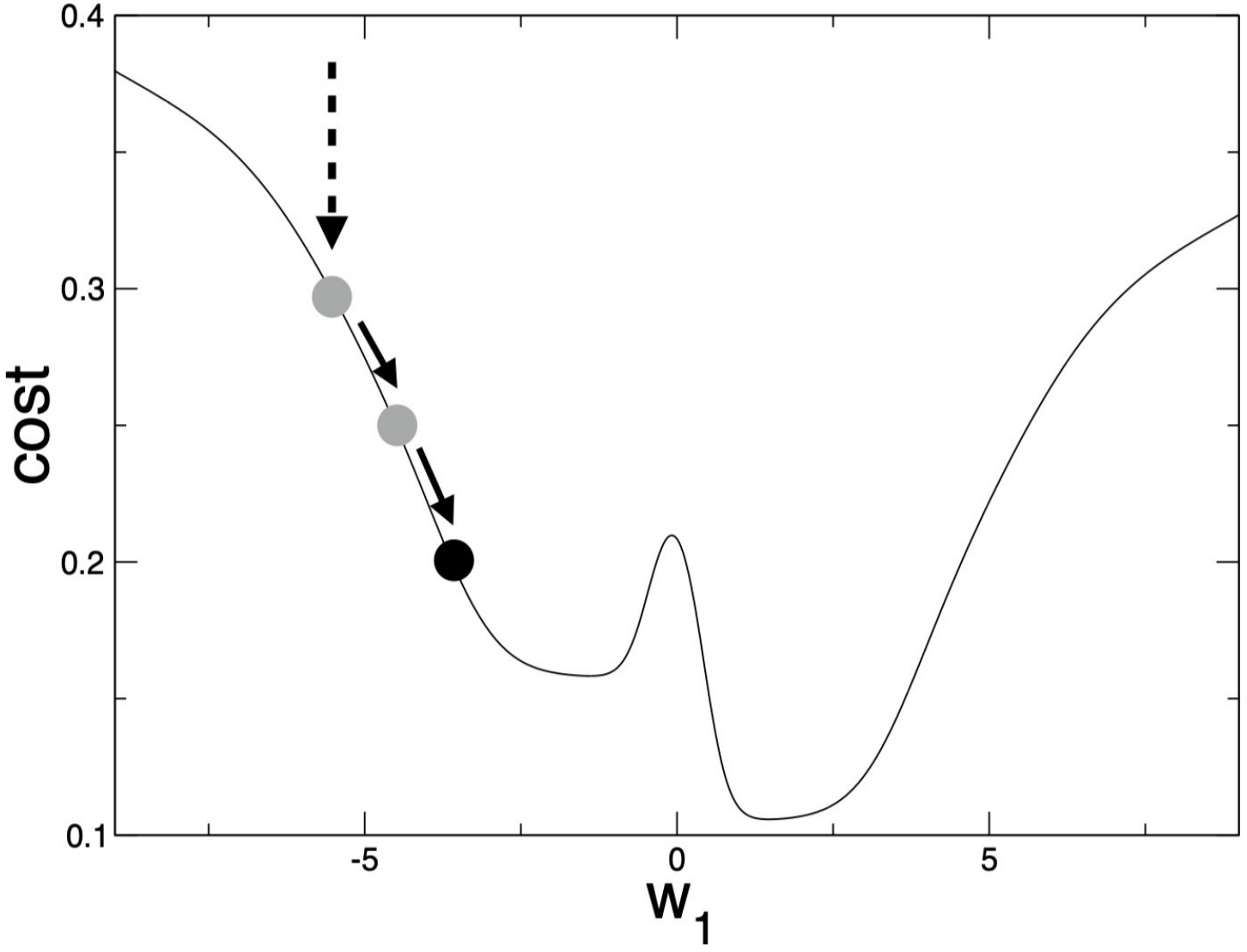
局部最优



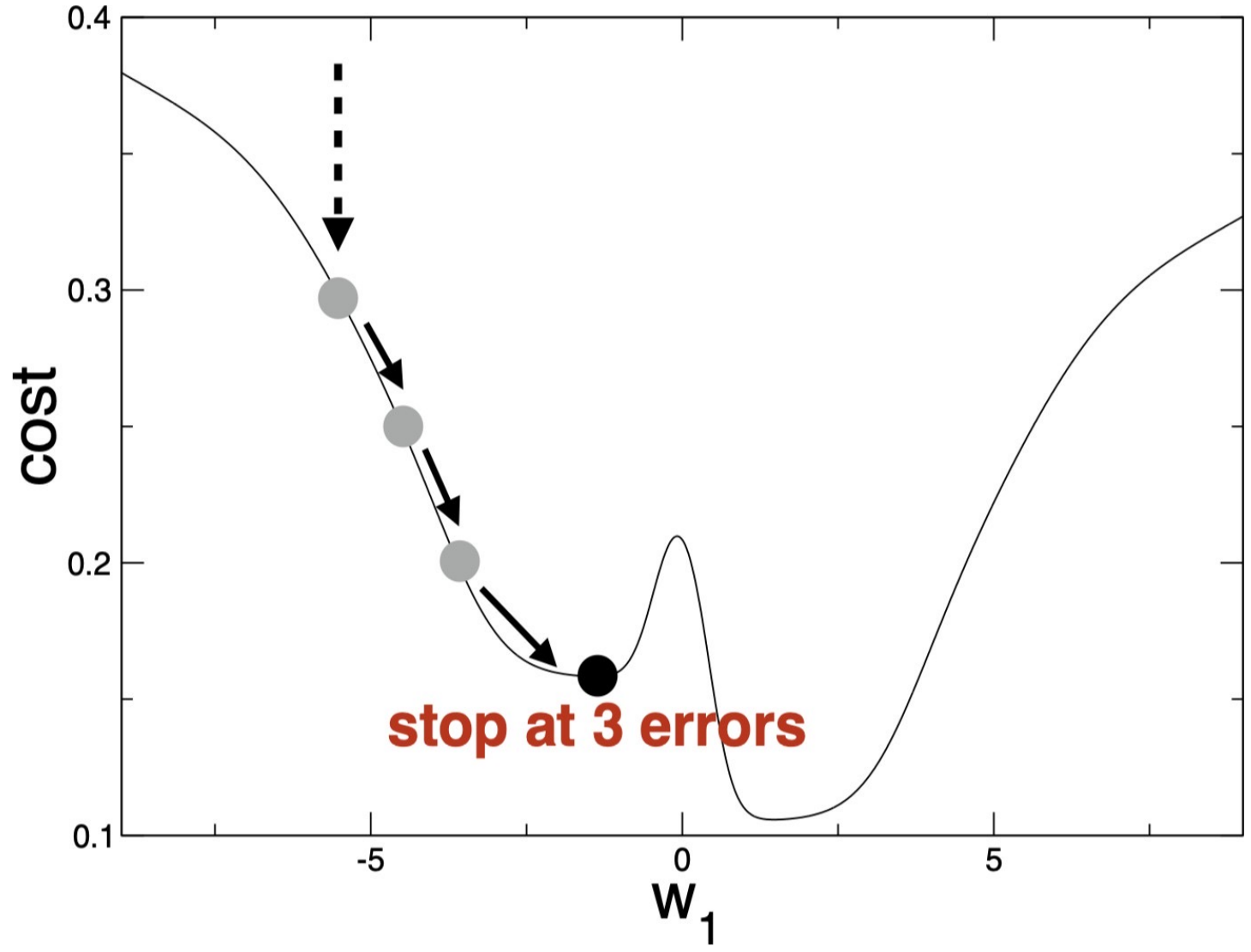
局部最优



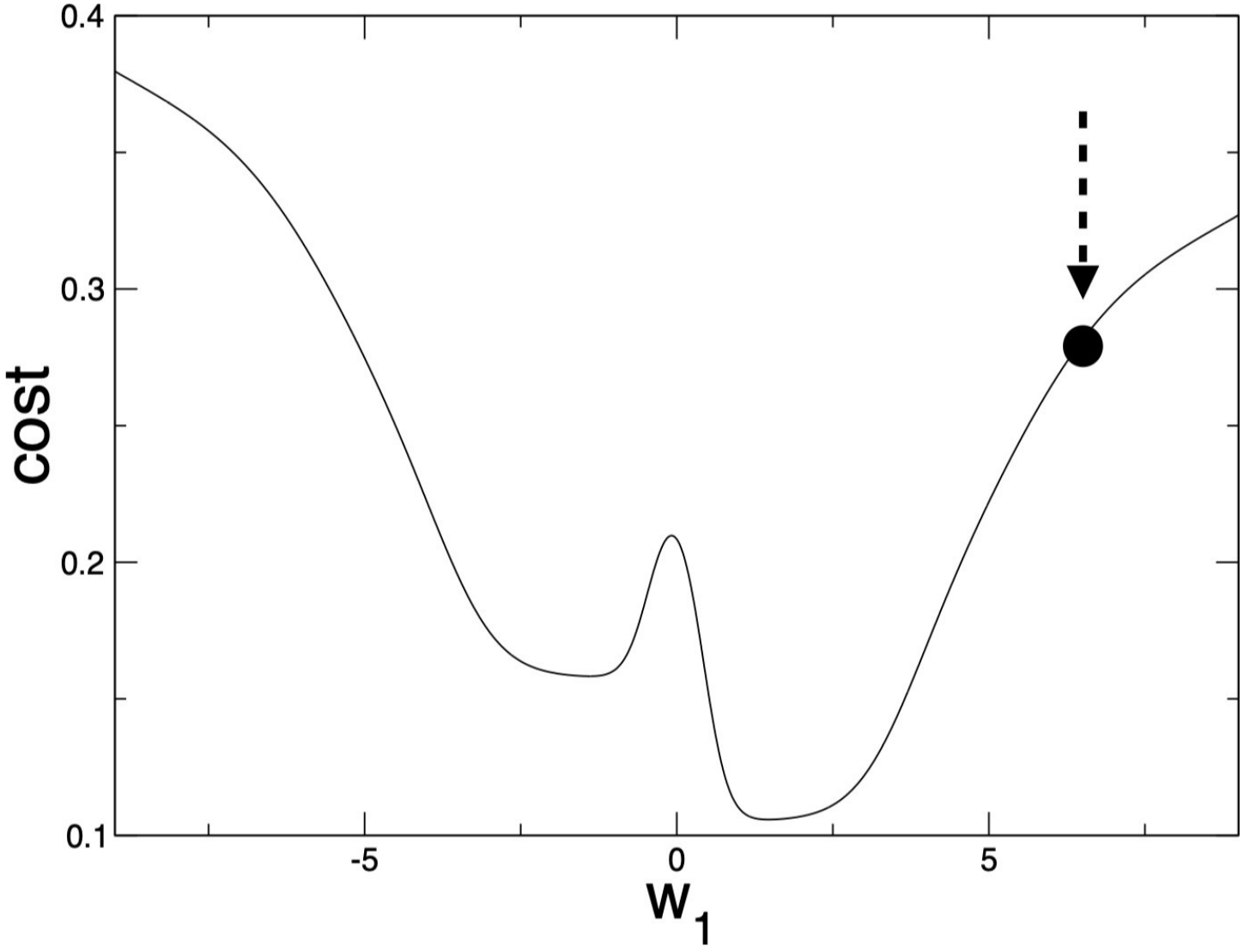
局部最优



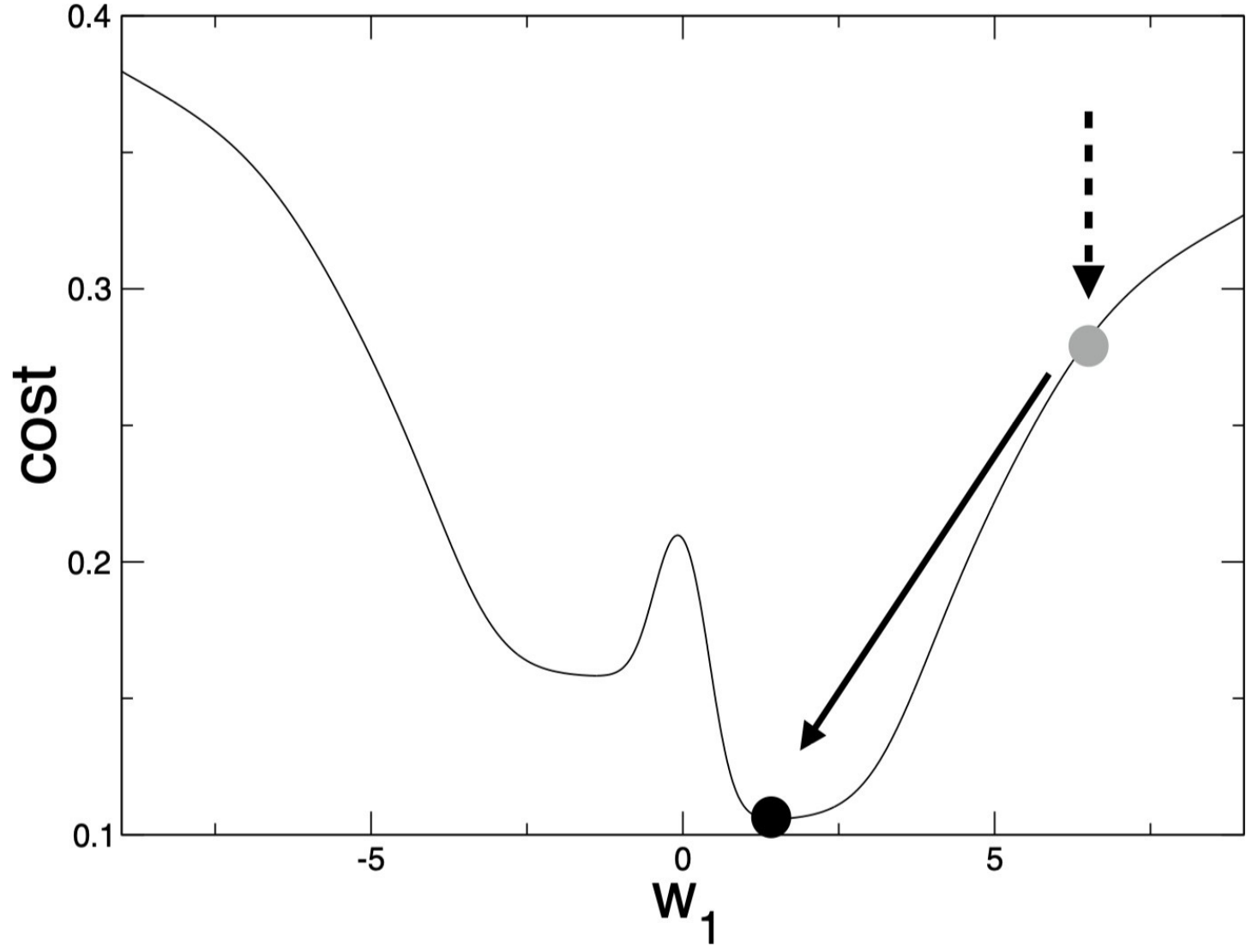
局部最优



局部最优

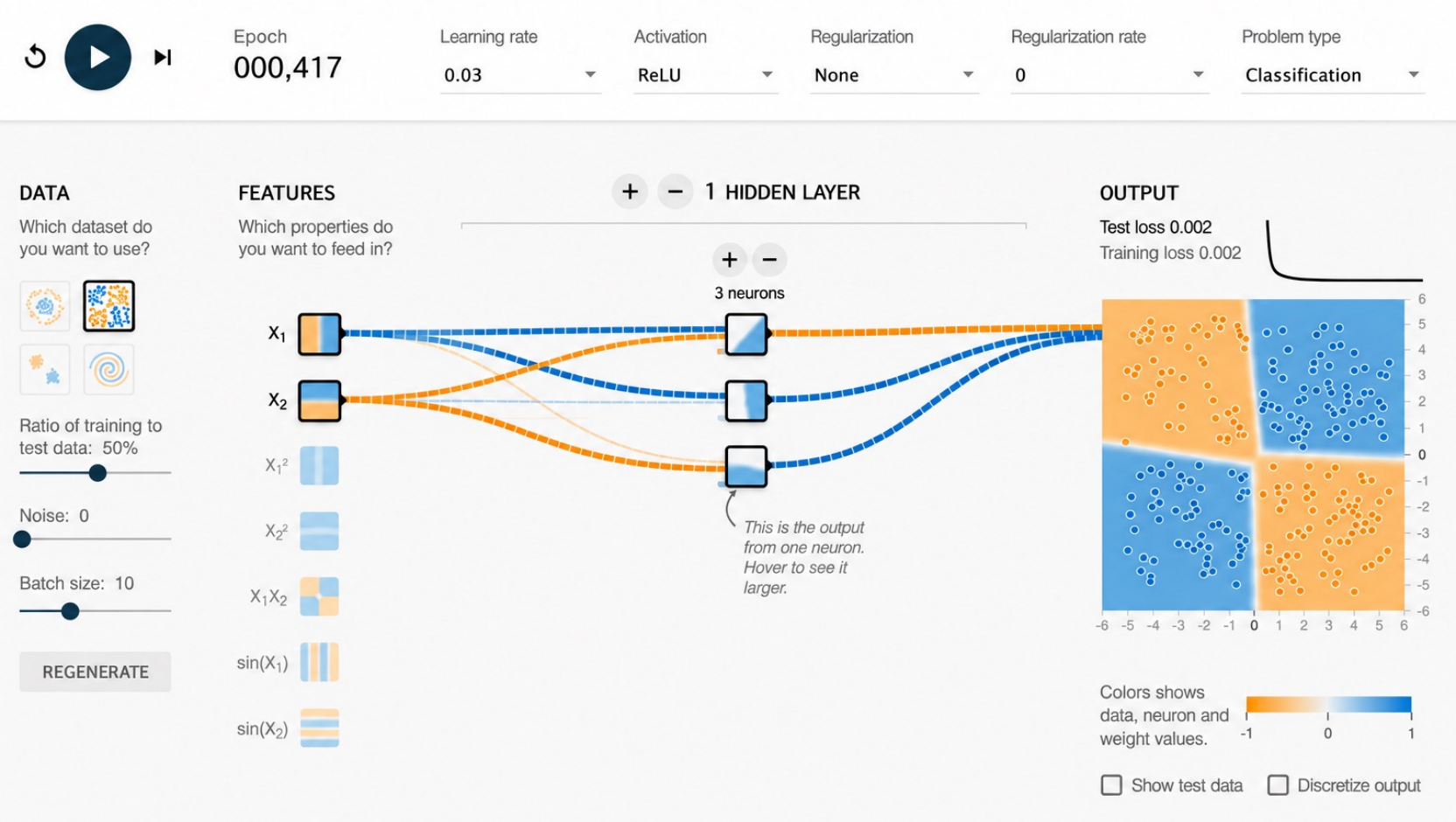


局部最优



局部最优

- 一种好的解


Epoch
000,417
Learning rate
0.03
Activation
ReLU
Regularization
None
Regularization rate
0
Problem type
Classification
DATA
Which dataset do you want to use?
Ratio of training to test data: 50%
Noise: 0
Batch size: 10
REGENERATE
FEATURES
Which properties do you want to feed in?
X1
X2
X1²
X2²
X1X2
sin(X1)
sin(X2)
1 HIDDEN LAYER
3 neurons
This is the output from one neuron. Hover to see it larger.
OUTPUT
Test loss 0.002
Training loss 0.002
Colors shows data, neuron and weight values.
Show test data
Discretize output

局部最优

- 局部最小解示例



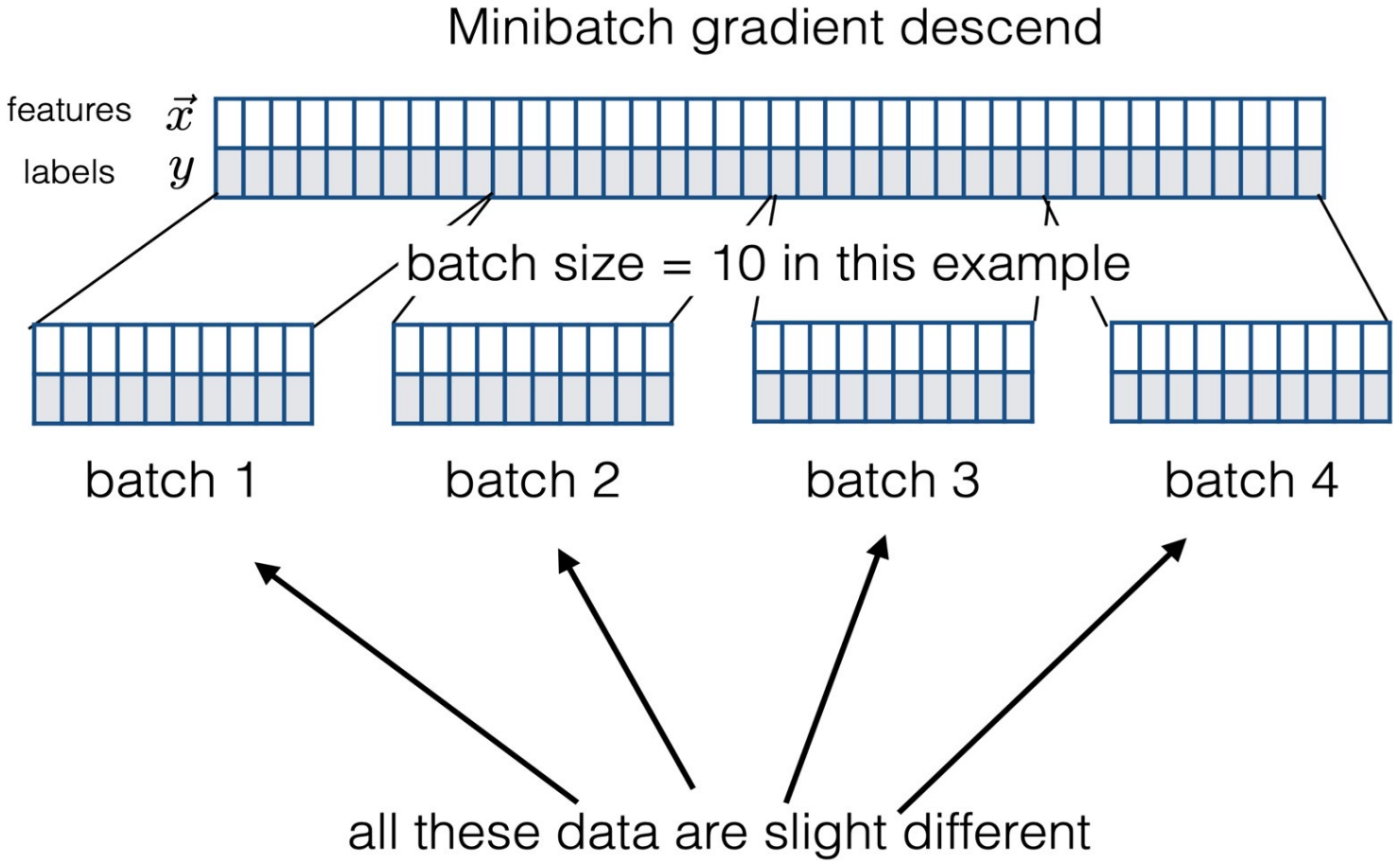
常见优化算法

1. Stochastic gradient descent (SGD)
- 随机梯度下降

2. Adam method, momentum
- **Momentum**：在更新时引入**“惯性”**，即新的方向不仅取决于当前梯度，还取决于之前的更新方向。
- **Adam**：**自适应**学习率方法

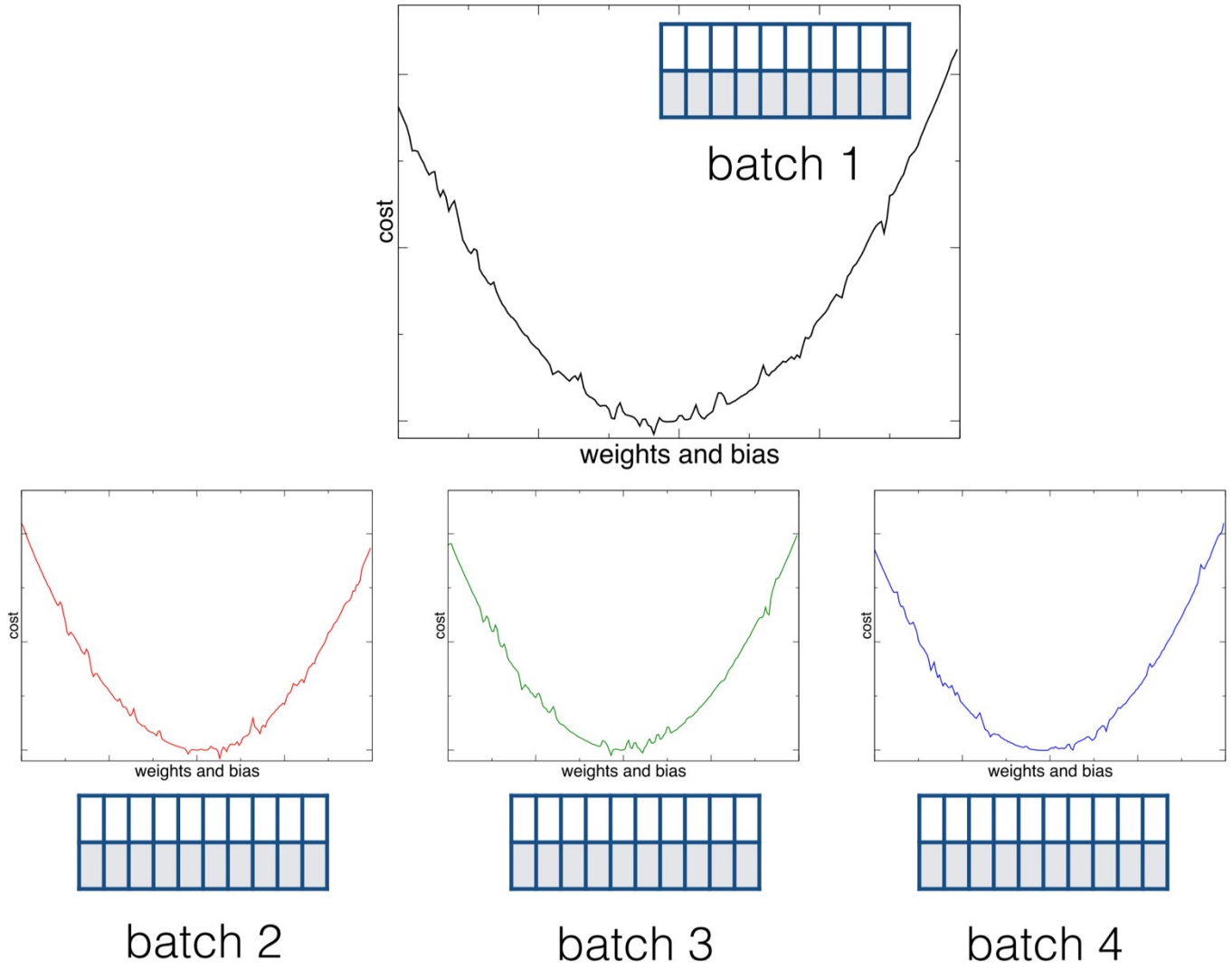
Minibatch梯度下降

- Minibatch gradient descend



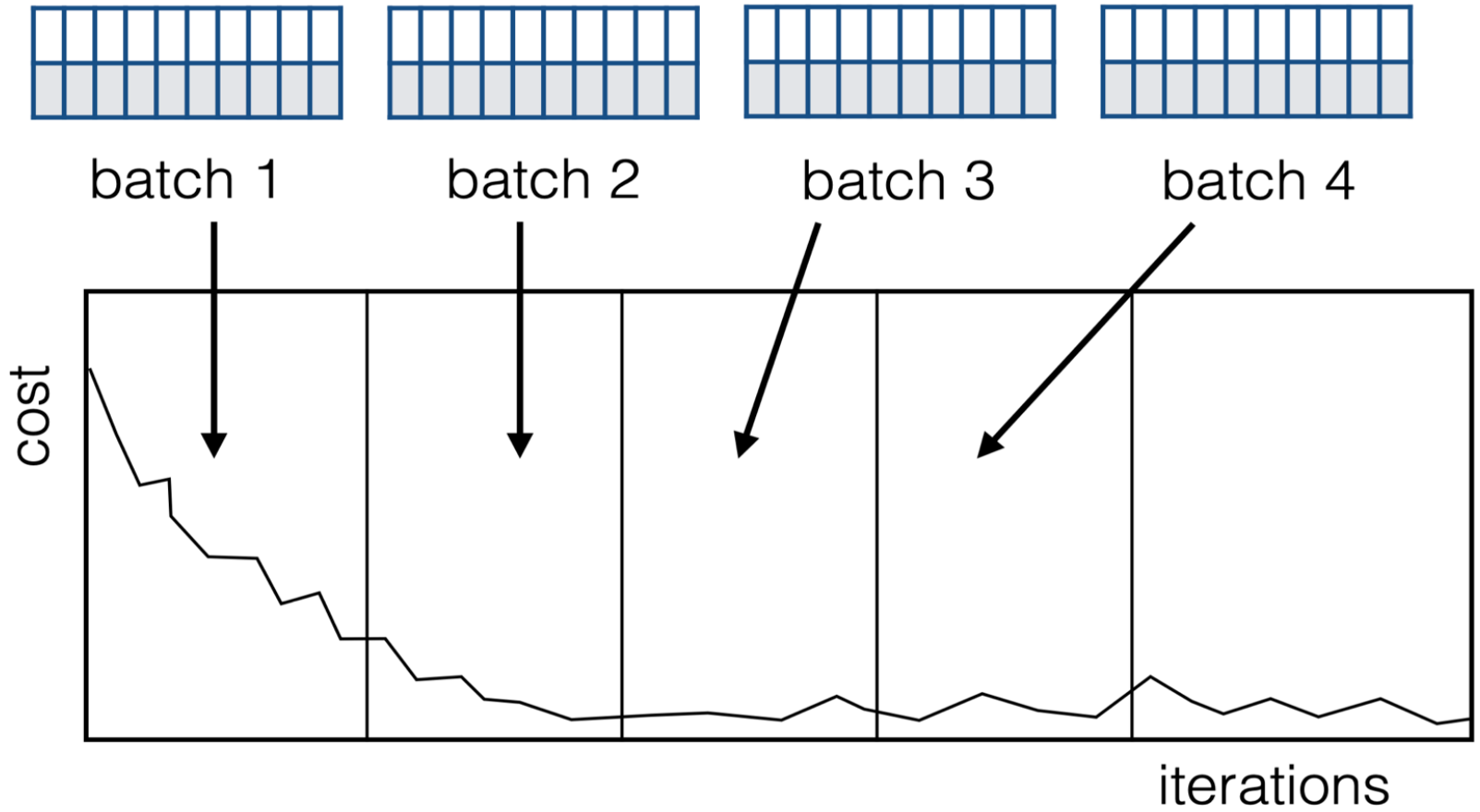
Cost surfaces are different for different data sets

Minibatch梯度下降

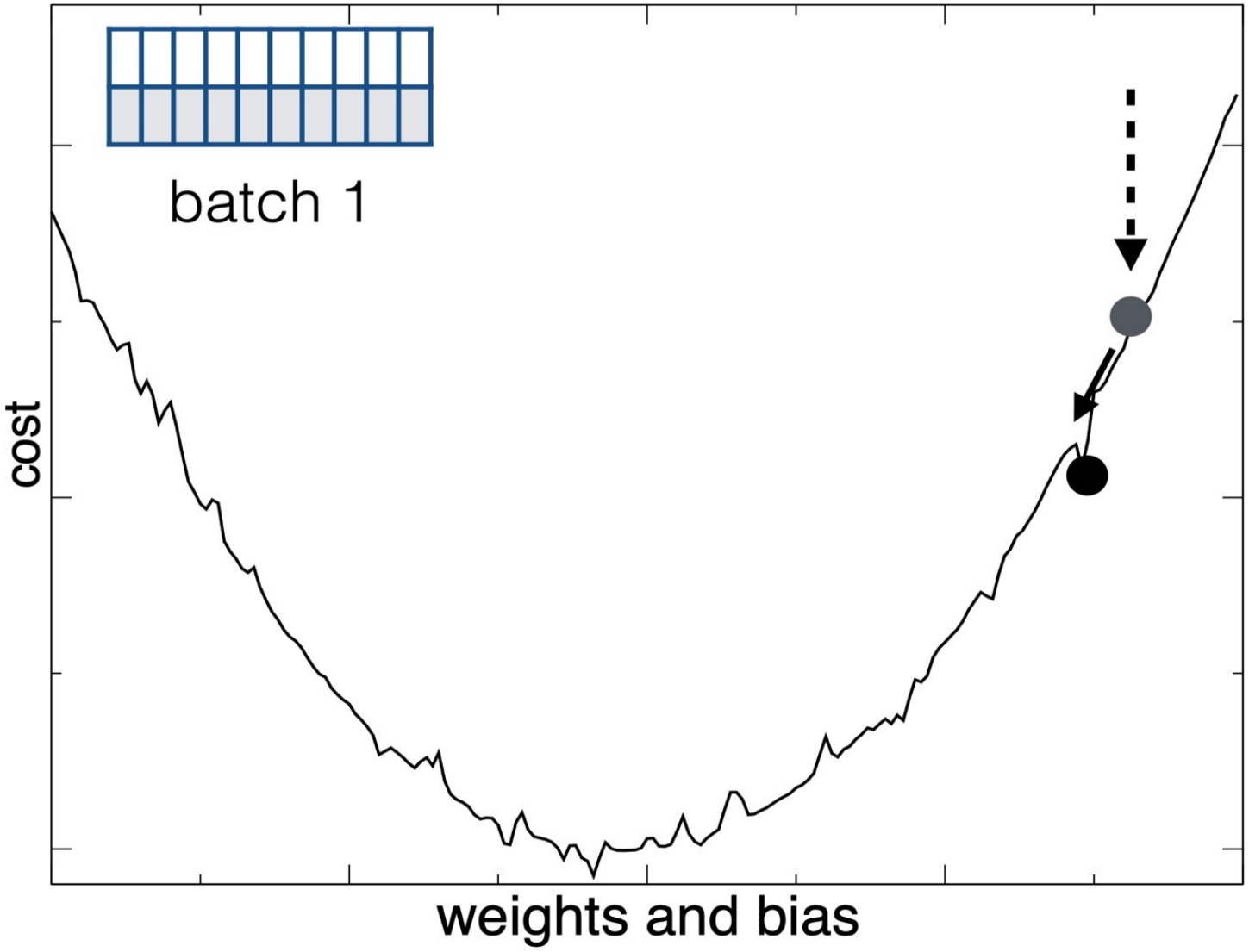


Minibatch梯度下降

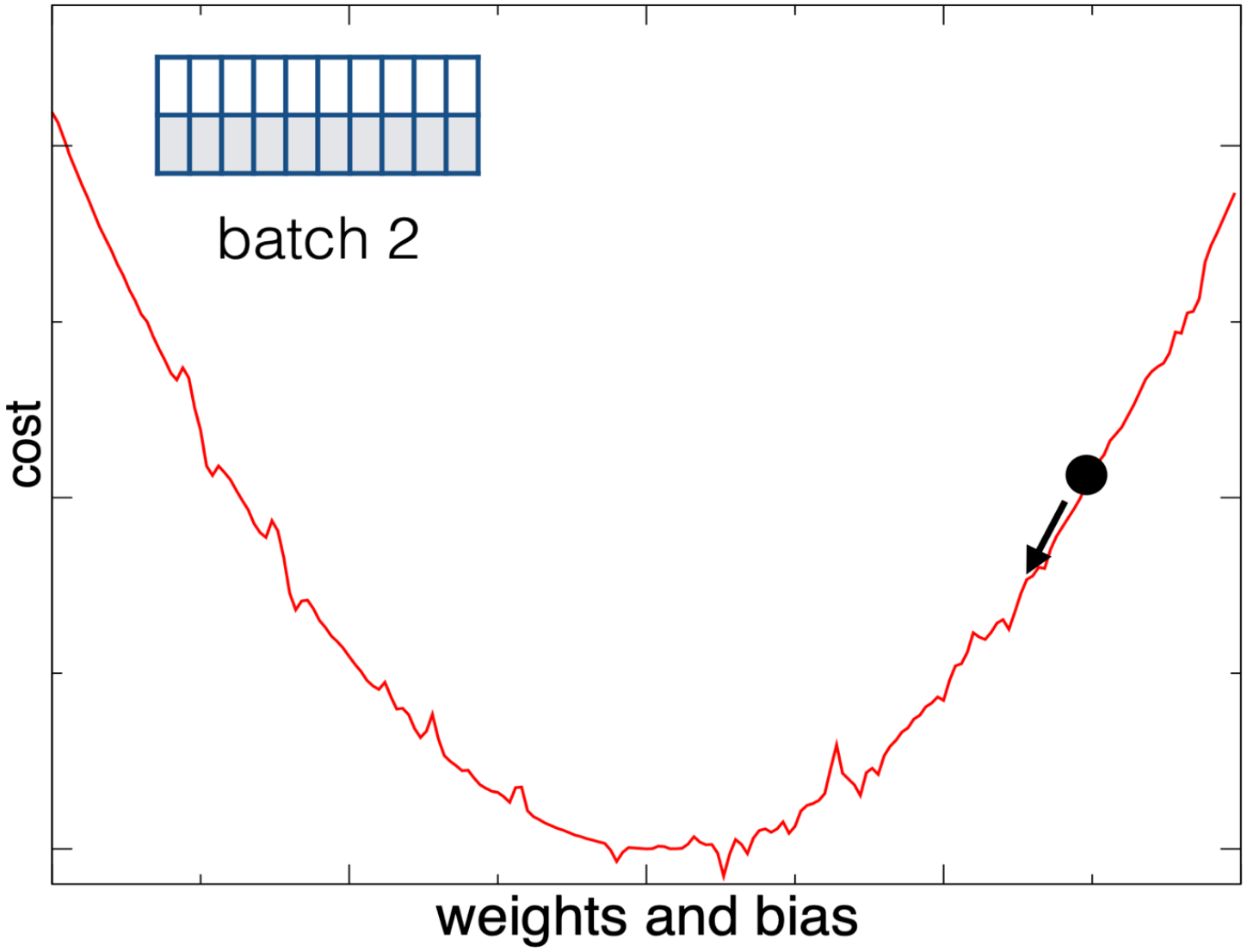
- 注意数据洗牌（shuffle）



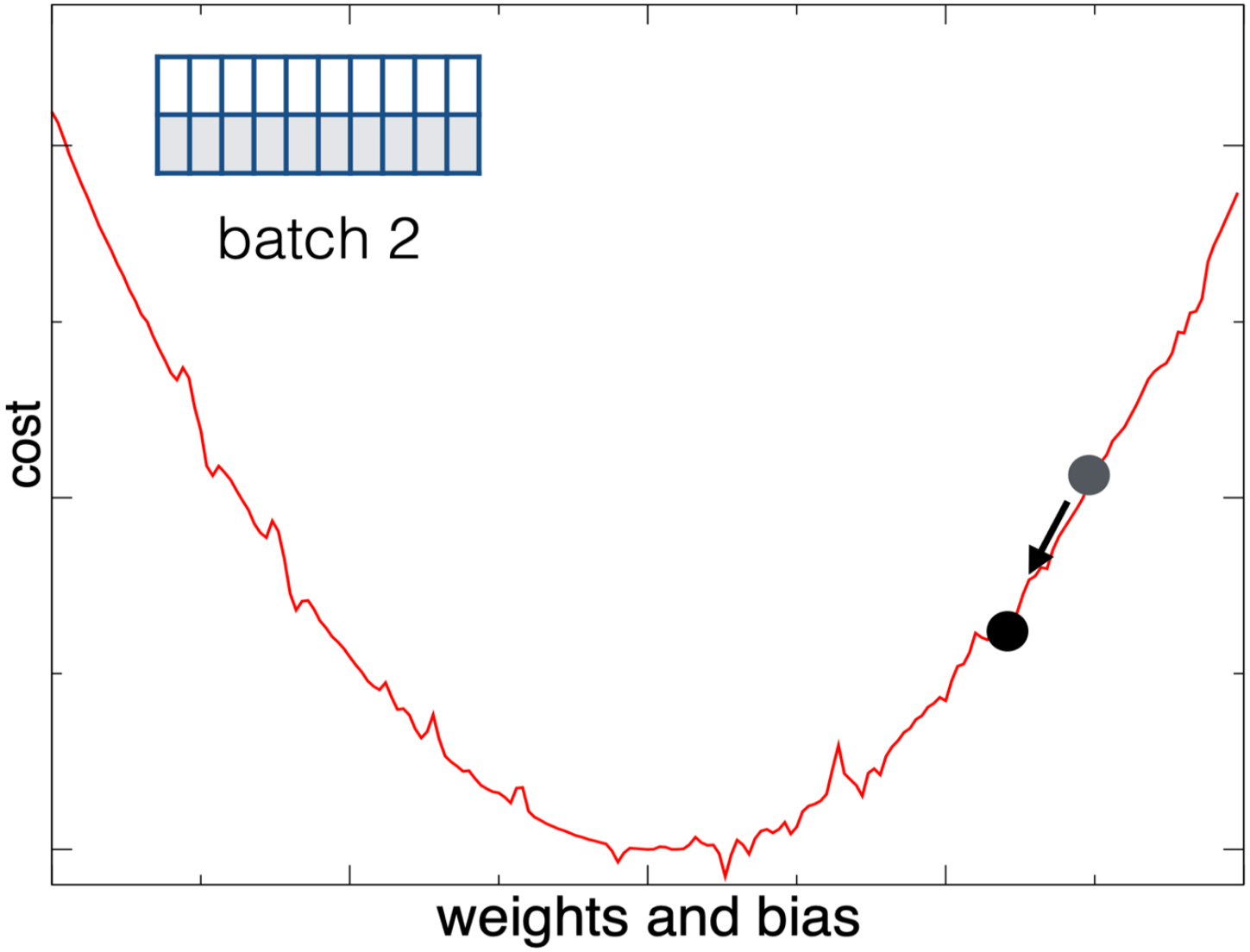
Minibatch梯度下降



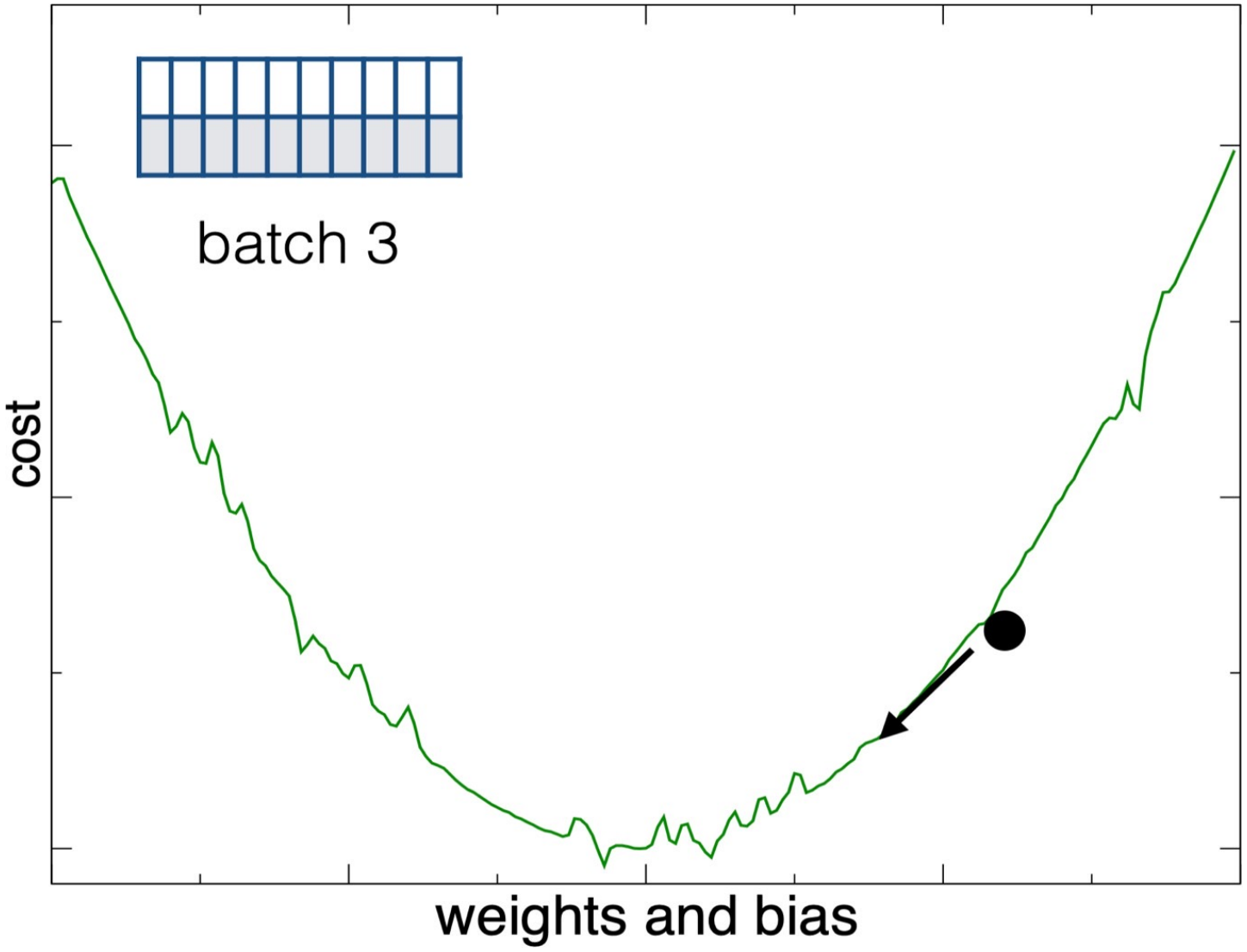
Minibatch梯度下降



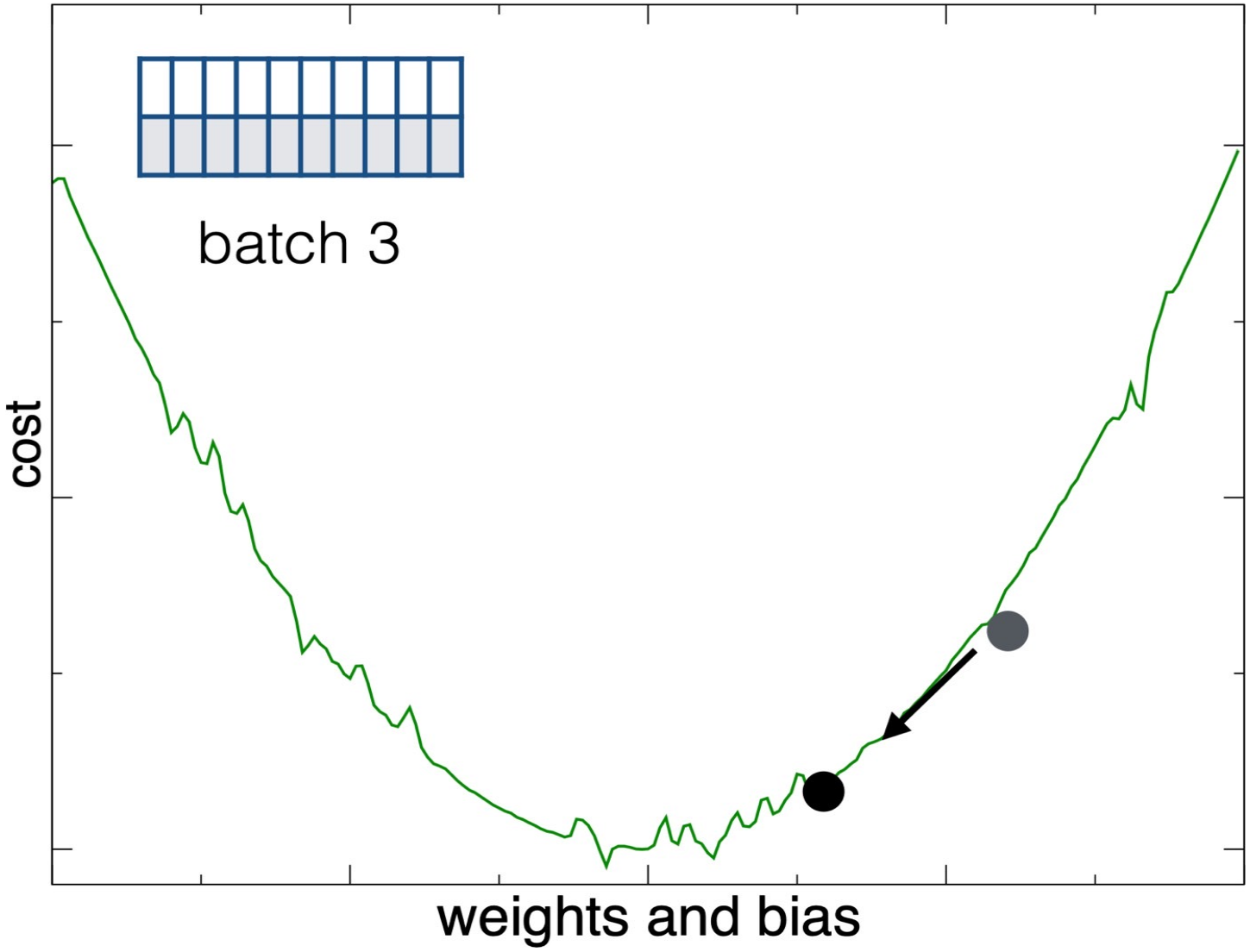
Minibatch梯度下降



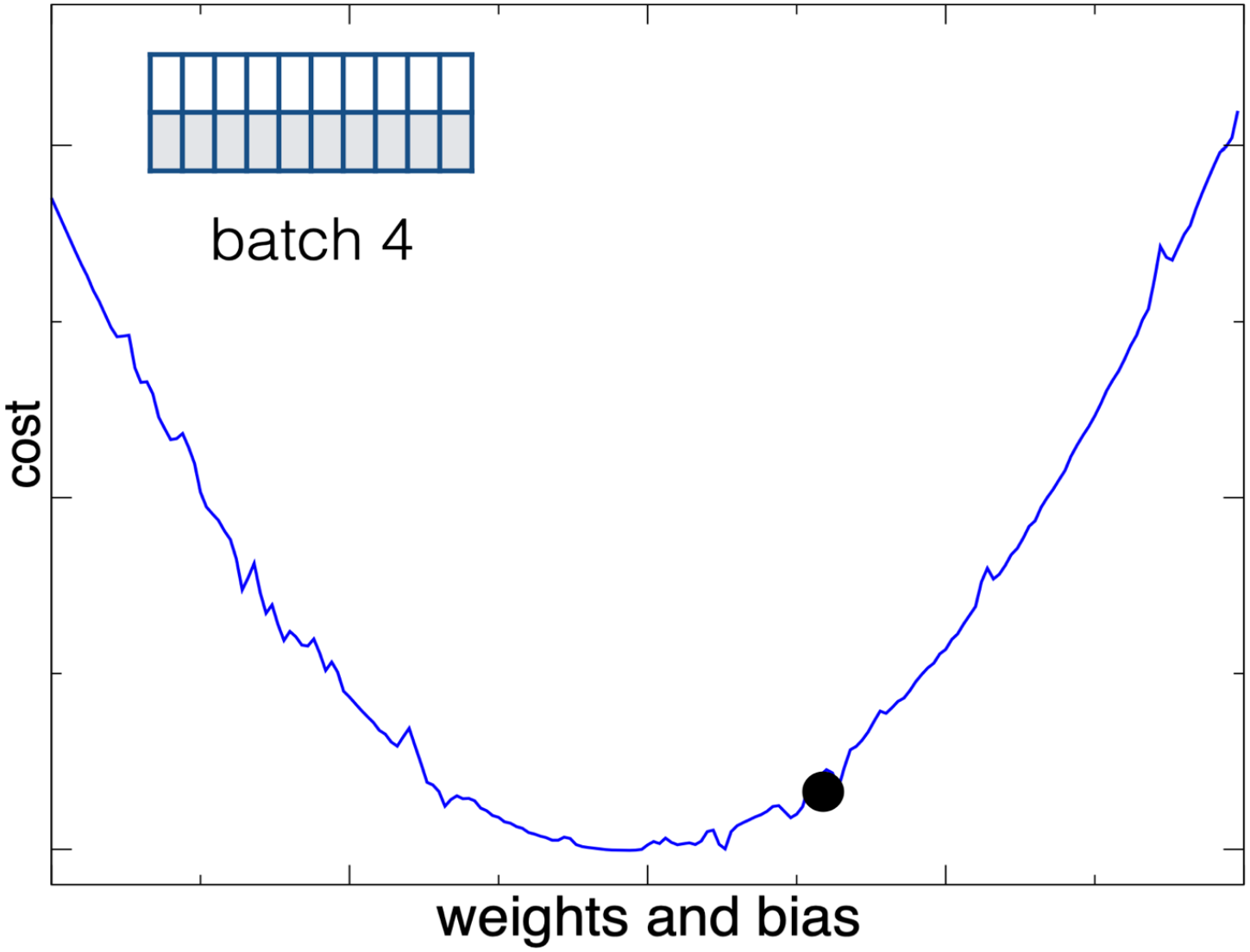
Minibatch梯度下降



Minibatch梯度下降

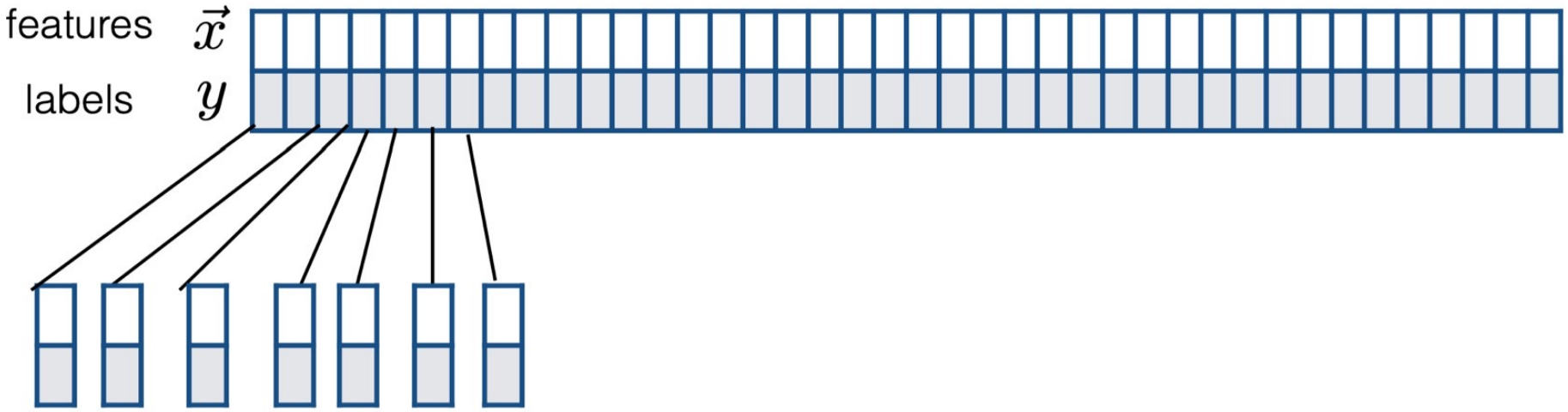


Minibatch梯度下降



随机梯度下降

- Stochastic gradient descend (SGD)
  - 批次大小为1的Minibatch梯度下降



Adam算法

Published as a conference paper at ICLR 2015

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

Diederik P. Kingma*
University of Amsterdam, OpenAI
dpkingma@openai.com

Jimmy Lei Ba*
University of Toronto
jimmy@psi.utoronto.ca

10年，22万citations!

Adam算法

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t.

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\theta)$: Stochastic objective function with parameters θ
Require: θ_0: Initial parameter vector
$m_0 \leftarrow 0$ (Initialize 1st moment vector)
$v_0 \leftarrow 0$ (Initialize 2nd moment vector)
$t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
$t \leftarrow t + 1$
$g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$ (Update parameters)
end while
return θ_t (Resulting parameters)

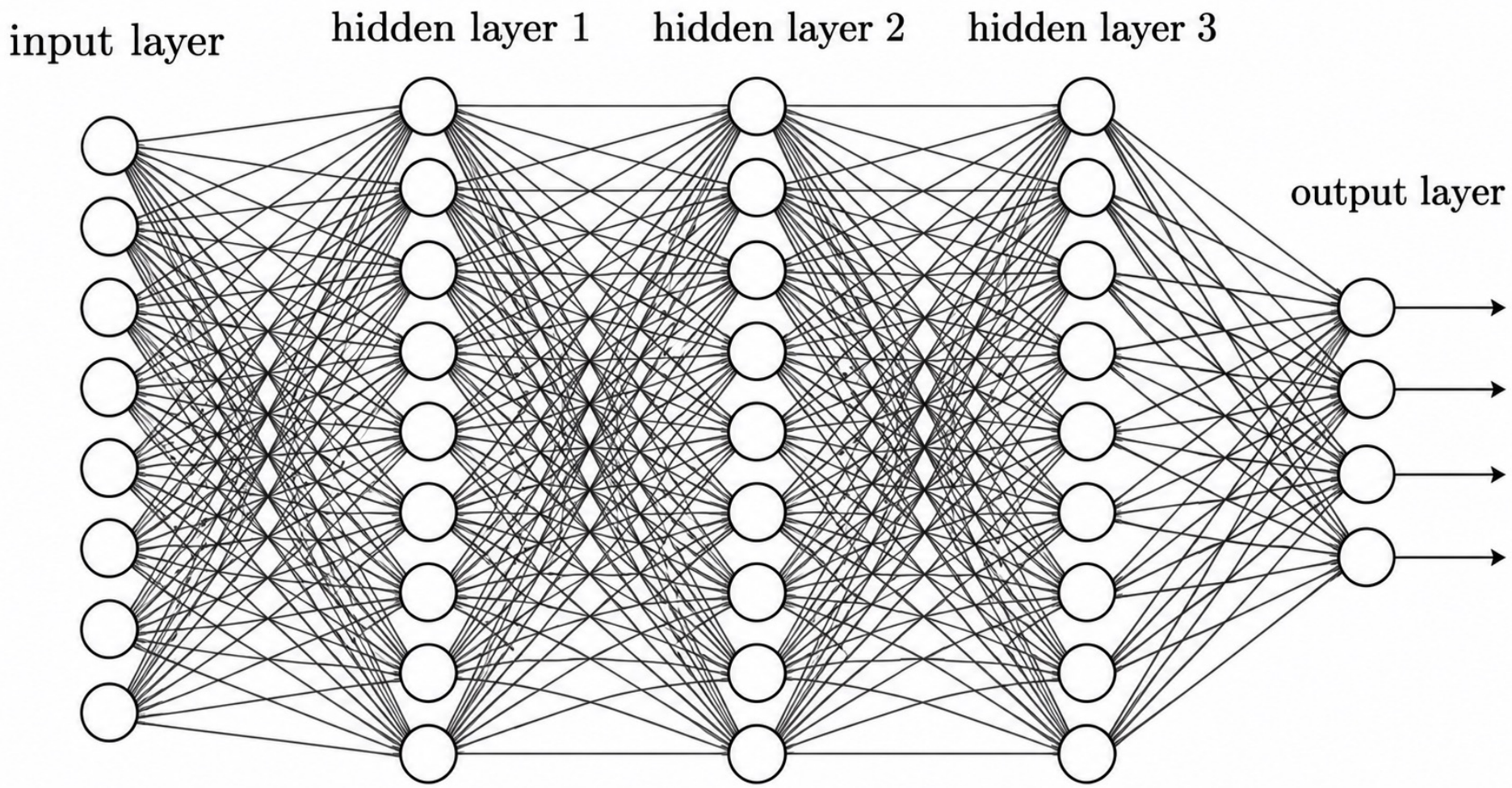
average the gradient direction over the past

move in the direction with “adaptive” step size

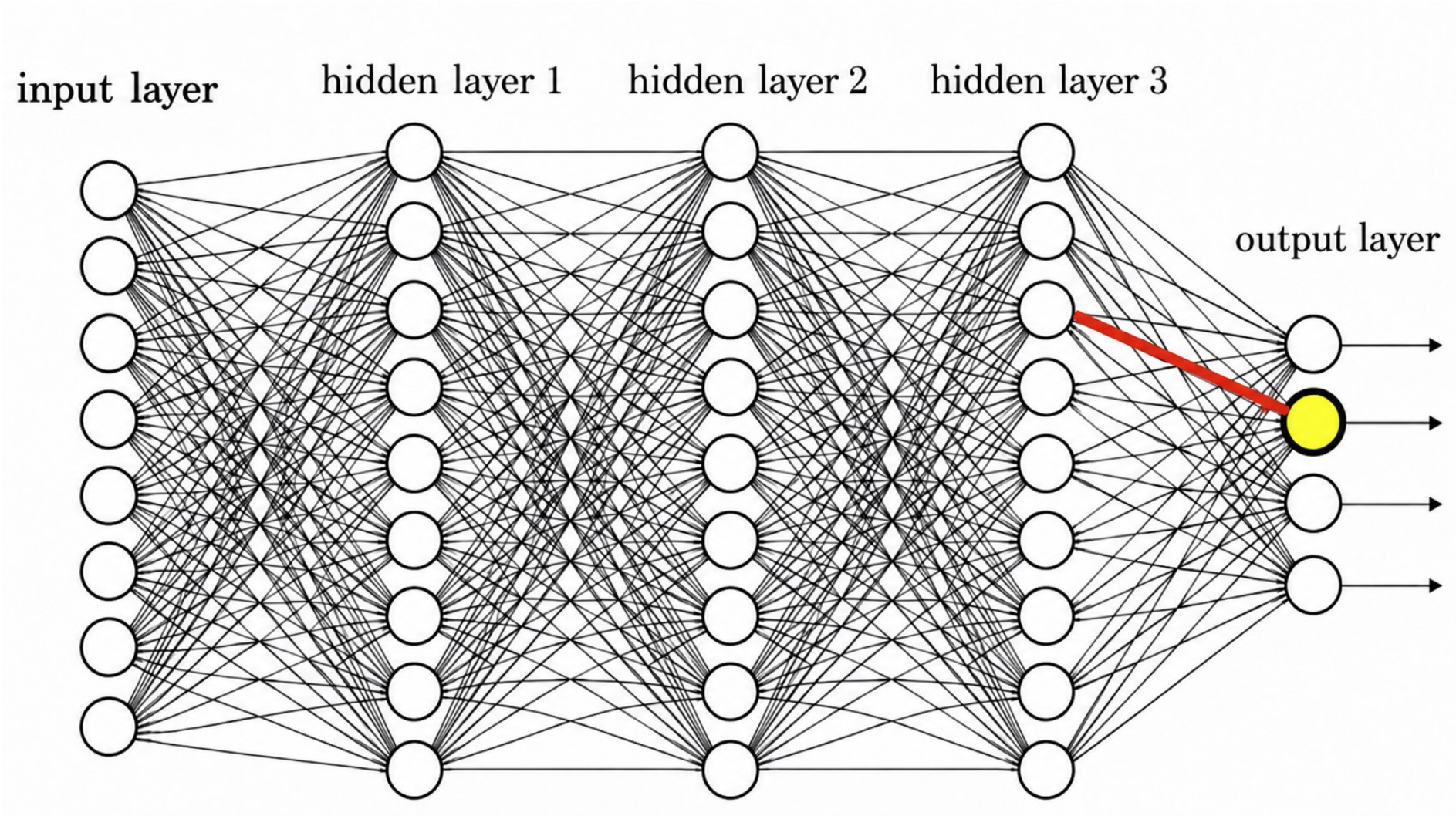
提纲

- 计算图与反向传播
- 向量与张量的反向传播
- **神经网络训练中的挑战**
  - 局部最小值与优化算法
  - 梯度消失
- 数据流形与模型泛化

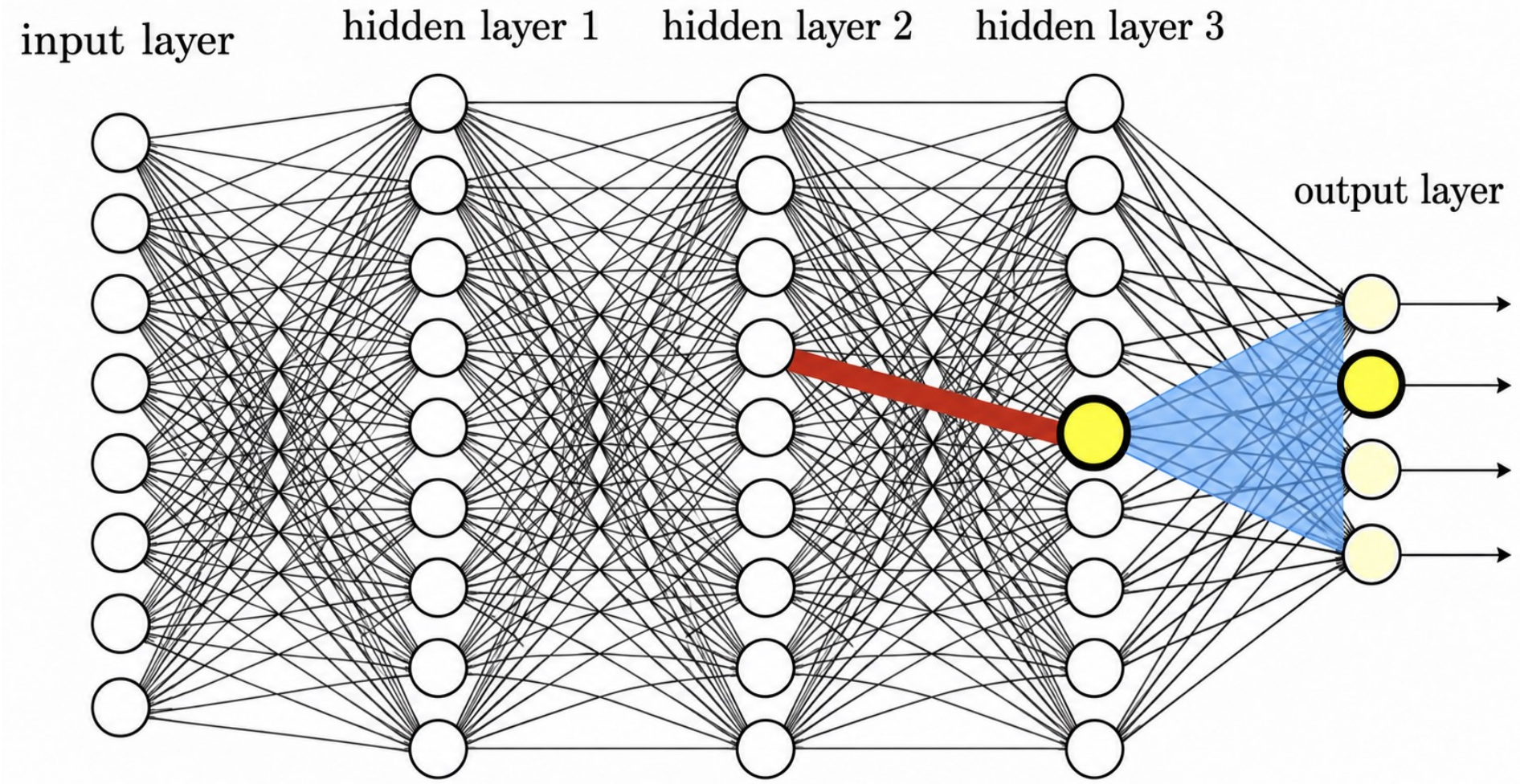
梯度消失的问题


input layer
hidden layer 1
hidden layer 2
hidden layer 3
output layer

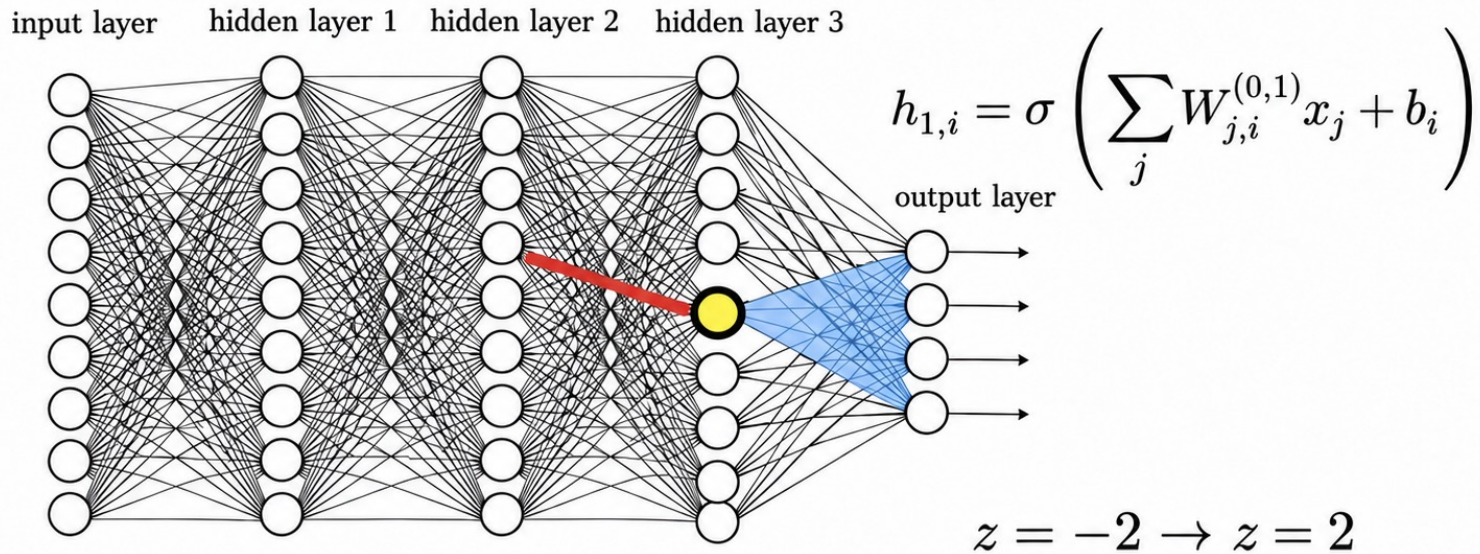
梯度消失的问题


input layer
hidden layer 1
hidden layer 2
hidden layer 3
output layer

梯度消失的问题

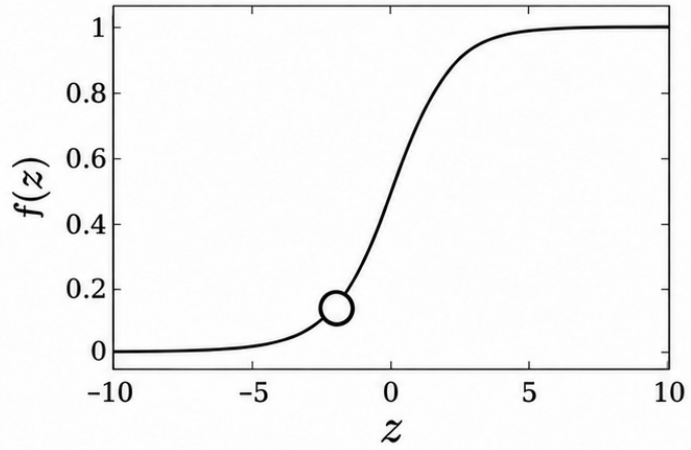

input layer
hidden layer 1
hidden layer 2
hidden layer 3
output layer

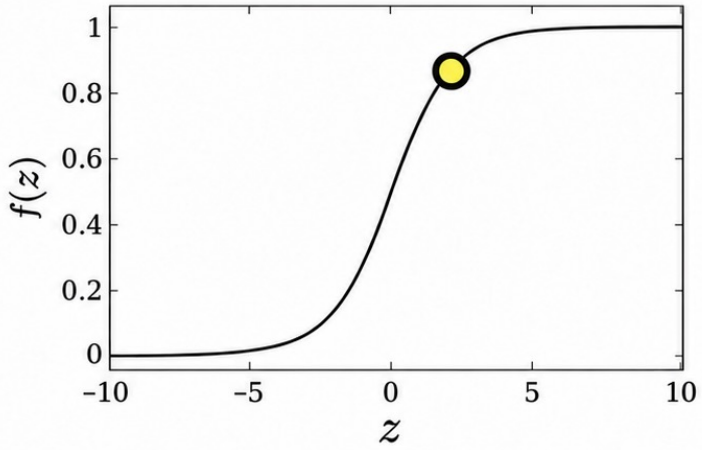
梯度消失的问题



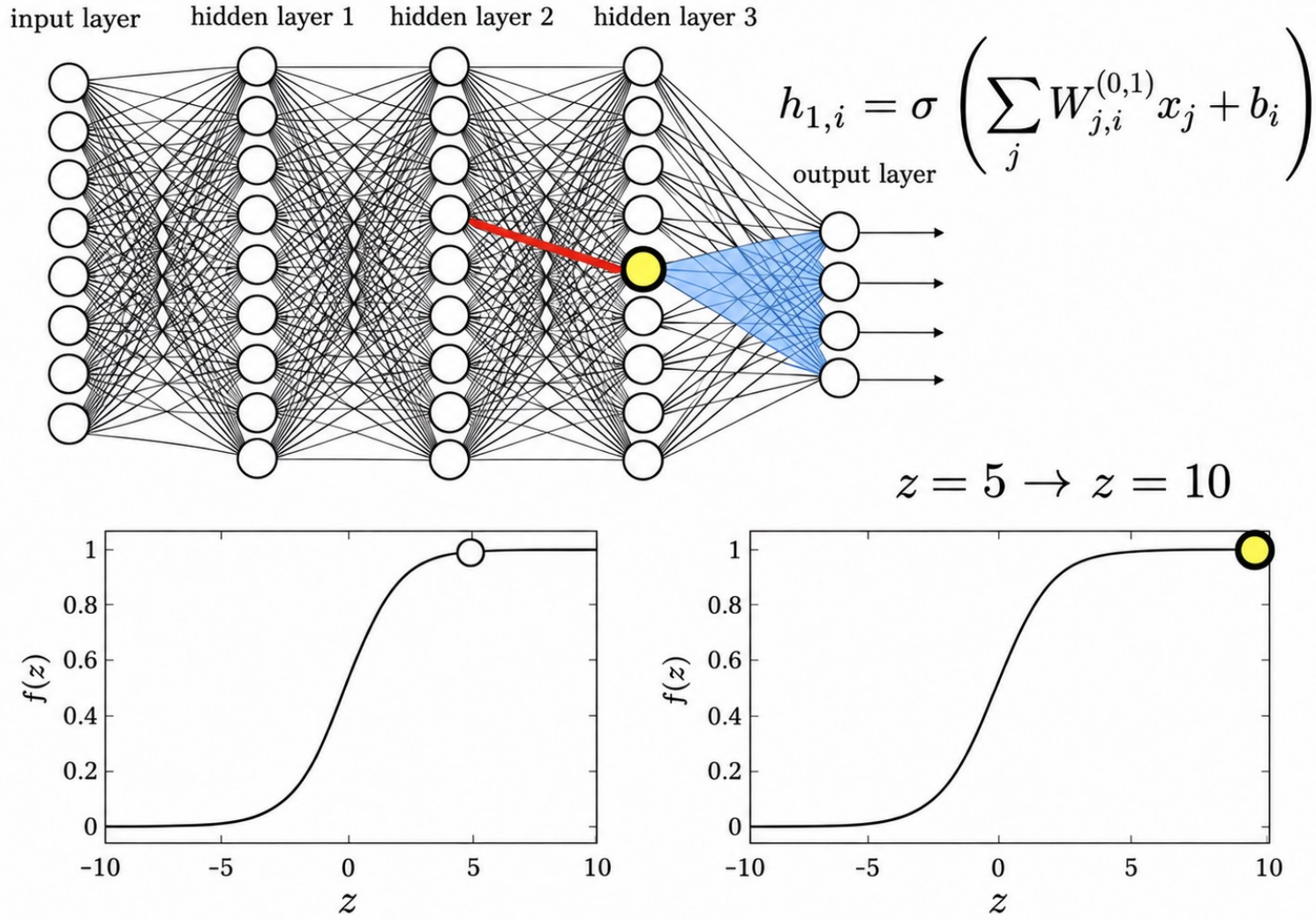
$$h_{1,i} = \sigma\left(\sum_j W_{j,i}^{(0,1)} x_j + b_i\right)$$

$$z = -2 \rightarrow z = 2$$

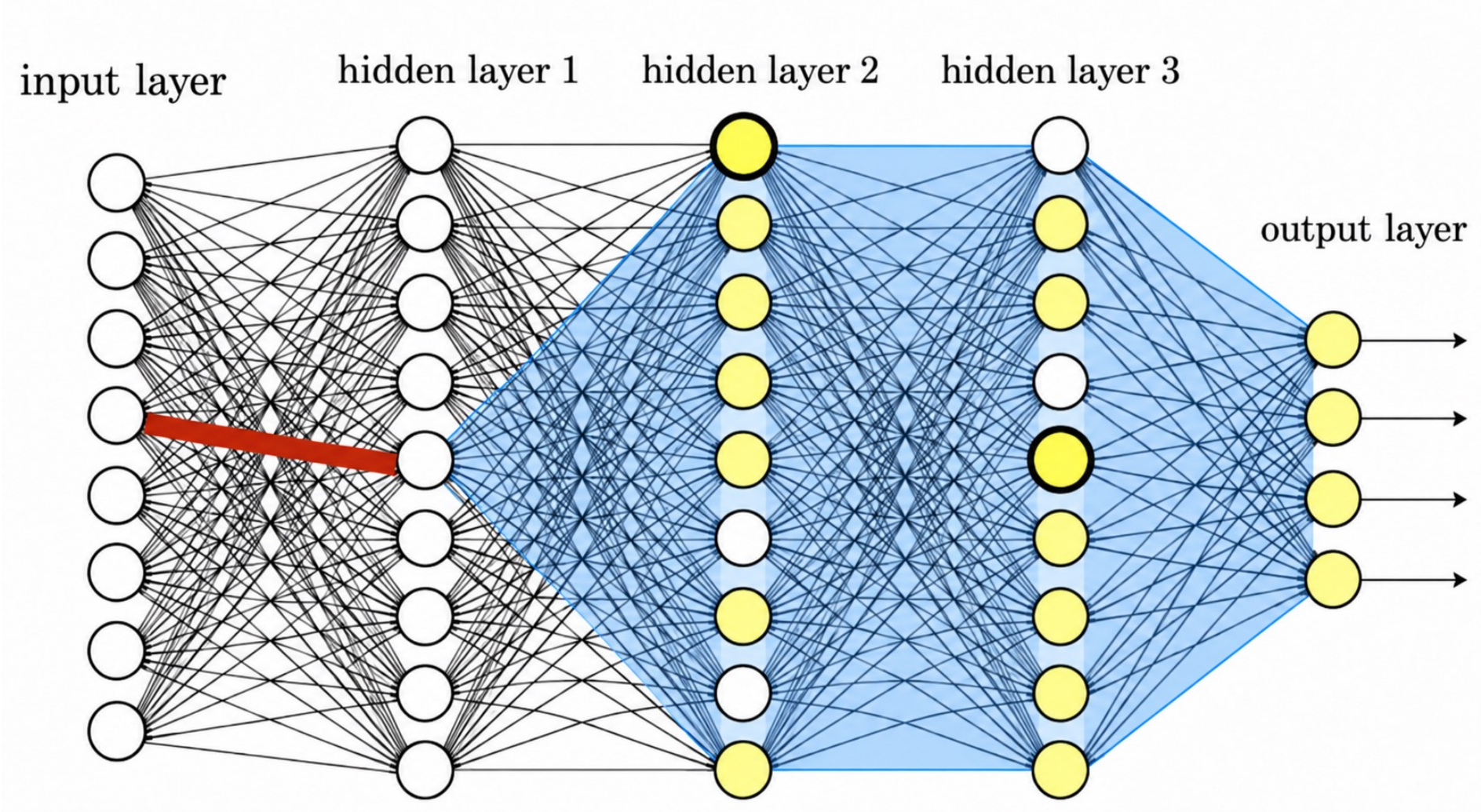




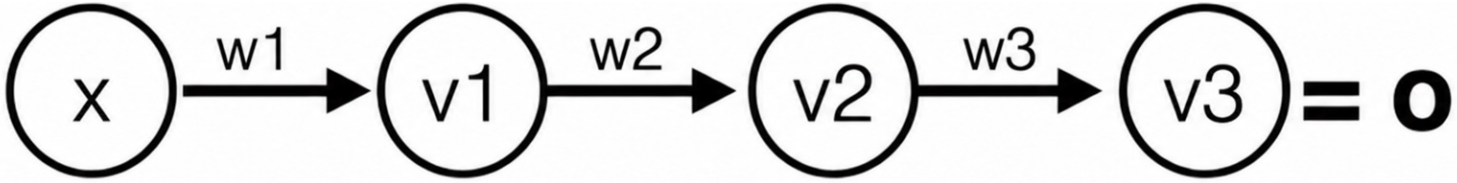
梯度消失的问题


input layer
hidden layer 1
hidden layer 2
hidden layer 3
output layer
$h_{1,i} = \sigma\left(\sum_j W_{j,i}^{(0,1)} x_j + b_i\right)$
$z = 5 \rightarrow z = 10$
f(z)
z
1
0.8
0.6
0.4
0.2
0
-10
-5
0
5
10

梯度消失的问题


input layer
hidden layer 1
hidden layer 2
hidden layer 3
output layer

梯度消失问题

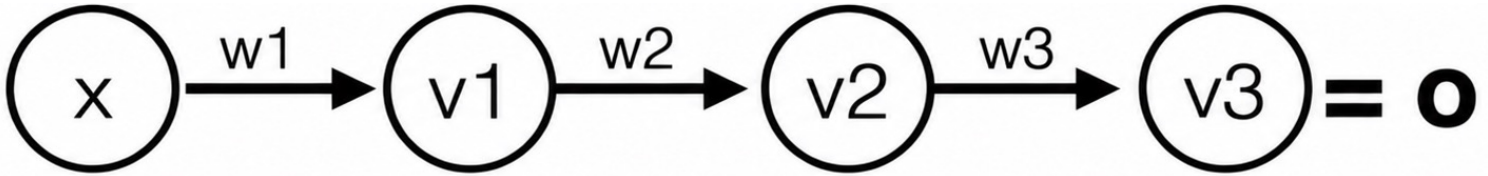


$$\frac{\partial v_3}{\partial w_3} = \frac{\partial v_3}{\partial z_3}\frac{\partial z_3}{\partial w_3} = \sigma'(z_3)v_2$$

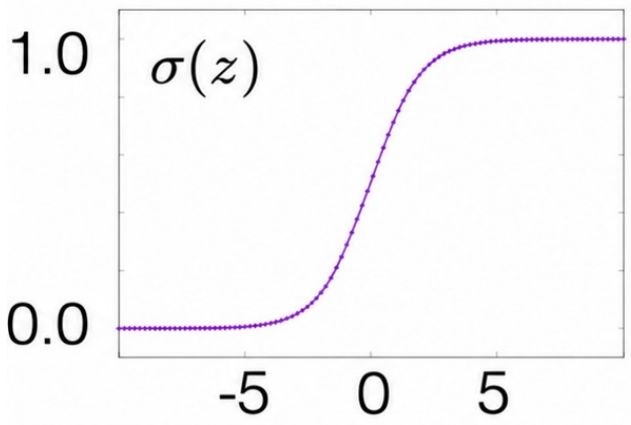
$$\frac{\partial v_3}{\partial w_2} = \frac{\partial v_3}{\partial z_3}\frac{\partial z_3}{\partial w_2} = \sigma'(z_3)w_3\frac{\partial v_2}{\partial w_2} = \sigma'(z_3)w_3\sigma'(z_2)v_1$$

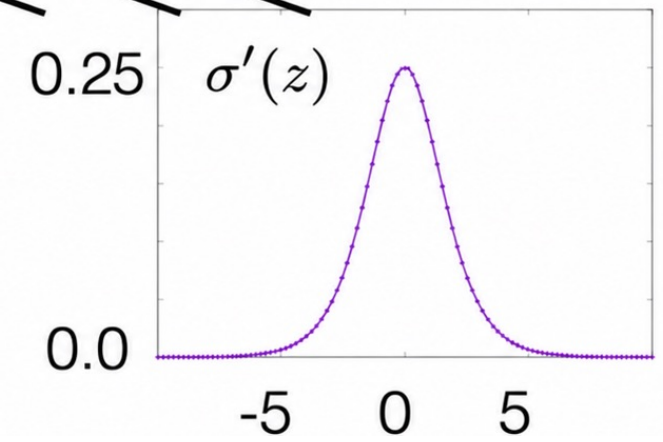
$$\begin{aligned}\frac{\partial v_3}{\partial w_1} &= \frac{\partial v_3}{\partial z_3}\frac{\partial z_3}{\partial w_1} = \sigma'(z_3)w_3\frac{\partial v_2}{\partial w_1} = \sigma'(z_3)w_3\sigma'(z_2)w_2\frac{\partial v_1}{\partial w_1} \\ &= \sigma'(z_3)w_3\sigma'(z_2)w_2\sigma'(z_1)x \\ &= \sigma'(z_3)\sigma'(z_2)\sigma'(z_1)w_3w_2x\end{aligned}$$

梯度消失问题



$$\frac{\partial v_3}{\partial w_1} = \frac{\partial v_3}{\partial z_3}\frac{\partial z_3}{\partial w_1} = \sigma'(z_3)w_3\frac{\partial v_2}{\partial w_1} = \sigma'(z_3)w_3\sigma'(z_2)w_2\frac{\partial v_1}{\partial w_1}$$
$$= \sigma'(z_3)w_3\sigma'(z_2)w_2\sigma'(z_1)x$$
$$= \sigma'(z_3)\sigma'(z_2)\sigma'(z_1)w_3w_2x$$



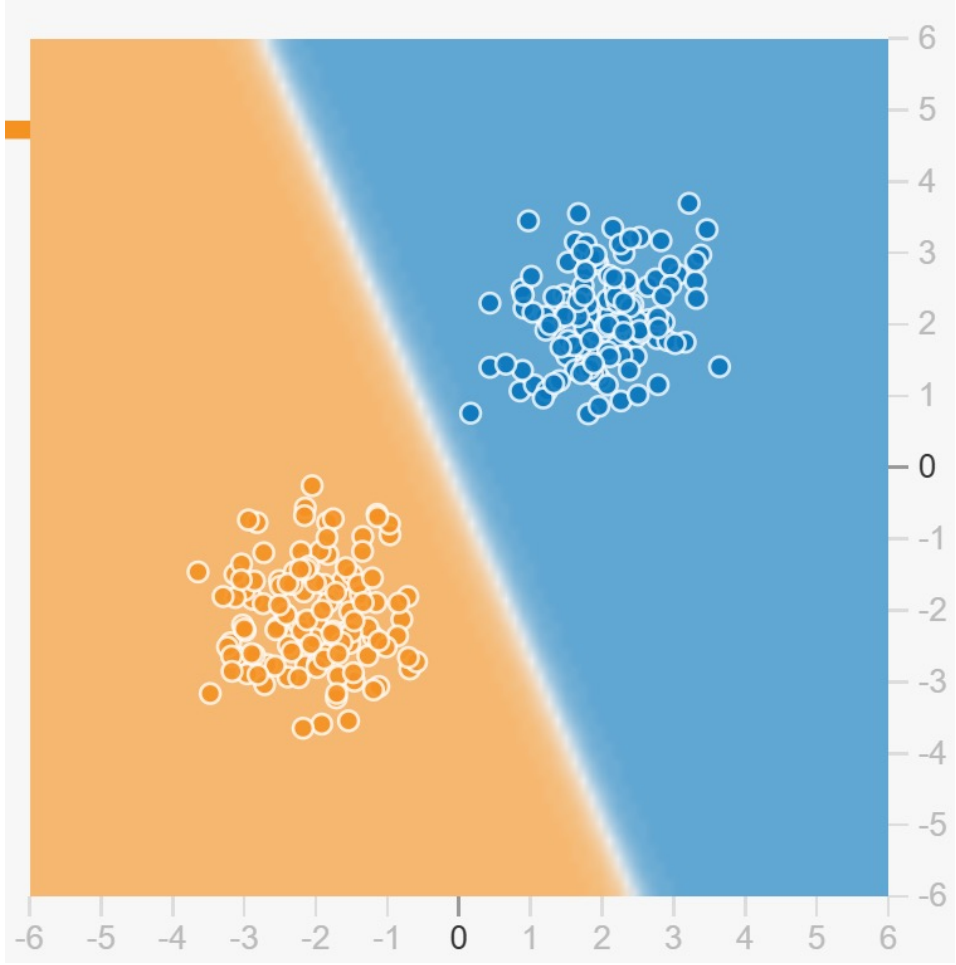


提纲

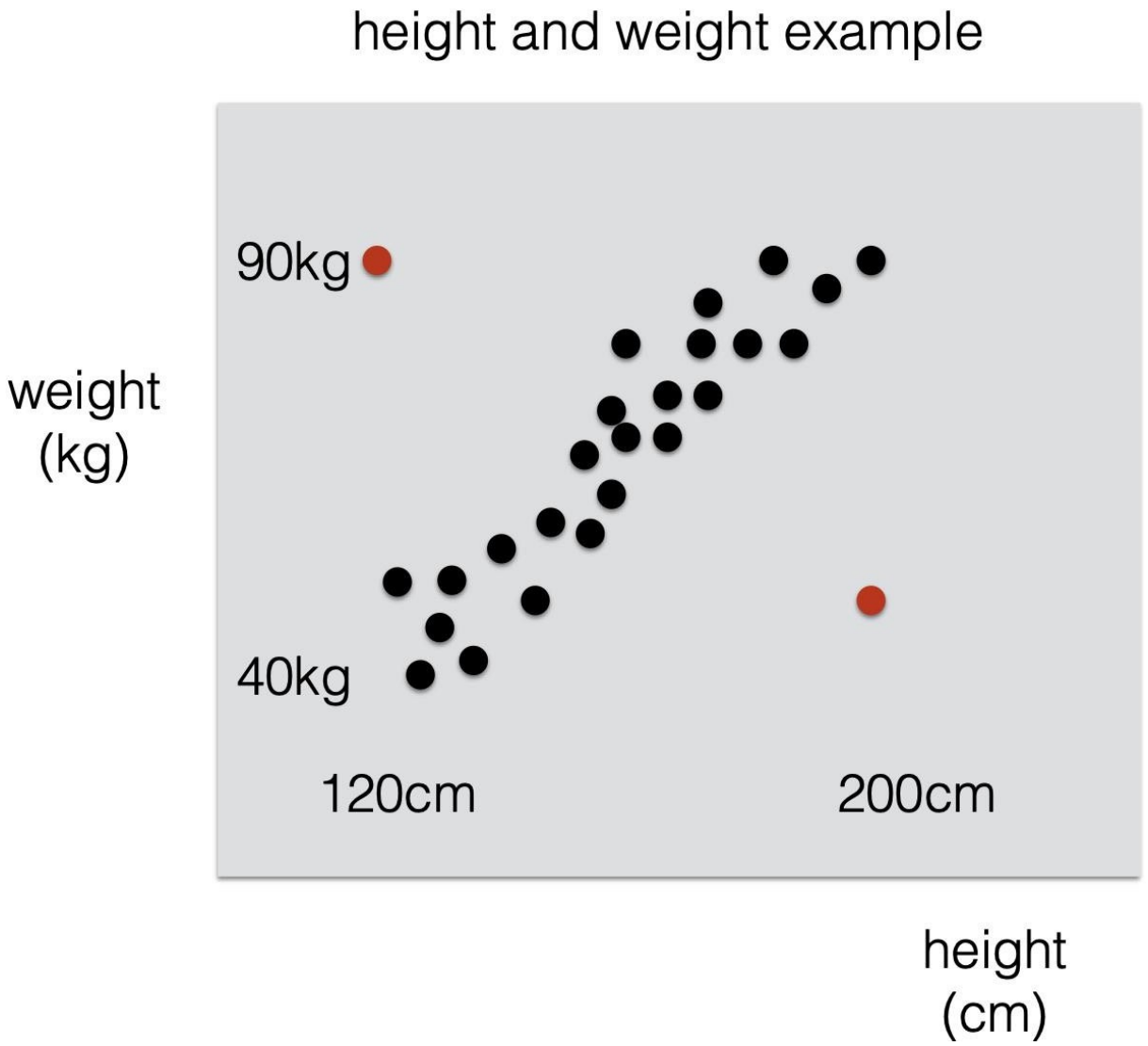
- 计算图与反向传播
- 向量与张量的反向传播
- 神经网络训练中的挑战
- **数据流形与模型泛化**

数据空间 vs 数据流形

- 这个数据集有什么问题?

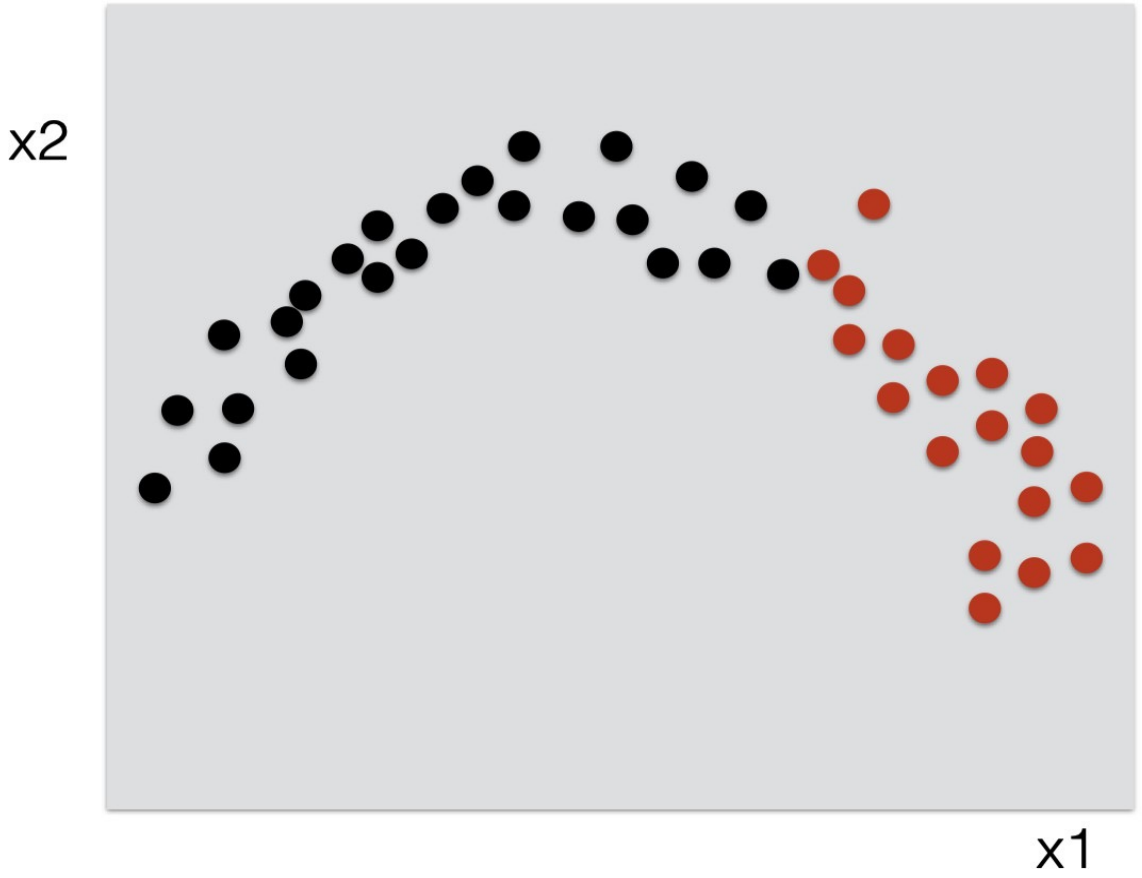


数据空间 vs 数据流形


height and weight example
90kg
weight
(kg)
40kg
120cm
200cm
height
(cm)

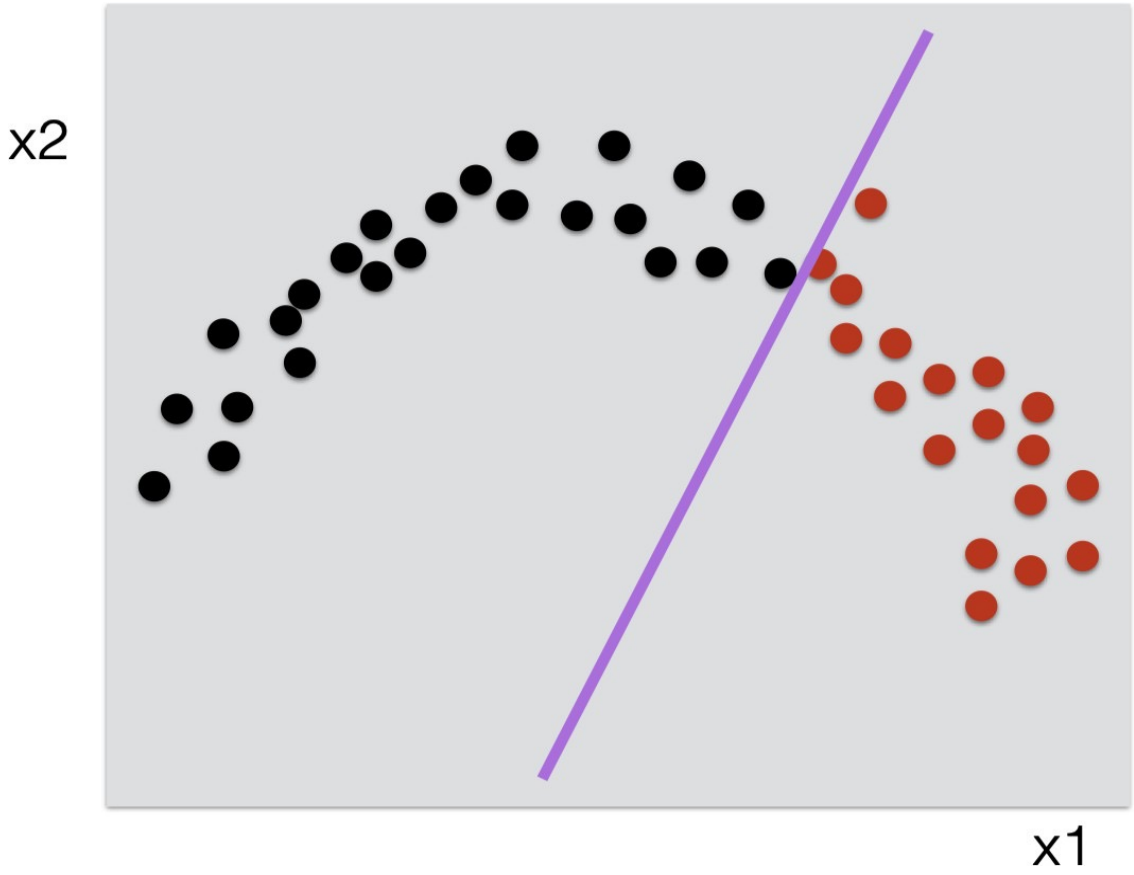
数据空间 vs 数据流形

- 如何确定决策边界?



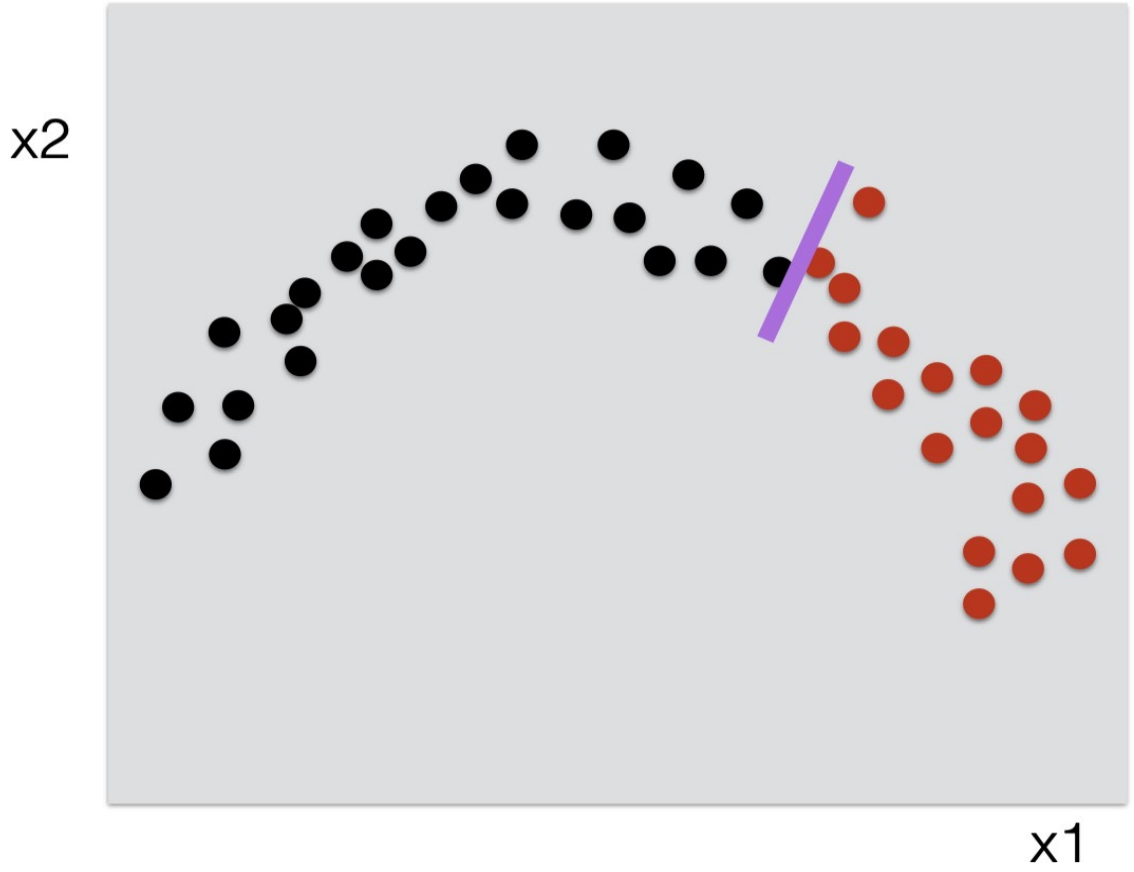
数据空间 vs 数据流形

- 直线边界



数据空间 vs 数据流形

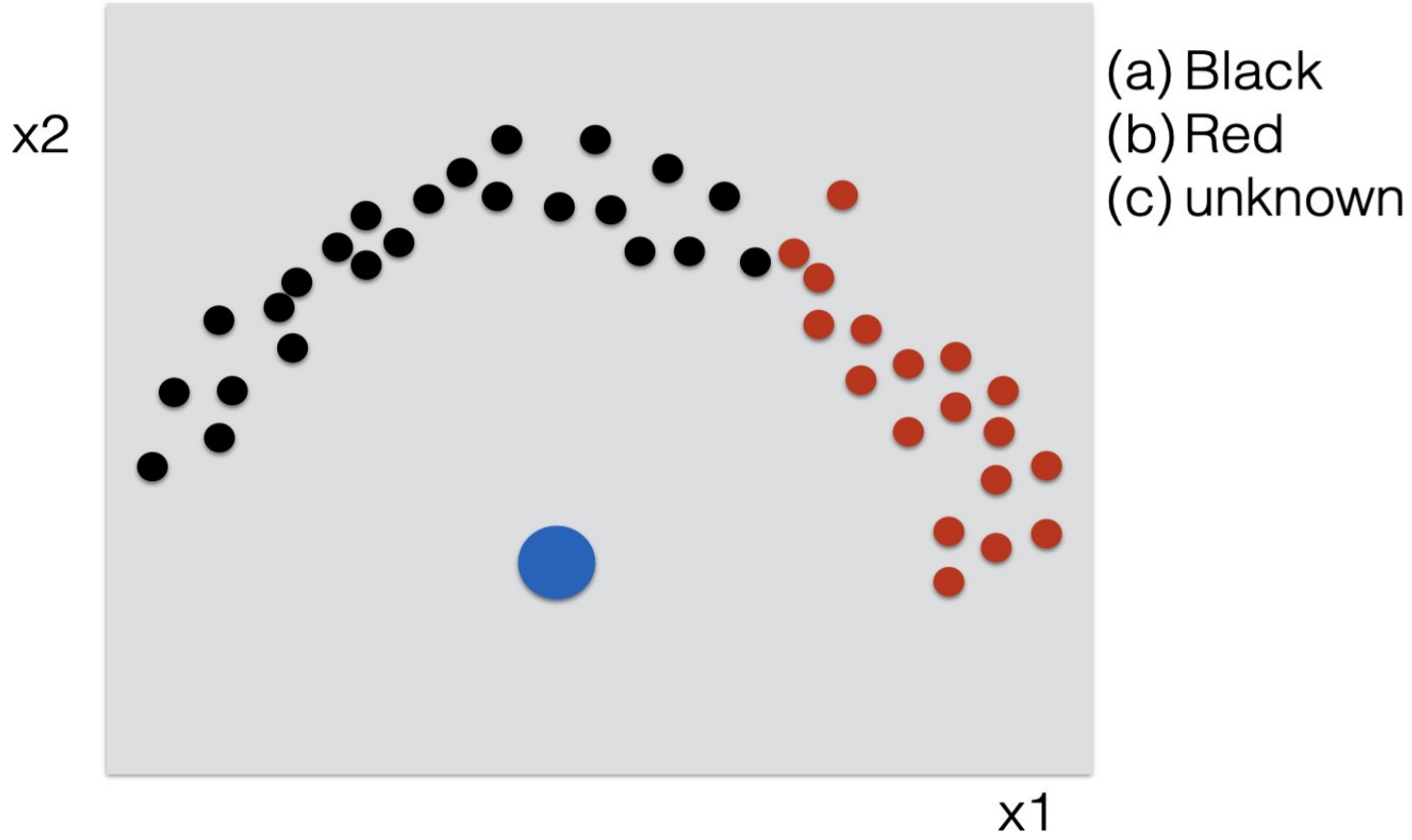
- 这是一个更好的边界?



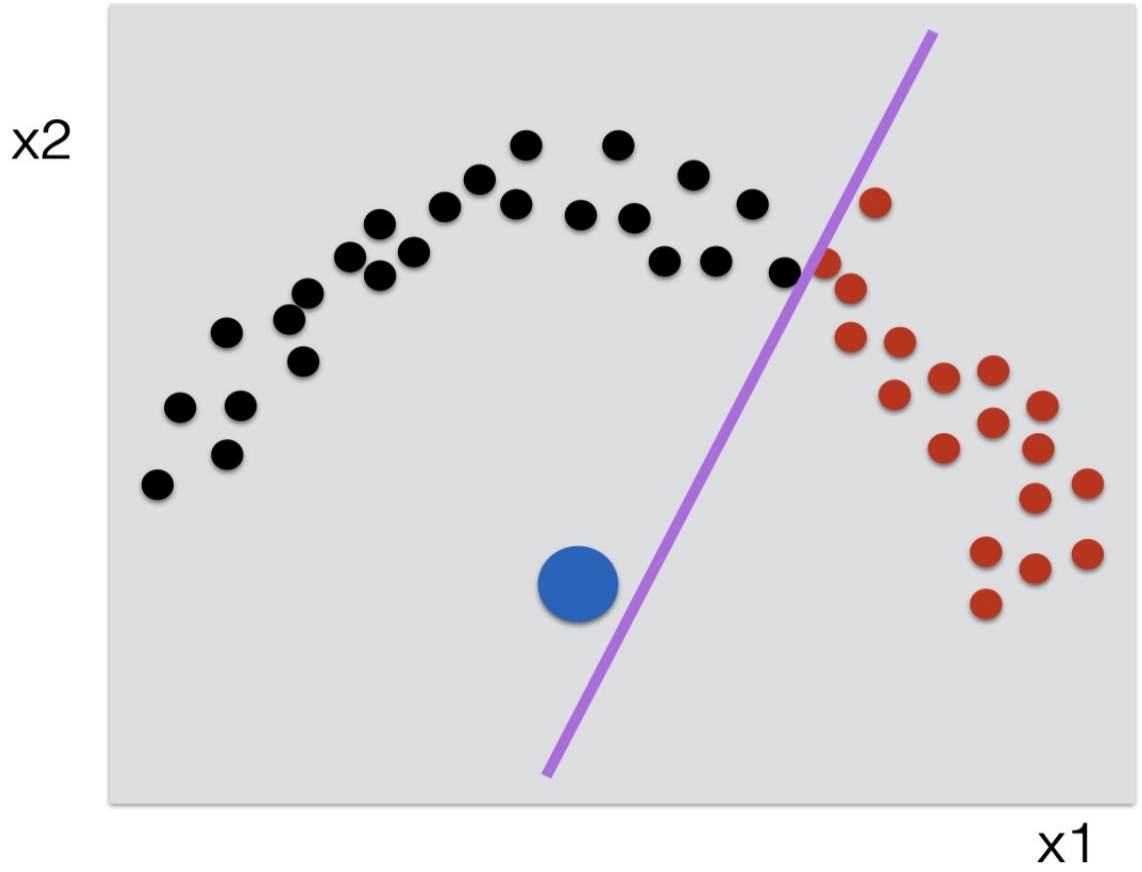
局部边界

数据空间 vs 数据流形

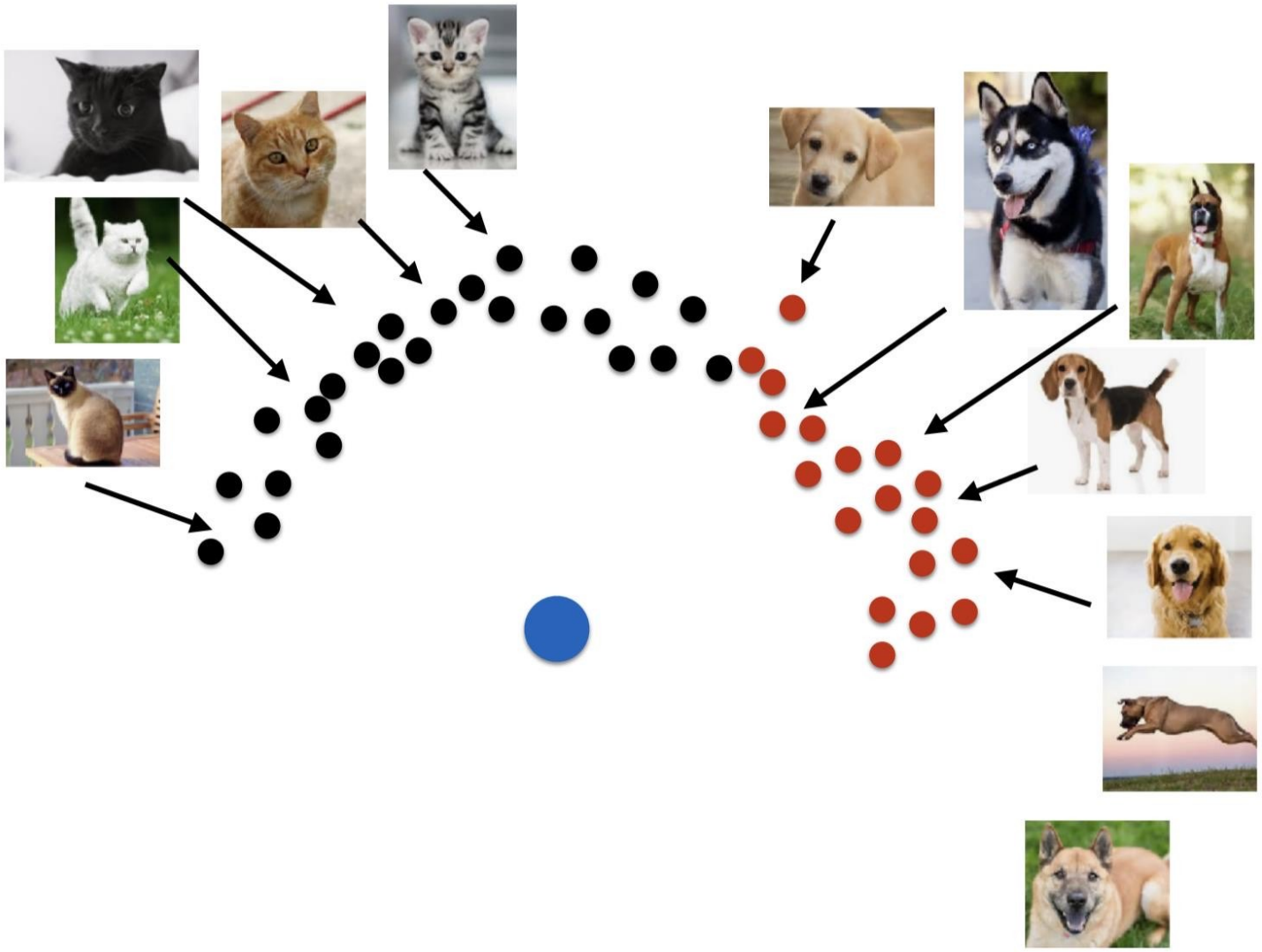
- 蓝色点应该属于哪一类?



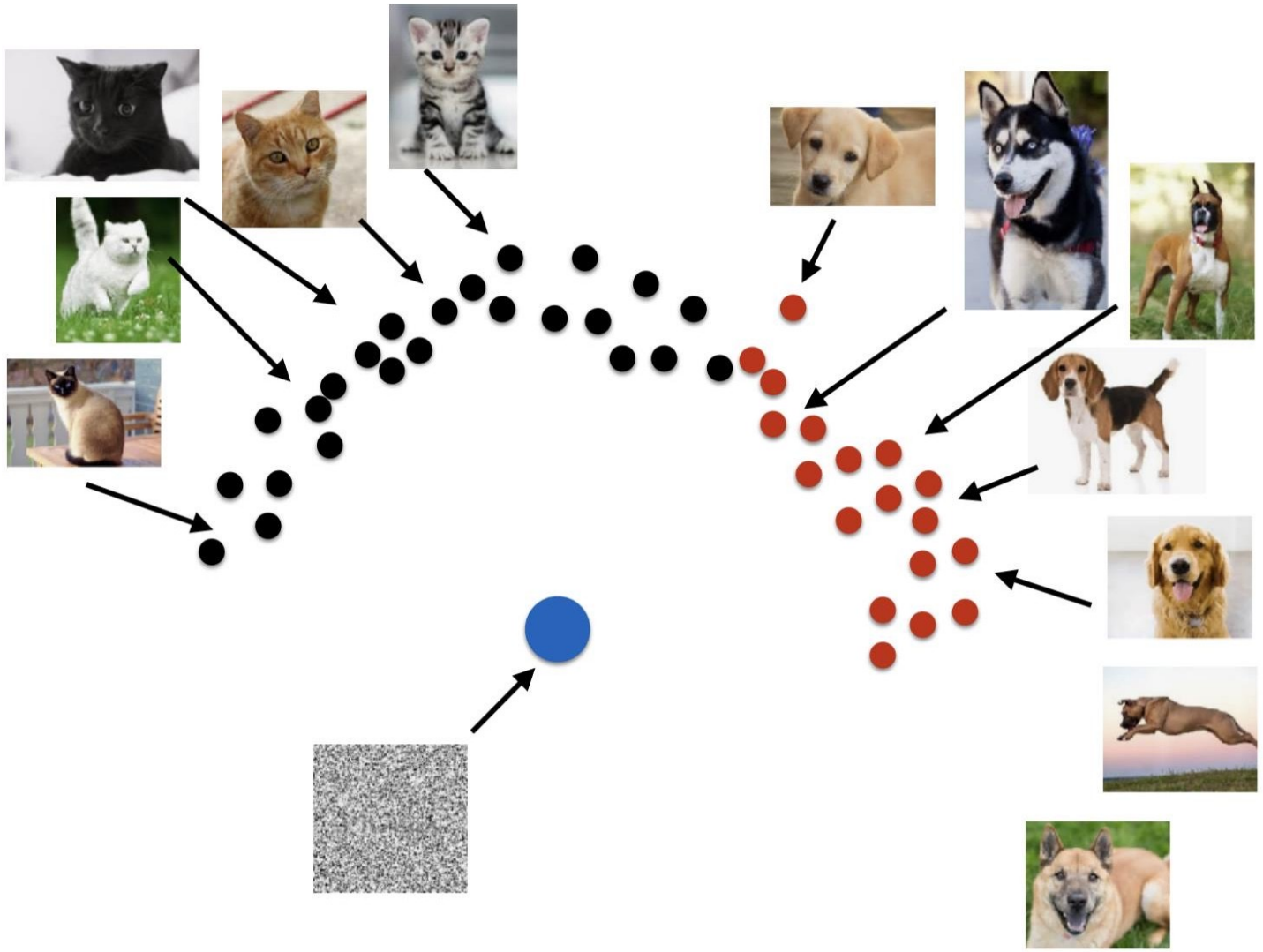
数据空间 vs 数据流形



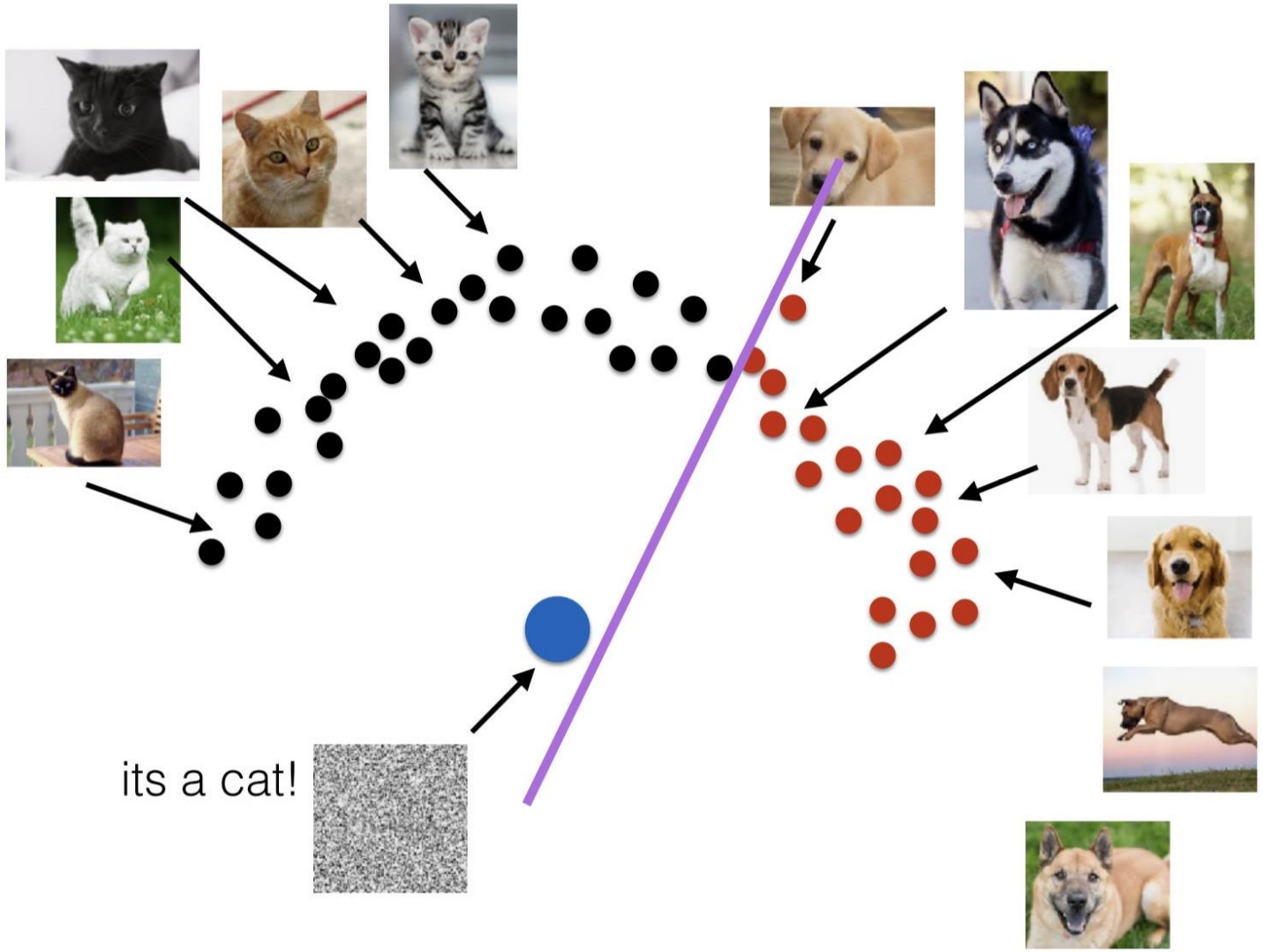
数据空间 vs 数据流形



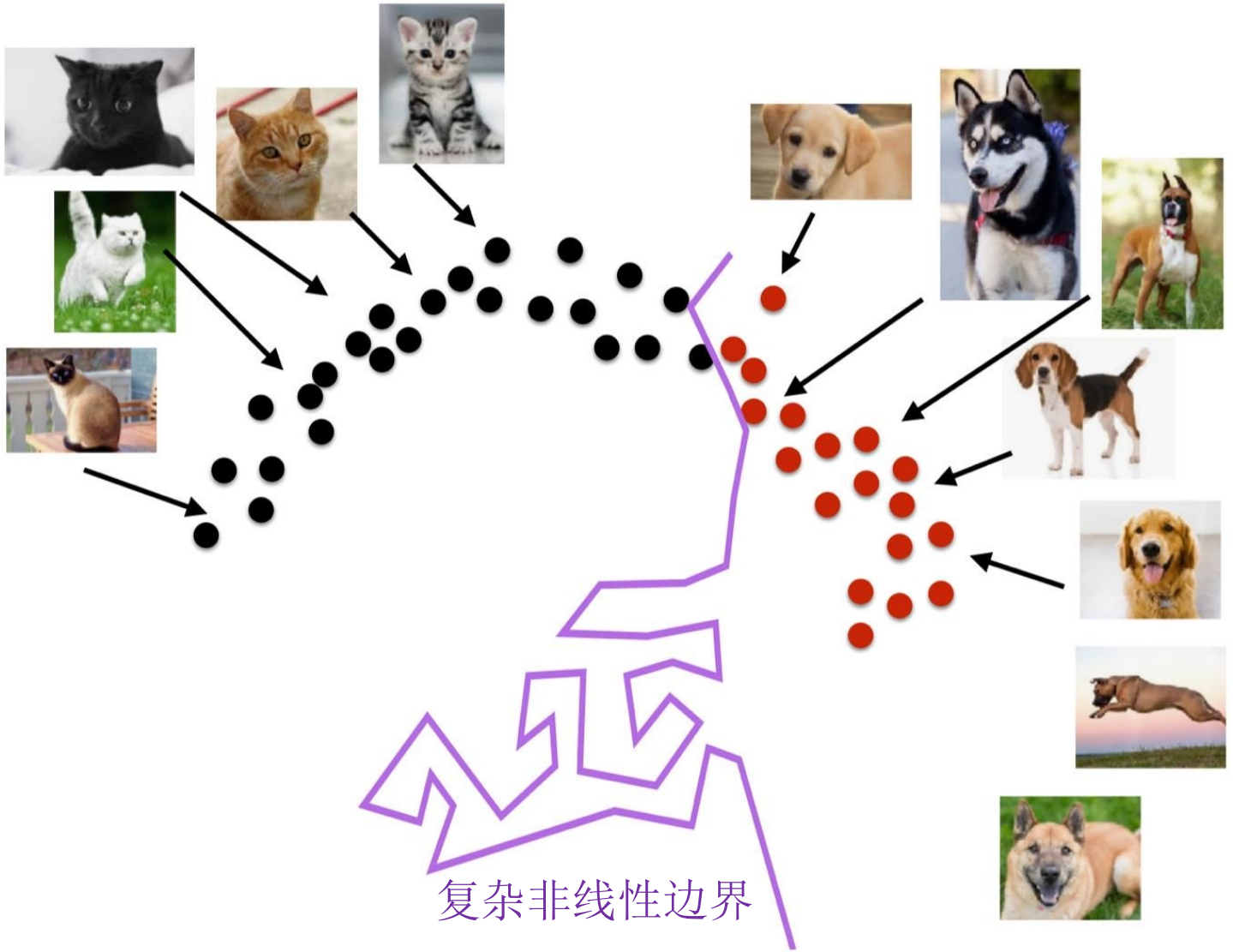
数据空间 vs 数据流形



数据空间 vs 数据流形

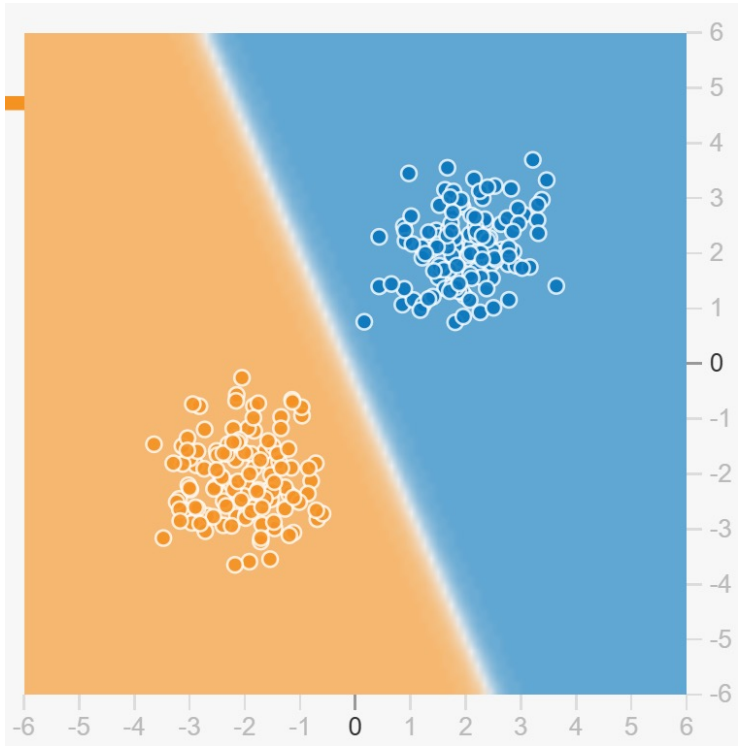


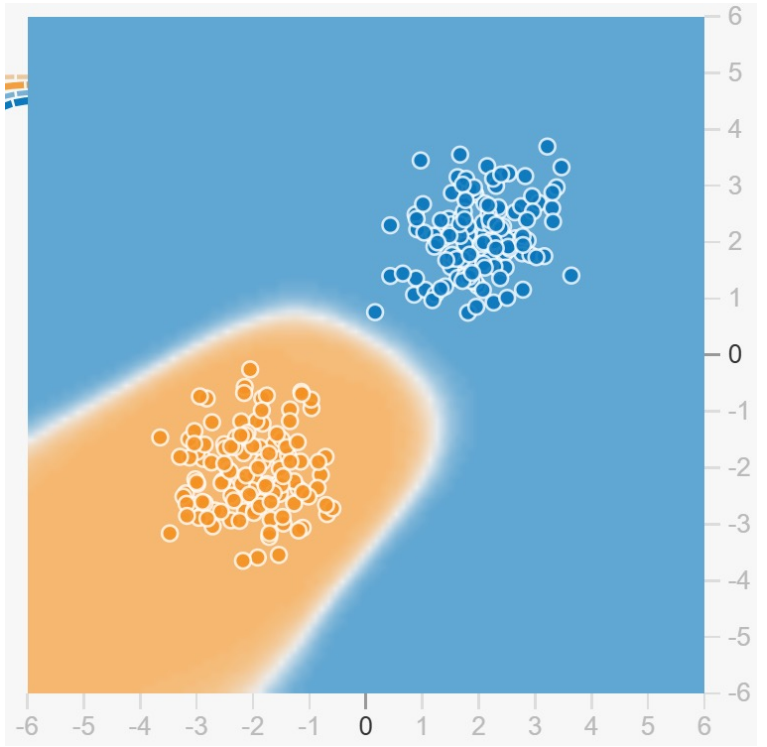
数据空间 vs 数据流形



数据空间 vs 数据流形

- 如何设计网络结构?
  - 更多层就更好么?

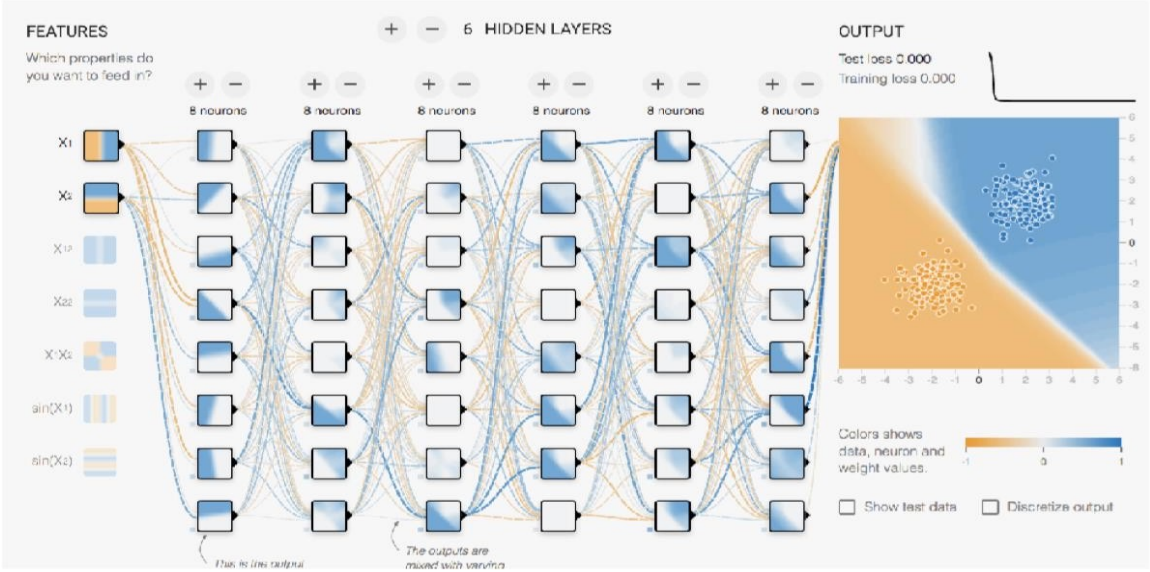




数据空间 vs 数据流形

- **如何在复杂的数据流形上学习合适的决策边界**，而不仅仅是对空间进行硬切割。这也引出深度学习中的关键问题——**泛化能力**

try this network, run with
(1) relu
(2) tanh
(3) sigmoid
(4) linear



https://playground.tensorflow.org

Questions



2026秋AI&ML
群号: 1109611031